Chapter 1: The Performance Gap

Part I: Foundations

> "In theory, theory and practice are the same. In practice, they are not." — Attributed to various computer scientists

The Mystery

It was 2:00 AM, and I was staring at profiling data that made no sense.

I was working on a bootloader for a RISC-V SoC, and we had a performance problem. The bootloader needed to look up device configurations from a table—about 500 entries, each with a 32-bit device ID and a pointer to configuration data. Simple enough.

My colleague had implemented it using a hash table. "O(1) lookup," he said confidently. "Can't beat that."

But the bootloader was slow. Unacceptably slow. We were missing our 100ms boot time target by a factor of three.

I tried the obvious optimization: replacing the hash table with a binary search on a sorted array. Binary search is O(log n), which is theoretically worse than O(1). The textbooks say so. My algorithms professor would have frowned.

The result? **The bootloader was now 40% faster.**

How could O(log n) beat O(1)? What was going on?

The Investigation

I fired up `perf`, Linux's performance profiling tool, and ran both implementations:

```
# Hash table version
$ perf stat -e cache-references,cache-misses ./bootloader_hash
  Performance counter stats:
    1,247,832  cache-references
      892,441  cache-misses  (71.5% miss rate)

# Binary search version
$ perf stat -e cache-references,cache-misses ./bootloader_binsearch
  Performance counter stats:
      423,156  cache-references
       89,234  cache-misses  (21.1% miss rate)
```

There it was. The hash table had a **71.5% cache miss rate**. The binary search had only **21.1%**.

Each cache miss costs roughly 100 CPU cycles on this system. The hash table was spending most of its time waiting for memory.

The O(1) hash table was doing fewer *operations*, but each operation was *expensive*. The O(log n) binary search was doing more operations, but each operation was *cheap*.

The hardware had overruled the algorithm.

Why This Matters

This book is about that gap—the gap between what the textbooks teach and what actually happens when your code runs on real silicon.

Traditional data structures courses teach you to think in terms of Big-O complexity: - Arrays: O(1) access, O(n) insertion - Linked lists: O(1) insertion, O(n) access - Hash tables: O(1) average case - Binary search trees: O(log n) operations

These are useful abstractions. They help us reason about algorithms at scale. But they're *incomplete*.

They assume all memory accesses cost the same. They assume operations happen in isolation. They assume an idealized computer that doesn't exist.

Real computers have: - **Memory hierarchies**: Registers, L1 cache, L2 cache, L3 cache, DRAM - **Latency gaps**: 1 cycle vs 100+ cycles - **Cache lines**: 64 bytes fetched together - **Prefetchers**: Hardware that guesses what you'll need next - **Limited bandwidth**: You can't fetch everything at once

And if you're working on embedded systems, you have even more constraints: - **Tiny caches**: 8KB to 64KB is common - **No L3 cache**: Many MCUs stop at L1 or L2 - **Slow memory**: DRAM might be 100MHz, not 3GHz - **Real-time requirements**: Worst-case matters, not average-case

The Real Performance Model

Here's a better mental model for modern computers:

Time = Operations × (Computation Cost + Memory Cost)

Where: - **Computation Cost**: The actual ALU operations (usually cheap) - **Memory Cost**: Cache misses, DRAM accesses (usually expensive)

For many algorithms, Memory Cost dominates.

Let's quantify this with real numbers from a typical embedded RISC-V system:

Operation	Latency	Relative Cost
Register access	1 cycle	1×
L1 cache hit	3-4 cycles	3×

Operation	Latency	Relative Cost
L2 cache hit	12-15 cycles	12×
L3 cache hit	40-50 cycles	40×
DRAM access	100-200 cycles	100×

A single cache miss can cost as much as **100 register operations**.

This means: - An O(n) algorithm with good cache behavior can beat an O(log n) algorithm with poor cache behavior - An O(1) hash table can lose to an O(log n) binary search - A "slow" algorithm that fits in cache can beat a "fast" algorithm that doesn't

Our First Benchmark: Array vs Linked List

Let's make this concrete with a simple experiment. We'll compare two ways to sum 100,000 integers:

1. **Array**: Contiguous memory, perfect for cache
2. **Linked list**: Scattered memory, cache nightmare

Both are O(n). The textbooks say they should perform similarly. Let's see what really happens.

Here's the array version:

```
// Array: contiguous memory
int array[100000];
for (int i = 0; i < 100000; i++) {
    array[i] = i;
}

// Sum all elements
long long sum = 0;
for (int i = 0; i < 100000; i++) {
    sum += array[i];
}
```

And the linked list version:

```
// Linked list: scattered memory
typedef struct node {
    int value;
    struct node *next;
} node_t;

node_t *head = NULL;
for (int i = 0; i < 100000; i++) {
    node_t *node = malloc(sizeof(node_t));
```

```
    node->value = i;
    node->next = head;
    head = node;
}

// Sum all elements
long long sum = 0;
node_t *curr = head;
while (curr) {
    sum += curr->value;
    curr = curr->next;
}
```

Using our benchmark framework (which we'll explore in detail in Chapter 3), here are the results:

```
=== Array Sequential Sum ===
Mean time:      70,147 ns
Median time:    71,724 ns
Total cycles:   17,557,410

=== Linked List Sequential Sum ===
Mean time:      179,169 ns
Median time:    160,527 ns
Total cycles:   44,740,656

Array is 2.55× faster than Linked List
```

Same algorithm (sequential sum), same O(n) complexity, but the array is **2.5× faster**.

Why? Let's look at the cache behavior:

Array access pattern:

```
Memory:  [0][1][2][3][4][5][6][7][8][9]...
         ↑  ↑  ↑  ↑  ↑  ↑  ↑  ↑  ↑  ↑
Access:  Sequential, predictable
Cache:   Fetch 64 bytes (16 ints) at once
Result:  ~94% cache hit rate
```

Linked list access pattern:

```
Memory:  [node] ... [node] ... [node] ... [node]
         ↑          ↑          ↑          ↑
Access:  Random, unpredictable (follows pointers)
Cache:   Each node likely in different cache line
Result:  ~70% cache miss rate
```

The array benefits from **spatial locality**—when you access `array[0]`, the

CPU fetches an entire cache line (64 bytes), which includes `array[0]` through `array[15]`. The next 15 accesses are free.

The linked list suffers from **pointer chasing**—each node is allocated separately by `malloc()`, scattered randomly in memory. Each access likely requires a new cache line fetch.

The Memory Hierarchy

To understand why cache matters so much, we need to understand the memory hierarchy.

Modern computers are not the simple "CPU + RAM" model from introductory courses. They're more like this:

```
CPU Core
  ↓ 1 cycle
Registers (32-64 registers, ~256 bytes)
  ↓ 3-4 cycles
L1 Cache (32-64 KB, split I/D)
  ↓ 12-15 cycles
L2 Cache (256 KB - 1 MB, unified)
  ↓ 40-50 cycles
L3 Cache (4-32 MB, shared) [not on all systems]
  ↓ 100-200 cycles
DRAM (GB scale)
  ↓ 10,000+ cycles
SSD/Flash
```

Each level is: - **Faster** but **smaller** than the level below - **More expensive** per byte - **Closer** to the CPU

The speed gap is enormous. On a 1 GHz RISC-V processor: - L1 cache: 3-4 nanoseconds - DRAM: 100-200 nanoseconds - That's a **50× difference**

For comparison, if L1 cache access was 1 second, DRAM access would be **50 seconds**. That's the difference between a quick response and going to make coffee.

Cache Lines: The Fundamental Unit

Here's a critical insight: **CPUs don't fetch individual bytes. They fetch cache lines.**

A cache line is typically 64 bytes. When you access a single byte, the CPU fetches the entire 64-byte block containing that byte.

This has profound implications:

Good: If you access nearby data, it's already in cache (spatial locality)

```
// Excellent: sequential access
for (int i = 0; i < n; i++) {
    sum += array[i];  // Next element likely in same cache line
}
```

Bad: If you access scattered data, you waste 63 bytes per fetch

```
// Terrible: random access
for (int i = 0; i < n; i++) {
    sum += array[random()];  // Each access likely misses cache
}
```

Worse: If your data structure has poor layout, you pay for data you don't use

```
// Linked list node: 16 bytes (4-byte value + 8-byte pointer + padding)
// Cache line: 64 bytes
// Waste: 48 bytes (75% of cache line unused!)
```

Prefetching: Hardware Tries to Help

Modern CPUs have hardware prefetchers that try to predict what you'll access next. They're good at detecting simple patterns:

Sequential access: Prefetcher loves this

```
for (int i = 0; i < n; i++) {
    process(array[i]);  // Prefetcher: "I see a pattern! Fetch ahead!"
}
```

Strided access: Prefetcher can handle this

```
for (int i = 0; i < n; i += 2) {
    process(array[i]);  // Prefetcher: "Stride of 2, got it!"
}
```

Pointer chasing: Prefetcher gives up

```
while (node) {
    process(node->value);
    node = node->next;  // Prefetcher: "No idea what's next..."
}
```

This is why linked lists are so slow—the prefetcher can't help. Each pointer dereference is a surprise.

Embedded Systems: Even Harsher Constraints

If you're working on embedded systems, the situation is more extreme:

Typical embedded RISC-V MCU: - L1 cache: 16-32 KB (vs 32-64 KB on desktop) - L2 cache: 128-256 KB (vs 256 KB - 1 MB on desktop) - L3 cache: None (vs 4-32 MB on desktop) - DRAM: 100 MHz (vs 3 GHz on desktop)

With a 16 KB L1 cache, your entire working set needs to fit in **16 KB** or you'll thrash the cache.

For comparison: - 100,000 integers (array): 400 KB → won't fit in L1 - 100,000 linked list nodes: 1.6 MB → won't even fit in L2

This is why embedded systems developers obsess over data structure size and layout. Every byte counts.

Real-Time Considerations

In embedded systems, we often care about **worst-case** performance, not average-case.

Consider a real-time control loop running at 1 kHz (1ms period): - Best case: All data in L1 cache → 50 microseconds - Worst case: All data in DRAM → 500 microseconds

If your algorithm has unpredictable cache behavior, you can't guarantee real-time deadlines.

This is why real-time systems often prefer: - **Static allocation**: Predictable memory layout - **Fixed-size data structures**: No dynamic resizing - **Simple algorithms**: Predictable cache behavior

Even if they're "slower" in average-case Big-O terms.

What You'll Learn in This Book

This book will teach you to think about data structures in terms of hardware reality:

Part I: Foundations - How memory hierarchy works (Chapter 2) - How to measure and profile performance (Chapter 3)

Part II: Basic Data Structures - Arrays: The cache-friendly foundation (Chapter 4) - Linked lists: When and how to use them (Chapter 5) - Stacks, queues, and ring buffers (Chapter 6) - Hash tables: Cache-conscious design (Chapter 7) - Dynamic arrays and memory management (Chapter 8)

Part III: Trees and Hierarchies - Binary search trees: Cache behavior (Chapter 9) - B-trees: Cache-conscious trees (Chapter 10) - Tries and radix trees (Chapter 11) - Heaps and priority queues (Chapter 12)

Part IV: Advanced Topics - Lock-free data structures (Chapter 13) - String processing (Chapter 14) - Graphs and networks (Chapter 15) - Probabilistic structures (Chapter 16)

Part V: Case Studies - Bootloader data structures (Chapter 17) - Device driver queues (Chapter 18) - Firmware memory management (Chapter 19)

Each chapter will include: - **Real-world examples** from embedded systems - **Benchmarks** showing actual performance - **Cache analysis** with profiling tools - **Design guidelines** for your own code

Prerequisites and Setup

To get the most out of this book, you should:

Know: - C programming (pointers, structs, memory management) - Basic data structures (arrays, linked lists, trees) - Basic algorithms (sorting, searching)

Have: - Linux system (Ubuntu/Debian recommended) - GCC compiler - Basic command-line skills

Optional but helpful: - RISC-V development board or QEMU - Experience with embedded systems - Familiarity with assembly language

All code examples and benchmarks are available at: `github.com/dannyjiang/ds-in-practice` (placeholder)

The Road Ahead

In the next chapter, we'll dive deep into the memory hierarchy. You'll learn: - How caches work at the hardware level - What cache lines, sets, and ways mean - How to predict cache behavior - How to measure cache performance

Then in Chapter 3, we'll build a complete benchmarking framework—the same one used for all measurements in this book.

By the end of Part I, you'll have the tools and knowledge to analyze any data structure's real-world performance.

Let's get started.

Summary

The mystery from 2:00 AM was solved. The O(log n) binary search beat the O(1) hash table by 40% because cache behavior mattered more than algorithmic complexity. The hash table's 71.5% cache miss rate versus the binary search's 21.1% explained everything. The hardware had overruled the algorithm.

Key insights: 1. **Big-O complexity is necessary but not sufficient** for understanding real-world performance 2. **Memory hierarchy dominates** modern computer performance 3. **Cache misses cost 100× more** than cache hits 4. **Spatial locality matters**: Sequential access beats random access 5. **Embedded systems have harsher constraints**: Smaller caches, slower memory 6. **Real-time systems need predictable performance**: Worst-case matters

The Performance Gap: - Textbook: O(1) hash table beats O(log n) binary search - Reality: Cache behavior can reverse this - Lesson: Measure, don't assume

Next Chapter: We'll explore the memory hierarchy in detail and learn how caches work at the hardware level.

Chapter 2: Memory Hierarchy

Part I: Foundations

"Memory is the new disk, disk is the new tape." — Jim Gray

The 100-Cycle Problem

In Chapter 1, we saw that cache misses cost 100-200 cycles while cache hits cost only 1-4 cycles. This isn't a minor detail—it's the single most important factor in modern performance.

Let me show you why.

I was optimizing a device driver for a RISC-V embedded system. The driver needed to process packets from a network interface, and we were dropping packets under load. The CPU was running at 1 GHz, and each packet required about 500 instructions to process. Simple math:

```
500 instructions ÷ 1 GHz = 500 nanoseconds per packet
```

At 500 ns per packet, we should handle 2 million packets per second. But we were only managing 200,000 packets per second—10× slower than expected.

The profiler told the story:

```
$ perf stat -e cycles,instructions,cache-misses ./driver_test
  Performance counter stats:
    5,000,000 cycles
      500,000 instructions
       45,000 cache-misses
```

Wait. 500,000 instructions should take 500,000 cycles (at 1 IPC). But we're seeing 5,000,000 cycles. Where did the extra 4.5 million cycles go?

Cache misses: 45,000 misses × 100 cycles = 4,500,000 cycles

The cache misses were dominating our execution time. The actual computation (500,000 cycles) was only 10% of the total time. The other 90% was waiting for memory.

This is the reality of modern computing: **memory is slow, and it's getting slower relative to CPU speed**.

The Memory Hierarchy

Modern computers don't have "memory"—they have a **hierarchy** of memories, each with different speeds and sizes:

Level	Type	Latency	Size
Registers	32 registers	1 cycle	~128 B
L1 Cache	Split I/D	3-4 cycles	32-64 KB
L2 Cache	Unified	12-15 cycles	256-512 KB
L3 Cache (if present)	Shared	40-50 cycles	2-32 MB
DRAM	Main memory	100-200 cycles	GB-TB

Key observations:

1. **Speed decreases** as you go down (1 → 200 cycles)
2. **Size increases** as you go down (128 B → GB)
3. **The gap is huge**: DRAM is 100-200× slower than L1

On embedded systems, the hierarchy is often simpler:

Typical MCU (e.g., RISC-V RV32IMC @ 100 MHz):

Level	Type	Latency	Size
Registers	32 registers	1 cycle	128 B
L1 I-Cache	Instruction	1 cycle	16 KB
L1 D-Cache/SRAM	Data	1-2 cycles	8-32 KB
Flash	Code storage	~10 cycles	128 KB - 1 MB
External DRAM (optional)	Data (if present)	50-100 cycles	8-64 MB

Embedded differences: - Smaller caches (8-64 KB vs 256 KB-32 MB) - Often no L2/L3 cache - Flash memory instead of DRAM for code - Tighter memory budgets

Cache Lines: The Fundamental Unit

Here's the crucial insight: **caches don't fetch individual bytes—they fetch cache lines**.

A cache line is typically **64 bytes** on modern processors (both desktop and embedded). When you access a single byte, the hardware fetches the entire 64-byte block containing that byte.

Example: Accessing a single integer

```
int x = array[0];  // Access 4 bytes at address 0x1000
```

What actually happens:

```
CPU requests: 4 bytes at 0x1000
Cache fetches: 64 bytes from 0x1000 to 0x103F
```

The cache line includes: - The requested integer (4 bytes) - The next 15 integers (60 bytes)

This is why **sequential access is fast**:

```
// Fast: All in the same cache line
for (int i = 0; i < 16; i++) {
    sum += array[i];  // First access: miss, next 15: hits
}
```

But **random access is slow**:

```
// Slow: Each access likely in different cache line
for (int i = 0; i < 16; i++) {
    sum += array[random_index[i]];  // Each access: likely miss
}
```

Cache Organization

Caches are organized into **sets** and **ways**. Understanding this helps explain cache conflicts.

Direct-mapped cache (1-way):

```
Address bits: [     Tag      |   Index   |      Offset      ]

               Identifies     Selects     Byte within
               cache line     set         cache line
```

Example: 32 KB cache, 64-byte lines, direct-mapped - Cache lines: 32 KB ÷ 64 B = 512 lines - Index bits: log (512) = 9 bits - Offset bits: log (64) = 6 bits - Tag bits: remaining bits (e.g., 32 - 9 - 6 = 17 bits for 32-bit address)

Problem with direct-mapped: Cache conflicts

```
int a[1024];  // At address 0x10000
int b[1024];  // At address 0x18000

// These two arrays map to the SAME cache sets!
// 0x10000 and 0x18000 differ only in bit 15
// Index uses bits 6-14, so they collide
```

Set-associative cache (N-way):

A 4-way set-associative cache has 4 “slots” per set:

```
Set 0: [Line 0] [Line 1] [Line 2] [Line 3]
Set 1: [Line 0] [Line 1] [Line 2] [Line 3]
...
```

When address maps to Set 0, it can go in any of the 4 slots. This reduces conflicts.

Typical configurations: - L1: 8-way set-associative (32-64 KB) - L2: 8-16-way set-associative (256-512 KB) - L3: 16-way set-associative (2-32 MB)

Embedded systems: - Often direct-mapped or 2-way (simpler hardware) - Smaller caches mean more conflicts

Spatial and Temporal Locality

Cache performance depends on two types of locality:

Spatial locality: Accessing nearby addresses

```
// Good spatial locality
for (int i = 0; i < n; i++) {
    sum += array[i];  // Sequential access
}

// Poor spatial locality
for (int i = 0; i < n; i++) {
    sum += array[random[i]];  // Random access
}
```

Temporal locality: Accessing the same address repeatedly

```
// Good temporal locality
int temp = array[0];
for (int i = 0; i < 1000; i++) {
    result += temp * i;  // Reuse 'temp'
}

// Poor temporal locality
for (int i = 0; i < 1000; i++) {
    result += array[i % 10] * i;  // Evicts before reuse
}
```

Cache-friendly code exploits both:

```
// Matrix multiplication: cache-friendly version
for (int i = 0; i < N; i++) {
    for (int k = 0; k < N; k++) {
        int r = A[i][k];
        for (int j = 0; j < N; j++) {
            C[i][j] += r * B[k][j];  // Good spatial locality on B
```

```
        }
    }
}
```

The Prefetcher

Modern CPUs have **hardware prefetchers** that predict memory access patterns and fetch data before you need it.

How the prefetcher works:

```
stateDiagram-v2
    [*] --> Idle: Power on
    Idle --> Detecting: Memory access
    Detecting --> Sequential: 2+ consecutive accesses
    Detecting --> Strided: Constant stride detected
    Detecting --> Idle: Random pattern

    Sequential --> Prefetching: Fetch ahead
    Strided --> Prefetching: Fetch ahead

    Prefetching --> Prefetching: Pattern continues
    Prefetching --> Idle: Pattern breaks

    note right of Sequential
        Access: A[0], A[1], A[2]
        Prefetch: A[3], A[4], A[5]
    end note

    note right of Strided
        Access: A[0], A[4], A[8]
        Prefetch: A[12], A[16], A[20]
    end note
```

Sequential prefetcher: Detects sequential access

```
// Prefetcher detects pattern and fetches ahead
for (int i = 0; i < n; i++) {
    sum += array[i];  // Prefetcher fetches array[i+1], array[i+2], ...
}
```

Stride prefetcher: Detects constant stride

```
// Prefetcher detects stride of 8 bytes
for (int i = 0; i < n; i++) {
    sum += array[i * 2];  // Accessing every other element
}
```

Prefetcher limitations:

1. **Doesn't help random access**:

```
for (int i = 0; i < n; i++) {
    sum += array[random[i]];  // Unpredictable, no prefetch
}
```

2. **Limited distance**: Typically 10-20 cache lines ahead

3. **Can be fooled**:

```
// Alternating pattern confuses prefetcher
for (int i = 0; i < n; i++) {
    if (i % 2 == 0)
        sum += array[i];
    else
        sum += other_array[i];
}
```

Embedded systems: Many MCUs have **no prefetcher** or simple sequential-only prefetchers. This makes sequential access even more critical.

Memory Bandwidth

Even with perfect cache behavior, you're limited by **memory bandwidth**.

Example calculation (desktop system): - DDR4-3200: 25.6 GB/s per channel - Dual channel: 51.2 GB/s total - L3 cache: ~200 GB/s - L2 cache: ~400 GB/s - L1 cache: ~1000 GB/s

Implication: Streaming through large arrays is bandwidth-limited

```
// Bandwidth-limited: streaming through 1 GB array
for (int i = 0; i < 256*1024*1024; i++) {
    array[i] = 0;  // Limited by DRAM bandwidth
}
```

Embedded systems have much lower bandwidth: - Typical MCU SRAM: 1-4 GB/s - External DRAM (if present): 100-500 MB/s

This makes **working set size** critical—keep data in on-chip SRAM.

Cache Coherency (Multi-core)

On multi-core systems, caches must stay **coherent**—all cores see consistent data.

MESI protocol (common on x86, ARM): - **M**odified: This cache has the only valid copy, modified - **E**xclusive: This cache has the only valid copy, clean - **S**hared: Multiple caches have valid copies - **I**nvalid: This cache line is invalid

False sharing: Performance killer on multi-core

```c
// BAD: False sharing
struct {
    int counter_core0;  // Used by core 0
    int counter_core1;  // Used by core 1
} shared;  // Both in same cache line!

// Core 0 writes counter_core0 → invalidates core 1's cache line
// Core 1 writes counter_core1 → invalidates core 0's cache line
// Ping-pong effect: terrible performance
```

Solution: Pad to separate cache lines

```c
// GOOD: No false sharing
struct {
    int counter_core0;
    char pad[60];       // Pad to 64 bytes
    int counter_core1;
} shared;
```

RISC-V: Uses RVWMO (RISC-V Weak Memory Ordering) with `fence` instructions for synchronization.

RISC-V Memory Model

RISC-V has a **weak memory model**—memory operations can be reordered unless you use fences.

Memory ordering:

```
// Without fence: these can be reordered
store A
store B
load C
load D
```

Fence instruction:

```
sw   a0, 0(a1)    # Store A
fence w, w        # Ensure store completes before next store
sw   a2, 0(a3)    # Store B
```

Fence types: - `fence r, r`: Load-load fence - `fence w, w`: Store-store fence - `fence rw, rw`: Full fence - `fence.i`: Instruction fence (for self-modifying code)

Atomic operations (A extension):

```
lr.w  a0, (a1)    # Load-reserved
# ... modify a0 ...
sc.w  a2, a0, (a1) # Store-conditional (fails if reservation broken)
```

Practical Guidelines

Based on this understanding of memory hierarchy, here are practical guidelines for data structure design:

1. Minimize cache misses - Use sequential access patterns when possible - Keep working set small (fit in L1/L2) - Avoid pointer chasing (linked lists, trees)

2. Exploit cache lines - Pack related data together (structs) - Align data structures to cache line boundaries - Avoid false sharing on multi-core

3. Consider prefetcher - Use predictable access patterns - Sequential or constant-stride access - Avoid random access when possible

4. Know your hardware - Cache sizes (L1, L2, L3) - Cache line size (usually 64 bytes) - Associativity (affects conflicts) - Prefetcher capabilities

5. Measure, don't guess - Use `perf` to measure cache misses - Profile before optimizing - Test on target hardware

Summary

The 100-cycle problem was solved by understanding the memory hierarchy. The device driver's packet loss came from 45,000 cache misses consuming 4.5 million cycles—90% of execution time spent waiting for memory. Optimizing memory access patterns reduced cache misses and brought throughput from 200,000 to the expected 2 million packets per second.

Key insights: - Cache misses cost 100-200 cycles (vs 1-4 for hits) - Caches fetch 64-byte lines, not individual bytes - Sequential access is 10-100× faster than random access - Embedded systems have smaller caches and simpler hierarchies

Design implications: - Arrays beat linked lists (spatial locality) - Small working sets beat large ones (temporal locality) - Sequential beats random (prefetcher) - Measurement beats intuition (use profiling tools)

Next Chapter: We'll build a comprehensive benchmarking framework to measure these effects precisely and learn how to use profiling tools effectively.

Chapter 3: Benchmarking and Profiling

Part I: Foundations

> "In God we trust. All others must bring data." — W. Edwards Deming

The Measurement Problem

After learning about memory hierarchy in Chapter 2, you might be eager to optimize your code. But there's a problem: **how do you know if your optimization actually worked?**

I learned this lesson the hard way.

I was optimizing a hash table implementation for a bootloader. Based on my understanding of cache behavior, I rewrote the hash function to be "more cache-friendly." I was confident it would be faster.

I ran the code. It felt faster. I committed the change.

A week later, a colleague ran benchmarks and found that my "optimization" had made the code **15% slower**. I had optimized for the wrong thing, and I had no data to prove my assumptions.

The lesson: Never trust your intuition. Always measure.

This chapter is about how to measure correctly. We'll build a comprehensive benchmarking framework and learn to use profiling tools effectively.

High-Precision Timing

The first challenge: how do you measure time accurately?

Bad approach: Using `time()`

```
time_t start = time(NULL);
run_test();
time_t end = time(NULL);
printf("Time: %ld seconds\n", end - start);
```

Problem: 1-second resolution. Useless for fast operations.

Better approach: Using `clock_gettime()`

```
struct timespec start, end;
clock_gettime(CLOCK_MONOTONIC, &start);
run_test();
clock_gettime(CLOCK_MONOTONIC, &end);

long ns = (end.tv_sec - start.tv_sec) * 1000000000L +
          (end.tv_nsec - start.tv_nsec);
printf("Time: %ld ns\n", ns);
```

Advantages: - Nanosecond resolution - `CLOCK_MONOTONIC`: Not affected by system time changes - Portable (POSIX)

Best approach: Using CPU cycle counters

On RISC-V:

```c
static inline uint64_t rdcycle(void) {
    uint64_t cycles;
    asm volatile ("rdcycle %0" : "=r" (cycles));
    return cycles;
}

uint64_t start = rdcycle();
run_test();
uint64_t end = rdcycle();
printf("Cycles: %lu\n", end - start);
```

On x86_64:

```c
static inline uint64_t rdtsc(void) {
    uint32_t lo, hi;
    asm volatile ("rdtsc" : "=a" (lo), "=d" (hi));
    return ((uint64_t)hi << 32) | lo;
}
```

On ARM64:

```c
static inline uint64_t rdcycle(void) {
    uint64_t val;
    asm volatile("mrs %0, pmccntr_el0" : "=r"(val));
    return val;
}
```

Advantages: - Highest precision (1 cycle) - Direct hardware measurement - No system call overhead

Disadvantages: - Architecture-specific - Can be affected by frequency scaling - May require kernel configuration (ARM)

Statistical Analysis

A single measurement is meaningless. You need **multiple runs** and **statistical analysis**.

Why? - Cache state varies between runs - OS interrupts affect timing - Branch prediction varies - Memory allocator behavior varies

Minimum approach: Run multiple times and report min/max/mean

```c
#define ITERATIONS 1000

uint64_t times[ITERATIONS];
for (int i = 0; i < ITERATIONS; i++) {
    uint64_t start = rdcycle();
    run_test();
    uint64_t end = rdcycle();
```

```
    times[i] = end - start;
}

// Calculate statistics
uint64_t min = times[0], max = times[0], sum = 0;
for (int i = 0; i < ITERATIONS; i++) {
    if (times[i] < min) min = times[i];
    if (times[i] > max) max = times[i];
    sum += times[i];
}
uint64_t mean = sum / ITERATIONS;

printf("Min: %lu cycles\n", min);
printf("Max: %lu cycles\n", max);
printf("Mean: %lu cycles\n", mean);
```

Better approach: Add median and standard deviation

```
// Sort for median
qsort(times, ITERATIONS, sizeof(uint64_t), compare_uint64);
uint64_t median = times[ITERATIONS / 2];

// Calculate standard deviation
double variance = 0;
for (int i = 0; i < ITERATIONS; i++) {
    double diff = (double)times[i] - (double)mean;
    variance += diff * diff;
}
double stddev = sqrt(variance / ITERATIONS);

printf("Median: %lu cycles\n", median);
printf("Std dev: %.2f cycles\n", stddev);
```

What to report? - **Minimum**: Best-case performance (warm cache) - **Median**: Typical performance (more robust than mean) - **Standard deviation**: Variability (lower is better) - **Maximum**: Worst-case (important for real-time systems)

Benchmark Framework Design

Let's build a reusable framework. Here's the interface:

```
typedef struct {
    const char *name;
    void (*setup)(void);
    void (*run)(void);
    void (*teardown)(void);
} benchmark_t;
```

```
void benchmark_run(benchmark_t *bench, int iterations);
```

Implementation:

```
void benchmark_run(benchmark_t *bench, int iterations) {
    uint64_t *times = malloc(iterations * sizeof(uint64_t));

    printf("Running benchmark: %s\n", bench->name);

    // Warmup run
    if (bench->setup) bench->setup();
    bench->run();
    if (bench->teardown) bench->teardown();

    // Actual measurements
    for (int i = 0; i < iterations; i++) {
        if (bench->setup) bench->setup();

        uint64_t start = rdcycle();
        bench->run();
        uint64_t end = rdcycle();

        if (bench->teardown) bench->teardown();

        times[i] = end - start;
    }

    // Calculate and report statistics
    report_statistics(bench->name, times, iterations);

    free(times);
}
```

Usage example:

```
// Test data
int array[1000];

void setup_array(void) {
    for (int i = 0; i < 1000; i++) {
        array[i] = i;
    }
}

void test_sequential_access(void) {
    volatile int sum = 0;
    for (int i = 0; i < 1000; i++) {
```

```
        sum += array[i];
    }
}

benchmark_t bench = {
    .name = "Sequential Array Access",
    .setup = setup_array,
    .run = test_sequential_access,
    .teardown = NULL
};

benchmark_run(&bench, 1000);
```

Cache Analysis with perf

Timing tells you **how long**, but not **why**. For that, you need cache analysis.

Linux perf is the standard tool for performance analysis:

```
# Basic cache statistics
$ perf stat -e cache-references,cache-misses ./program
  Performance counter stats:
    1,234,567 cache-references
       12,345 cache-misses              #    1.00% of all cache refs
```

Useful events: - `cache-references`: Total cache accesses - `cache-misses`: Cache misses (all levels) - `L1-dcache-loads`: L1 data cache loads - `L1-dcache-load-misses`: L1 data cache load misses - `LLC-loads`: Last-level cache loads - `LLC-load-misses`: Last-level cache load misses

Detailed analysis:

```
# L1 cache analysis
$ perf stat -e L1-dcache-loads,L1-dcache-load-misses ./program
  Performance counter stats:
    10,000,000 L1-dcache-loads
       100,000 L1-dcache-load-misses    #    1.00% miss rate

# All cache levels
$ perf stat -e cache-references,cache-misses,\
L1-dcache-loads,L1-dcache-load-misses,\
LLC-loads,LLC-load-misses ./program
```

Comparing implementations:

```
# Array version
$ perf stat -e cache-misses ./array_test
    1,234 cache-misses
```

```
# Linked list version
$ perf stat -e cache-misses ./list_test
   45,678 cache-misses              # 37× more misses!
```

Integrating perf with Benchmarks

We can integrate perf measurements into our benchmark framework:

```
typedef struct {
    uint64_t cycles;
    uint64_t cache_references;
    uint64_t cache_misses;
    uint64_t l1_loads;
    uint64_t l1_misses;
} perf_counters_t;

void benchmark_run_with_perf(benchmark_t *bench, int iterations) {
    // Setup perf counters
    int fd_cache_ref = perf_event_open(PERF_COUNT_HW_CACHE_REFERENCES);
    int fd_cache_miss = perf_event_open(PERF_COUNT_HW_CACHE_MISSES);

    // Run benchmark
    perf_counters_t counters = {0};

    for (int i = 0; i < iterations; i++) {
        if (bench->setup) bench->setup();

        // Read start counters
        uint64_t start_ref = read_counter(fd_cache_ref);
        uint64_t start_miss = read_counter(fd_cache_miss);
        uint64_t start_cycles = rdcycle();

        bench->run();

        // Read end counters
        uint64_t end_cycles = rdcycle();
        uint64_t end_miss = read_counter(fd_cache_miss);
        uint64_t end_ref = read_counter(fd_cache_ref);

        if (bench->teardown) bench->teardown();

        counters.cycles += end_cycles - start_cycles;
        counters.cache_references += end_ref - start_ref;
        counters.cache_misses += end_miss - start_miss;
    }
```

```
    // Report results
    printf("Benchmark: %s\n", bench->name);
    printf("  Cycles: %lu\n", counters.cycles / iterations);
    printf("  Cache refs: %lu\n", counters.cache_references / iterations);
    printf("  Cache misses: %lu (%.2f%%)\n",
           counters.cache_misses / iterations,
           100.0 * counters.cache_misses / counters.cache_references);
}
```

Common Pitfalls

1. Compiler optimizations

The compiler might optimize away your benchmark:

```
// BAD: Compiler optimizes this away
void test(void) {
    int sum = 0;
    for (int i = 0; i < 1000; i++) {
        sum += array[i];
    }
    // sum is never used, entire loop removed!
}

// GOOD: Use volatile or return value
void test(void) {
    volatile int sum = 0;  // Prevents optimization
    for (int i = 0; i < 1000; i++) {
        sum += array[i];
    }
}
```

2. Cold vs warm cache

First run is always slower (cold cache):

```
// First run: cold cache
run_test();  // 10,000 cycles

// Second run: warm cache
run_test();  // 1,000 cycles
```

Solution: Always do a warmup run, or report both cold and warm performance.

3. Measurement overhead

Timing code itself takes time:

```
uint64_t start = rdcycle();  // ~10 cycles
uint64_t end = rdcycle();    // ~10 cycles
```

```
printf("Overhead: %lu\n", end - start);  // ~20 cycles
```

Solution: Measure overhead and subtract it, or ensure test runs long enough that overhead is negligible.

4. System noise

OS interrupts, other processes, frequency scaling all add noise.

Solutions: - Run many iterations - Report median (robust to outliers) - Disable frequency scaling: `cpupower frequency-set -g performance` - Pin to CPU core: `taskset -c 0 ./program` - Increase priority: `nice -n -20 ./program`

Embedded Systems Considerations

Benchmarking on embedded systems has unique challenges:

1. Limited profiling tools

Many embedded systems don't have `perf` or similar tools.

Solution: Use hardware performance counters directly via memory-mapped registers.

Example (RISC-V):

```
// Enable performance counters (machine mode)
#define CSR_MCOUNTEREN 0x306
#define CSR_MCOUNTINHIBIT 0x320

void enable_perf_counters(void) {
    // Allow user mode to read counters
    asm volatile ("csrw %0, %1" :: "i"(CSR_MCOUNTEREN), "r"(0x7));
    // Enable all counters
    asm volatile ("csrw %0, %1" :: "i"(CSR_MCOUNTINHIBIT), "r"(0x0));
}
```

2. No operating system

Bare-metal systems have no `clock_gettime()`.

Solution: Use hardware timers or cycle counters.

```
// Use SoC timer (example)
#define TIMER_BASE 0x10000000
#define TIMER_MTIME (*(volatile uint64_t*)(TIMER_BASE + 0x00))

uint64_t get_time_us(void) {
    return TIMER_MTIME;  // Assuming 1 MHz timer
}
```

3. Real-time constraints

In real-time systems, **worst-case** matters more than average.

Solution: Report maximum time and 99th percentile.

```
// Sort times
qsort(times, iterations, sizeof(uint64_t), compare);

uint64_t min = times[0];
uint64_t max = times[iterations - 1];
uint64_t p50 = times[iterations / 2];
uint64_t p99 = times[(iterations * 99) / 100];

printf("Min: %lu cycles\n", min);
printf("P50: %lu cycles\n", p50);
printf("P99: %lu cycles\n", p99);
printf("Max: %lu cycles\n", max);
```

4. Limited memory

Can't store thousands of measurements.

Solution: Use online algorithms (running statistics).

```
typedef struct {
    uint64_t count;
    uint64_t min;
    uint64_t max;
    double mean;
    double m2;  // For variance calculation
} running_stats_t;

void update_stats(running_stats_t *stats, uint64_t value) {
    stats->count++;

    if (value < stats->min) stats->min = value;
    if (value > stats->max) stats->max = value;

    // Welford's online algorithm for mean and variance
    double delta = value - stats->mean;
    stats->mean += delta / stats->count;
    double delta2 = value - stats->mean;
    stats->m2 += delta * delta2;
}

double get_stddev(running_stats_t *stats) {
    return sqrt(stats->m2 / stats->count);
}
```

Practical Example: Array vs Linked List

Let's put it all together with a complete benchmark comparing arrays and linked lists:

```
#define SIZE 1000
#define ITERATIONS 1000

// Array implementation
int array[SIZE];

void setup_array(void) {
    for (int i = 0; i < SIZE; i++) {
        array[i] = i;
    }
}

void test_array_sequential(void) {
    volatile int sum = 0;
    for (int i = 0; i < SIZE; i++) {
        sum += array[i];
    }
}

// Linked list implementation
typedef struct node {
    int value;
    struct node *next;
} node_t;

node_t *list_head = NULL;

void setup_list(void) {
    list_head = NULL;
    for (int i = SIZE - 1; i >= 0; i--) {
        node_t *node = malloc(sizeof(node_t));
        node->value = i;
        node->next = list_head;
        list_head = node;
    }
}

void test_list_sequential(void) {
    volatile int sum = 0;
    node_t *curr = list_head;
    while (curr) {
```

```
        sum += curr->value;
        curr = curr->next;
    }
}

void teardown_list(void) {
    node_t *curr = list_head;
    while (curr) {
        node_t *next = curr->next;
        free(curr);
        curr = next;
    }
}

// Run benchmarks
int main(void) {
    benchmark_t benchmarks[] = {
        {
            .name = "Array Sequential",
            .setup = setup_array,
            .run = test_array_sequential,
            .teardown = NULL
        },
        {
            .name = "List Sequential",
            .setup = setup_list,
            .run = test_list_sequential,
            .teardown = teardown_list
        }
    };

    for (int i = 0; i < 2; i++) {
        benchmark_run_with_perf(&benchmarks[i], ITERATIONS);
    }

    return 0;
}
```

Expected output:

```
Benchmark: Array Sequential
  Cycles: 1,234
  Cache refs: 250
  Cache misses: 16 (6.40%)

Benchmark: List Sequential
  Cycles: 4,567
```

```
Cache refs: 1,000
Cache misses: 950 (95.00%)
```

Analysis: - Array: 3.7× faster - Array: 15.8× fewer cache misses - List: 95% cache miss rate (almost every access misses)

Summary

The measurement problem was solved by building a rigorous benchmarking framework. The "optimization" that felt faster turned out to be 15% slower—intuition failed, but data didn't. The framework revealed the truth through high-precision timing, statistical analysis, and cache profiling.

Measurement techniques: - High-precision timing (`clock_gettime()`, cycle counters) - Statistical analysis (min, median, stddev) - Cache analysis (`perf`, hardware counters)

Framework design: - Reusable benchmark structure - Setup/run/teardown phases - Warmup runs - Multiple iterations

Common pitfalls: - Compiler optimizations (use `volatile`) - Cold vs warm cache (warmup runs) - Measurement overhead (subtract or minimize) - System noise (many iterations, report median)

Embedded considerations: - Direct hardware counter access - Worst-case analysis (max, P99) - Online statistics (limited memory) - Bare-metal timing

Next Chapter: Armed with our benchmarking framework, we'll dive deep into arrays and explore how to maximize cache locality and performance.

Chapter 4: Arrays and Cache Locality

Part II: Basic Data Structures

> "The array is the most important data structure in computer science."
> — Donald Knuth (paraphrased)

The Simplest Data Structure

Arrays are so simple that we often take them for granted. Contiguous memory, O(1) access, what's there to optimize?

Everything.

I was working on a packet processing pipeline for a network switch. The code was straightforward: read packets from a ring buffer (an array), process them, and write results to another array. Simple, right?

The performance was terrible. We were processing 100,000 packets per second when the hardware should handle 1 million.

The profiler showed something strange:

```
$ perf stat -e cache-misses,instructions ./packet_processor
  Performance counter stats:
    450,000 cache-misses
  1,000,000 instructions
```

450,000 cache misses for 1,000,000 instructions? That's a cache miss every 2-3 instructions. For simple array operations, this made no sense.

The problem wasn't the arrays themselves—it was **how we were using them**.

Memory Layout Matters

Let's start with the basics. An array is contiguous memory:

```
int array[8] = {0, 1, 2, 3, 4, 5, 6, 7};
```

In memory (assuming 4-byte integers):

```
Address:  0x1000  0x1004  0x1008  0x100C  0x1010  0x1014  0x1018  0x101C
Value:    0       1       2       3       4       5       6       7

                        One 64-byte cache line
```

Key insight: All 8 integers fit in a single 64-byte cache line.

Accessing the array sequentially:

```
int sum = 0;
for (int i = 0; i < 8; i++) {
    sum += array[i];
}
```

Cache behavior with prefetching:

```
sequenceDiagram
    participant CPU
    participant Cache
    participant Prefetcher
    participant Memory

    CPU->>Cache: Request array[0]
    Cache->>Memory: MISS - Fetch cache line
    Memory-->>Cache: Return 64 bytes (array[0-15])
    Prefetcher->>Prefetcher: Detect sequential pattern
    Prefetcher->>Memory: Prefetch next cache line
    Memory-->>Cache: Prefetch array[16-31]
```

```
    CPU->>Cache: Request array[1]
    Cache-->>CPU: HIT (already in cache)

    CPU->>Cache: Request array[2-15]
    Cache-->>CPU: HIT (all in cache)

    CPU->>Cache: Request array[16]
    Cache-->>CPU: HIT (prefetched!)
    Prefetcher->>Memory: Prefetch array[32-47]

    Note over CPU,Memory: Prefetcher stays ahead,<br/>hiding memory latency
```

Cache behavior: - First access (`array[0]`): **Cache miss** (100 cycles) - Fetches entire cache line (64 bytes = 16 integers) - Next 7 accesses (`array[1]` to `array[7]`): **Cache hits** (1 cycle each) - **Prefetcher**: Detects pattern, fetches ahead

Total cost: 100 + 7 = 107 cycles for 8 accesses = **13.4 cycles per access**

Compare this to random access:

```
int indices[8] = {7, 2, 5, 0, 3, 6, 1, 4};
int sum = 0;
for (int i = 0; i < 8; i++) {
    sum += array[indices[i]];
}
```

If `indices` causes accesses to different cache lines: - Each access: **Cache miss** (100 cycles) - **Total cost**: 800 cycles for 8 accesses = **100 cycles per access**

Sequential is 7.5× faster than random, even though both are O(n).

Stride Patterns

Not all sequential access is equal. **Stride** matters.

Stride-1 access (best case):

```
for (int i = 0; i < n; i++) {
    sum += array[i];  // Stride = 1 element = 4 bytes
}
```

Stride-2 access (still good):

```
for (int i = 0; i < n; i += 2) {
    sum += array[i];  // Stride = 2 elements = 8 bytes
}
```

Large stride (worse):

```
for (int i = 0; i < n; i += 16) {
    sum += array[i];  // Stride = 16 elements = 64 bytes
}
```

Why does stride matter?

1. **Cache line utilization**: Stride-1 uses all 64 bytes fetched. Stride-16 uses only 4 bytes per cache line (6.25% utilization).
2. **Prefetcher effectiveness**: Hardware prefetchers detect stride patterns, but large strides may exceed prefetch distance.

Benchmark (1M element array):

```
Stride-1:   1.2 ms  (100% cache line utilization)
Stride-2:   1.3 ms  (50% utilization, still prefetched)
Stride-4:   1.5 ms  (25% utilization)
Stride-8:   2.1 ms  (12.5% utilization)
Stride-16:  3.8 ms  (6.25% utilization)
Stride-64:  8.5 ms  (1.56% utilization, new cache line each access)
```

Guideline: Keep stride small (8 elements) for good performance.

Real-World Tool: lmbench lat_mem_rd

The classic `lmbench` benchmark suite includes `lat_mem_rd`, which measures memory latency across different array sizes and strides. This is exactly what we've been discussing.

How it works:

```
// Simplified version of lmbench lat_mem_rd
char *p = array;
for (int i = 0; i < iterations; i++) {
    // Pointer chasing with configurable stride
    p = *(char **)p;  // Follow pointer to next element
}
```

The array is initialized so each element points to the next element at distance `stride`:

```
// Initialize array with stride
for (size_t i = 0; i < size; i += stride) {
    array[i] = &array[(i + stride) % size];
}
```

Running lmbench:

```
$ lat_mem_rd 64M 128
# Array size: 64 MB, stride: 128 bytes

Output:
```

```
Stride  Latency
  128     3.2 ns   (L1 cache)
  256     3.5 ns   (L1 cache)
  512     4.1 ns   (L1 cache)
 1024     5.8 ns   (L2 cache)
 4096    12.5 ns   (L2 cache)
16384    45.0 ns   (L3 cache)
65536   102.0 ns   (DRAM)
```

What this shows: - Small strides (128-512 bytes): Stay in L1 cache (~3-4 ns) - Medium strides (1-4 KB): L2 cache (~6-12 ns) - Large strides (16-64 KB): L3 cache or DRAM (45-100+ ns)

Why stride affects latency: - **Small stride**: Sequential access, prefetcher helps, stays in L1 - **Large stride**: Jumps across cache lines, defeats prefetcher, evicts from L1

Key insight: This is why **data structure layout matters**. If your struct is 128 bytes and you iterate through an array of them, you're doing stride-128 access. If only 8 bytes of the struct are "hot" (frequently accessed), you're wasting 93.75% of each cache line.

Embedded perspective: On embedded systems without L3 cache, the latency cliff is steeper. Once you exceed L1/L2 capacity, you go straight to DRAM or flash (100-1000× slower).

Multi-Dimensional Arrays

Multi-dimensional arrays introduce a critical choice: **row-major** vs **column-major** layout.

C uses row-major order:

```c
int matrix[4][4] = {
    {0,  1,  2,  3},
    {4,  5,  6,  7},
    {8,  9,  10, 11},
    {12, 13, 14, 15}
};
```

Memory layout (row-major):

```
Address:  0x1000  0x1004  0x1008  0x100C  0x1010  0x1014  0x1018  0x101C  ...
Value:    0       1       2       3       4       5       6       7       ...
                  Row 0                 Row 1
```

Row-major traversal (good):

```c
for (int i = 0; i < 4; i++) {
    for (int j = 0; j < 4; j++) {
        sum += matrix[i][j];  // Sequential in memory
```

```
    }
}
```

Column-major traversal (bad):

```
for (int j = 0; j < 4; j++) {
    for (int i = 0; i < 4; i++) {
        sum += matrix[i][j];  // Stride = 4 elements = 16 bytes
    }
}
```

Benchmark (1024×1024 matrix):

```
Row-major:     12 ms   (sequential access)
Column-major:  45 ms   (stride-1024 access)
```

Column-major is 3.75× slower for the same algorithm!

The Matrix Multiplication Problem

Matrix multiplication is the classic example of cache optimization:

```
// Naive implementation
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        for (int k = 0; k < N; k++) {
            C[i][j] += A[i][k] * B[k][j];
        }
    }
}
```

Access patterns: - `A[i][k]`: Row-major, good (stride-1) - `C[i][j]`: Same element repeatedly, excellent (temporal locality) - `B[k][j]`: **Column-major, terrible** (stride-N)

For N=1024: Accessing `B[k][j]` has stride of 1024 elements = 4096 bytes = 64 cache lines!

Solution 1: Loop reordering (ikj order)

```
// Better: ikj order
for (int i = 0; i < N; i++) {
    for (int k = 0; k < N; k++) {
        int r = A[i][k];
        for (int j = 0; j < N; j++) {
            C[i][j] += r * B[k][j];  // Now B is row-major!
        }
    }
}
```

Access patterns now: - `A[i][k]`: Row-major, good - `B[k][j]`: Row-major, good (was column-major) - `C[i][j]`: Row-major, good

Benchmark (512×512 matrices):

```
ijk order (naive):  2,450 ms
ikj order:            680 ms   (3.6× faster)
```

Solution 2: Blocking (tiling)

For very large matrices that don't fit in cache, use **blocking**:

```
#define BLOCK_SIZE 64

for (int ii = 0; ii < N; ii += BLOCK_SIZE) {
    for (int jj = 0; jj < N; jj += BLOCK_SIZE) {
        for (int kk = 0; kk < N; kk += BLOCK_SIZE) {
            // Process BLOCK_SIZE × BLOCK_SIZE submatrix
            for (int i = ii; i < ii + BLOCK_SIZE && i < N; i++) {
                for (int k = kk; k < kk + BLOCK_SIZE && k < N; k++) {
                    int r = A[i][k];
                    for (int j = jj; j < jj + BLOCK_SIZE && j < N; j++) {
                        C[i][j] += r * B[k][j];
                    }
                }
            }
        }
    }
}
```

Why blocking works: - Processes small blocks that fit in L1 cache - Reuses data before eviction - Reduces cache misses dramatically

Benchmark (1024×1024 matrices):

```
Naive (ijk):     18,500 ms
Reordered (ikj):  5,200 ms   (3.6× faster)
Blocked:          1,800 ms   (10.3× faster than naive, 2.9× faster than reordered)
```

Structure of Arrays vs Array of Structures

How you organize data in arrays has huge performance implications.

Memory layout comparison:

```
graph TD
    subgraph "AoS: Array of Structures"
        A1["Cache Line 0<br/>[x,y,z,vx,vy,vz,mass,id]<br/>Particle 0"]
        A2["Cache Line 1<br/>[x,y,z,vx,vy,vz,mass,id]<br/>Particle 1"]
        A3["Cache Line 2<br/>[x,y,z,vx,vy,vz,mass,id]<br/>Particle 2"]
        A1 --> A2 --> A3
```

```
        style A1 fill:#ffcccb
        style A2 fill:#ffcccb
        style A3 fill:#ffcccb
    end

    subgraph "SoA: Structure of Arrays"
        S1["Cache Line 0<br/>[x[0], x[1], ..., x[15]]"]
        S2["Cache Line 1<br/>[x[16], x[17], ..., x[31]]"]
        S3["Cache Line N<br/>[vx[0], vx[1], ..., vx[15]]"]
        S1 --> S2 --> S3
        style S1 fill:#90ee90
        style S2 fill:#90ee90
        style S3 fill:#90ee90
    end

    note1["AoS: 37.5% cache utilization<br/>Only need x,y,z,vx,vy,vz<br/>but fetch mass,id t
    note2["SoA: 100% cache utilization<br/>Each cache line contains<br/>only needed data"]

    A3 -.-> note1
    S3 -.-> note2
```

Array of Structures (AoS):

```c
typedef struct {
    float x, y, z;    // Position (12 bytes)
    float vx, vy, vz; // Velocity (12 bytes)
    float mass;       // Mass (4 bytes)
    int id;           // ID (4 bytes)
} particle_t;        // Total: 32 bytes

particle_t particles[1000];

// Update positions
for (int i = 0; i < 1000; i++) {
    particles[i].x += particles[i].vx * dt;
    particles[i].y += particles[i].vy * dt;
    particles[i].z += particles[i].vz * dt;
}
```

Memory layout:

```
Cache line 0: [p0.x, p0.y, p0.z, p0.vx, p0.vy, p0.vz, p0.mass, p0.id]
Cache line 1: [p1.x, p1.y, p1.z, p1.vx, p1.vy, p1.vz, p1.mass, p1.id]
...
```

Problem: Each cache line contains data we don't need (`mass`, `id`). We're using only 24 bytes out of 64 (37.5% utilization).

Structure of Arrays (SoA):

```c
typedef struct {
    float x[1000];
    float y[1000];
    float z[1000];
    float vx[1000];
    float vy[1000];
    float vz[1000];
    float mass[1000];
    int id[1000];
} particles_t;

particles_t particles;

// Update positions
for (int i = 0; i < 1000; i++) {
    particles.x[i] += particles.vx[i] * dt;
    particles.y[i] += particles.vy[i] * dt;
    particles.z[i] += particles.vz[i] * dt;
}
```

Memory layout:

```
Cache line 0: [x[0], x[1], x[2], ..., x[15]]
Cache line 1: [x[16], x[17], ..., x[31]]
...
```

Advantage: 100% cache line utilization. Each cache line contains only the data we need.

Benchmark (1M particles, 1000 iterations):

```
AoS:  2,850 ms
SoA:  1,200 ms   (2.4× faster)
```

When to use SoA: - Operations access only a few fields - Large arrays (> cache size) - Performance-critical loops

When to use AoS: - Operations access all fields - Small arrays (< cache size) - Code clarity matters more than performance

Alignment and Padding

Memory alignment affects both correctness and performance.

Natural alignment: - `char`: 1-byte aligned - `short`: 2-byte aligned - `int`: 4-byte aligned - `long`: 8-byte aligned - `double`: 8-byte aligned

Unaligned access:

```c
char buffer[16];
int *p = (int*)(buffer + 1);  // Unaligned!
```

```
*p = 42;  // May be slow or crash
```

On x86: Unaligned access works but is slower (may cross cache line boundary)
On ARM/RISC-V: May trap or require multiple accesses

Structure padding:

```
struct bad {
    char a;     // 1 byte
    int b;      // 4 bytes, needs 4-byte alignment
    char c;     // 1 byte
};  // Size: 12 bytes (with padding)
```

Memory layout:

```
Offset:  0    1    2    3    4    5    6    7    8    9    10   11
Value:   a    pad  pad  pad  b    b    b    b    c    pad  pad  pad
```

Better ordering:

```
struct good {
    int b;      // 4 bytes
    char a;     // 1 byte
    char c;     // 1 byte
};  // Size: 8 bytes (with padding)
```

Memory layout:

```
Offset:  0    1    2    3    4    5    6    7
Value:   b    b    b    b    a    c    pad  pad
```

Guideline: Order struct members from largest to smallest to minimize padding.

Cache line alignment:

For performance-critical structures, align to cache line boundaries:

```
struct __attribute__((aligned(64))) cache_aligned {
    int data[16];
};
```

Why? - Prevents false sharing on multi-core - Ensures structure doesn't span cache lines - Predictable cache behavior

Array Bounds and Prefetching

Modern CPUs prefetch data, but they can't prefetch past array bounds they don't know.

Helping the prefetcher:

```
// BAD: Unpredictable loop bound
for (int i = 0; i < get_count(); i++) {
    sum += array[i];
```

```
}

// GOOD: Constant loop bound
int n = get_count();
for (int i = 0; i < n; i++) {
    sum += array[i];
}

// BETTER: Compiler can see bound
#define SIZE 1000
for (int i = 0; i < SIZE; i++) {
    sum += array[i];
}
```

Loop unrolling helps prefetching:

```
// Manual unrolling
for (int i = 0; i < n; i += 4) {
    sum += array[i];
    sum += array[i+1];
    sum += array[i+2];
    sum += array[i+3];
}
```

Benefits: - Reduces loop overhead - Exposes more parallelism - Helps prefetcher see pattern

Compiler can auto-unroll:

```
#pragma GCC unroll 4
for (int i = 0; i < n; i++) {
    sum += array[i];
}
```

Embedded Systems: Small Arrays, Big Impact

On embedded systems with tiny caches (8-32 KB), array optimization is even more critical.

Example: RISC-V MCU with 16 KB L1 cache

```
// This array is 40% of your entire cache!
int buffer[1000];  // 4 KB
```

Guidelines for embedded:

1. Keep arrays small

```
// BAD: Wastes cache
int large_buffer[10000];  // 40 KB, doesn't fit in cache
```

```
// GOOD: Fits in cache
int small_buffer[1000];   // 4 KB, fits comfortably
```

2. Reuse arrays

```
// BAD: Multiple arrays compete for cache
int input[1000];
int temp[1000];
int output[1000];

// GOOD: Reuse same buffer
int buffer[1000];
process_in_place(buffer);
```

3. Use smaller types

```
// BAD: Wastes memory and cache
int32_t values[1000];  // 4 KB

// GOOD: If range allows
int16_t values[1000];  // 2 KB, 2× more data in cache
```

4. Pack data

```
// BAD: 4 bytes per flag
int flags[1000];  // 4 KB

// GOOD: 1 bit per flag
uint32_t flags[32];  // 128 bytes, 32× smaller!

void set_flag(int i) {
    flags[i / 32] |= (1 << (i % 32));
}

int get_flag(int i) {
    return (flags[i / 32] >> (i % 32)) & 1;
}
```

Real-World Example: Packet Buffer

Back to my packet processing problem. Here's what was wrong:

Original code (bad):

```
typedef struct {
    uint8_t data[1500];     // Packet data
    uint32_t length;        // Packet length
    uint32_t timestamp;     // Timestamp
    uint32_t src_ip;        // Source IP
    uint32_t dst_ip;        // Dest IP
```

```
    uint16_t src_port;      // Source port
    uint16_t dst_port;      // Dest port
    uint8_t protocol;       // Protocol
    uint8_t flags;          // Flags
} packet_t;  // Total: ~1520 bytes

packet_t packets[1000];  // 1.52 MB

// Process packets
for (int i = 0; i < count; i++) {
    if (packets[i].protocol == TCP) {
        process_tcp(&packets[i]);
    }
}
```

Problem: Each iteration fetches 1520 bytes (24 cache lines) just to check `protocol` (1 byte).

Fixed code (good):

```
// Separate hot and cold data
typedef struct {
    uint8_t protocol;       // Hot: checked every iteration
    uint8_t flags;          // Hot: checked often
    uint16_t length;        // Hot: used for processing
    uint32_t data_offset;   // Offset into data array
} packet_header_t;  // 8 bytes

packet_header_t headers[1000];  // 8 KB
uint8_t packet_data[1500 * 1000];  // 1.5 MB

// Process packets
for (int i = 0; i < count; i++) {
    if (headers[i].protocol == TCP) {
        uint8_t *data = &packet_data[headers[i].data_offset];
        process_tcp(&headers[i], data);
    }
}
```

Result: - Headers fit in cache (8 KB vs 1.52 MB) - First loop: 8× fewer cache lines - Only fetch packet data when needed - **Performance**: 100K → 950K packets/sec (9.5× faster)

Summary

The packet processing pipeline's terrible performance—100,000 packets per second instead of 1 million—was fixed by understanding array access patterns. The 450,000 cache misses came from poor memory layout and access order.

Restructuring to Structure of Arrays and optimizing traversal order brought performance to 950,000 packets per second, nearly 10× faster.

Key principles: - Sequential access beats random (7-10× faster) - Small strides beat large strides - Row-major traversal for C arrays - SoA beats AoS for selective field access - Alignment matters (correctness and performance) - Keep working set in cache

Optimization techniques: - Loop reordering (ikj vs ijk) - Blocking/tiling for large arrays - Structure of Arrays (SoA) - Proper alignment and padding - Loop unrolling

Embedded considerations: - Keep arrays small (fit in cache) - Reuse buffers - Use smaller types - Pack data (bit arrays) - Separate hot/cold data

Measurement: - Profile before optimizing - Measure cache misses - Test on target hardware

Next Chapter: We've seen why arrays are fast. Now let's explore why linked lists are slow—and when you should use them anyway.

Chapter 5: Linked Lists - The Cache Killer

Part II: Basic Data Structures

> "Linked lists are the goto of data structures." — Attributed to various systems programmers

The Textbook Story

Every computer science student learns about linked lists in their first data structures course. The pitch is compelling:

Advantages (according to textbooks): - **O(1) insertion and deletion** at known positions - **Dynamic size**: Grows and shrinks as needed - **No wasted space**: Allocate exactly what you need - **Flexible**: Easy to implement stacks, queues, and other structures

Disadvantages (according to textbooks): - **O(n) search**: Must traverse from head - **Extra memory**: Pointers add overhead - **No random access**: Can't jump to arbitrary positions

The textbook conclusion: "Use linked lists when you need frequent insertions/deletions and don't need random access."

Sounds reasonable, right?

The Reality Check

Here's what the textbooks don't tell you: **Linked lists are almost always the wrong choice.**

Not because the Big-O analysis is wrong—it's correct. But because it's incomplete. It ignores the hardware.

Let's run a simple experiment. We'll compare three operations on 100,000 elements:

1. **Sequential traversal**: Visit every element
2. **Random access**: Access elements in random order
3. **Insertion**: Add elements one by one

We'll test both arrays and linked lists. Here are the results:

```
=== Sequential Traversal ===
Array:        70  s
Linked List:  179  s
Winner: Array (2.5× faster)

=== Random Access ===
Array:        95  s
Linked List:  2,847  s
Winner: Array (30× faster!)

=== Insertion (at end) ===
Array:        42  s
Linked List:  1,234  s
Winner: Array (29× faster!)
```

Wait, what? The array is faster at *insertion*? But that's supposed to be O(n) for arrays and O(1) for linked lists!

Welcome to the reality of modern hardware.

Why Linked Lists Are Slow

The problem is **pointer chasing**. Every time you follow a pointer, you're likely to miss the cache.

Memory layout comparison:

```
flowchart TD
    subgraph Array["Array: Contiguous Memory (Fast)"]
        direction LR
        A1["[0]<br/>0x1000"] --> A2["[1]<br/>0x1004"] --> A3["[2]<br/>0x1008"] --> A4["[3]<b
    end

    Array -.-> LinkedList
```

```
    subgraph LinkedList["Linked List: Scattered Memory (Slow)"]
        direction LR
        L1["Node A<br/>0x1000"] -.next.-> L2["Node B<br/>0x5000"] -.next.-> L3["Node C<br/>0
    end

    style A1 fill:#90ee90
    style A2 fill:#90ee90
    style A3 fill:#90ee90
    style A4 fill:#90ee90
    style A5 fill:#90ee90
    style L1 fill:#ffcccb
    style L2 fill:#ffcccb
    style L3 fill:#ffcccb
    style L4 fill:#ffcccb
    style L5 fill:#ffcccb
```

Cache behavior during traversal:

```
stateDiagram-v2
    [*] --> ArrayStart: Array Traversal
    ArrayStart --> Array0: Access [0] @ 0x1000
    Array0 --> ArrayFetch: MISS (100 cycles)<br/>Fetch cache line
    ArrayFetch --> Array1_15: Access [1-15]
    Array1_15 --> Array1_15: HIT (1 cycle each)<br/>All in same cache line
    Array1_15 --> ArrayDone: Continue...

    [*] --> LLStart: Linked List Traversal
    LLStart --> NodeA: Access Node A @ 0x1000
    NodeA --> NodeB: MISS (100 cycles)<br/>Jump to 0x5000
    NodeB --> NodeC: MISS (100 cycles)<br/>Jump to 0x2000
    NodeC --> NodeD: MISS (100 cycles)<br/>Jump to 0x8000
    NodeD --> LLDone: Every access = MISS

    note right of ArrayFetch
        Prefetcher detects
        sequential pattern
        Fetches ahead
    end note

    note right of NodeB
        Random addresses
        Defeats prefetcher
        Every node = cache miss
    end note
```

The difference is dramatic:

```
Step 1: Access node A
  CPU: "Fetch address 0x1000"
  Cache: MISS (100 cycles)
  Memory: Returns node A + 63 bytes of nearby data

Step 2: Access node B (via A->next)
  CPU: "Fetch address 0x5000"  (random location)
  Cache: MISS (100 cycles)
  Memory: Returns node B + 63 bytes of nearby data

Step 3: Access node C (via B->next)
  CPU: "Fetch address 0x2000"  (random location)
  Cache: MISS (100 cycles)
  Memory: Returns node C + 63 bytes of nearby data
```

Each node access is a cache miss. Each cache miss costs ~100 cycles.

For 100,000 nodes, that's **10 million cycles** just waiting for memory.

Compare this to an array:

```
Step 1: Access array[0]
  CPU: "Fetch address 0x1000"
  Cache: MISS (100 cycles)
  Memory: Returns 64 bytes (16 integers)

Step 2-16: Access array[1] through array[15]
  CPU: "Fetch addresses 0x1004, 0x1008, ..."
  Cache: HIT (3 cycles each)

Step 17: Access array[16]
  CPU: "Fetch address 0x1040"
  Cache: MISS (100 cycles)
  Memory: Returns next 64 bytes (16 more integers)
```

Only 1 cache miss per 16 elements. That's **6,250 cache misses** for 100,000 elements.

10 million cycles vs 625,000 cycles. The array is 16× faster just from cache behavior.

The Memory Overhead

Linked lists also waste memory. A lot of memory.

Consider a simple linked list node storing a 32-bit integer:

```c
typedef struct node {
    int value;        // 4 bytes
```

```
    struct node *next; // 8 bytes (on 64-bit systems)
} node_t;              // Total: 12 bytes + 4 bytes padding = 16 bytes
```

For a 4-byte integer, you're using 16 bytes. That's **4× overhead**.

An array of 100,000 integers: - Array: 400 KB - Linked list: 1.6 MB

The linked list uses **4× more memory** and is **2.5× slower**. That's a terrible trade-off.

The Allocation Cost

There's another hidden cost: memory allocation.

Creating a linked list requires calling `malloc()` for each node:

```
// Linked list: 100,000 malloc calls
for (int i = 0; i < 100000; i++) {
    node_t *node = malloc(sizeof(node_t));  // Expensive!
    node->value = i;
    node->next = head;
    head = node;
}
```

Each `malloc()` call: - Searches the free list - Updates metadata - Potentially calls the kernel for more memory - Fragments the heap

Creating an array requires one allocation:

```
// Array: 1 malloc call
int *array = malloc(100000 * sizeof(int));  // Fast!
for (int i = 0; i < 100000; i++) {
    array[i] = i;
}
```

In our benchmarks, creating the linked list took **1,234 s** vs **42 s** for the array. That's **29× slower**.

When Linked Lists Make Sense

So when *should* you use linked lists? The answer is: **rarely**.

When to consider linked lists:

```
flowchart LR
    Start["Use linked list?"] --> Q1{"Kernel/OS<br/>development?"}
    Q1 -->|Yes| A1["  Intrusive list<br/>(Linux kernel style)"]
    Q1 -->|No| Q2{"Lock-free<br/>required?"}
    Q2 -->|Yes| A2["  Linked list<br/>+ memory pool"]
    Q2 -->|No| A3["  Use dynamic array<br/>(std::vector)"]
```

```
    style A1 fill:#90ee90
    style A2 fill:#ffeb3b
    style A3 fill:#ffcccb
```

Here are the few legitimate use cases:

1. Intrusive Lists in Kernels

The Linux kernel uses linked lists extensively, but not the textbook version. They use *intrusive lists*:

```c
struct list_head {
    struct list_head *next, *prev;
};

struct task_struct {
    // ... task data ...
    struct list_head tasks;  // Embedded list node
};
```

The list node is embedded in the data structure, not allocated separately. This: - Eliminates allocation overhead - Improves cache locality (data and links together) - Allows one object to be in multiple lists

2. Lock-Free Algorithms

Some lock-free data structures use linked lists because: - Atomic pointer updates are easier than array updates - No need to resize (which requires locks)

Example: Lock-free stack (Treiber stack):

```c
typedef struct node {
    int value;
    struct node *next;
} node_t;

void push(node_t **head, node_t *node) {
    do {
        node->next = *head;
    } while (!atomic_compare_exchange(head, &node->next, node));
}
```

But even here, you'd use a memory pool to avoid allocation overhead.

3. Rare Insertions in Large Datasets

If you have a large, mostly-static dataset with occasional insertions, a linked list *might* make sense.

But honestly? A dynamic array with amortized O(1) insertion is usually better.

Optimization Strategies

If you must use a linked list, here's how to make it less terrible:

Strategy 1: Memory Pools

Instead of calling `malloc()` for each node, allocate nodes from a pool:

```
#define POOL_SIZE 10000
node_t node_pool[POOL_SIZE];
int pool_index = 0;

node_t *alloc_node(void) {
    if (pool_index >= POOL_SIZE) return NULL;
    return &node_pool[pool_index++];
}
```

Benefits: - **Faster allocation**: No malloc overhead - **Better locality**: Nodes are contiguous - **Predictable memory**: No fragmentation

Benchmark results:

```
Linked list (malloc):  1,234  s
Linked list (pool):      287  s
Array:                    42  s
```

The pool is 4.3× faster than malloc, but still 6.8× slower than an array.

Strategy 2: Unrolled Linked Lists

Store multiple elements per node:

```
#define ELEMENTS_PER_NODE 16

typedef struct node {
    int values[ELEMENTS_PER_NODE];
    int count;
    struct node *next;
} unrolled_node_t;
```

Benefits: - **Better cache utilization**: 16 elements per cache miss instead of 1 - **Less pointer overhead**: 1 pointer per 16 elements - **Fewer allocations**: 1/16th the malloc calls

Benchmark results:

```
Standard linked list:  179  s
Unrolled linked list:   45  s
Array:                  70  s
```

Wait, the unrolled list is faster than the array? Not quite—this is for sequential traversal only. For random access, the array still wins.

Strategy 3: XOR Linked Lists

Save memory by XORing prev and next pointers:

```
typedef struct node {
    int value;
    struct node *prev_xor_next;  // prev XOR next
} xor_node_t;
```

To traverse:

```
node_t *prev = NULL;
node_t *curr = head;
while (curr) {
    node_t *next = (node_t *)((uintptr_t)prev ^ (uintptr_t)curr->prev_xor_next);
    prev = curr;
    curr = next;
}
```

Benefits: - **50% less pointer memory**: One pointer instead of two - **Same traversal cost**: Still one cache miss per node

Drawbacks: - **More complex code**: XOR logic is tricky - **No backward traversal from arbitrary node**: Need both prev and curr - **Debugging nightmare**: Can't inspect pointers directly

Verdict: **Not worth it** in most cases. The memory savings are small, and the complexity is high.

Real-World Case Study: RTOS Task Lists

Let's look at a real embedded systems use case: task scheduling in an RTOS.

Scenario: FreeRTOS manages ready tasks in priority-ordered lists.

Requirements: - Insert task when it becomes ready (O(1) or O(n)) - Remove highest-priority task (O(1)) - Occasional priority changes (O(n))

FreeRTOS's solution: Array of linked lists, one per priority level.

```
#define MAX_PRIORITIES 32

typedef struct {
    struct list_head ready_tasks[MAX_PRIORITIES];
    int highest_priority;
} scheduler_t;
```

Why this works: - **Small lists**: Typically 1-5 tasks per priority - **Embedded list nodes**: No allocation overhead - **Cache-friendly**: Task struct + list node together - **O(1) operations**: Insert/remove at known priority

Benchmark (on ARM Cortex-M4):

```
Insert task:     0.8 s
Remove task:     0.6 s
Find next task:  0.3 s
```

This is fast enough for a 1 kHz scheduler (1000 s period).

Key insight: The linked list works here because: 1. Lists are small (cache-friendly) 2. Nodes are embedded (no allocation) 3. Operations are simple (no complex traversal)

Embedded Systems Considerations

In embedded systems, linked lists are even more problematic:

Problem 1: Fragmentation

Repeated malloc/free causes heap fragmentation:

```
Initial heap: [----------------free----------------]
After 1000 allocations and 500 frees:
[used][free][used][free][used][free][used][free]...
```

Eventually, you can't allocate even though total free space is sufficient.

Solution: Use memory pools or avoid dynamic allocation entirely.

Problem 2: Unpredictable Timing

Cache misses make linked list traversal unpredictable:

```
Best case:  All nodes in cache → 50 s
Worst case: All nodes in DRAM → 500 s
```

For real-time systems, this 10× variance is unacceptable.

Solution: Use arrays with predictable access patterns.

Problem 3: Memory Overhead

On a system with 64 KB RAM, a linked list of 1000 elements uses: - Data: 4 KB (1000 × 4 bytes) - Pointers: 8 KB (1000 × 8 bytes) - Malloc overhead: ~2 KB (metadata) - Total: 14 KB (22% of RAM!)

An array would use 4 KB (6% of RAM).

Solution: Use arrays or unrolled lists.

Design Guidelines

Here's a decision tree for choosing between arrays and linked lists:

```
flowchart TD
    Start["Choosing data structure"] --> Q1{"Dynamic size?"}

    Q1 -->|No| A1[" Use fixed array"]
    Q1 -->|Yes| Q2{"Random access<br/>needed?"}

    Q2 -->|Yes| A2[" Use dynamic array<br/>(vector/ArrayList)"]
    Q2 -->|No| Q3{"Frequent insertions<br/>in middle?"}

    Q3 -->|No| A3[" Use dynamic array<br/>(append-only)"]
    Q3 -->|Yes| Q4{"List size?"}

    Q4 -->|"< 100"| Q5{"Embedded<br/>system?"}
    Q4 -->|"> 100"| A4[" B-tree or skip list"]

    Q5 -->|Yes| A5[" Avoid linked list<br/>Use array with pool"]
    Q5 -->|No| A6[" Linked list OK<br/>(but test array first)"]

    style A1 fill:#90ee90
    style A2 fill:#90ee90
    style A3 fill:#90ee90
    style A4 fill:#90ee90
    style A5 fill:#ffcccb
    style A6 fill:#ffeb3b
```

Rule of thumb: If you're considering a linked list, try a dynamic array first. You'll probably be happier.

Benchmarking Linked Lists

Let's do a comprehensive benchmark comparing arrays and linked lists across different operations:

Test Setup

- 100,000 elements
- x86_64 system, 32 KB L1 cache
- GCC -O2 optimization

Results

Operation	Array	Linked List	Speedup
Sequential traversal	70 s	179 s	2.5×
Random access	95 s	2,847 s	30×
Insert at end	42 s	1,234 s	29×
Insert at beginning	0.01 s	0.02 s	2×

Operation	Array	Linked List	Speedup
Delete from middle	45 s	1,150 s	25×
Search for element	82 s	2,234 s	27×

Key observations: 1. **Arrays win almost everything** by 2-30× 2. **Only exception**: Insert at beginning (but who does that?) 3. **Cache behavior dominates**: Random access is 30× slower for lists

Cache Analysis

Using `perf` to measure cache behavior:

```
$ perf stat -e cache-references,cache-misses ./benchmark

Array traversal:
  423,156 cache-references
   89,234 cache-misses (21.1% miss rate)

Linked list traversal:
  1,247,832 cache-references
    892,441 cache-misses (71.5% miss rate)
```

The linked list has **3.4× more cache misses**. That's why it's slow.

Summary

The textbook story about linked lists was contradicted by reality. Arrays beat linked lists in every benchmark: 2.5× faster for sequential traversal, 30× faster for random access, even 3× faster for insertions in many cases. The linked list's 71.5% cache miss rate versus the array's 20.9% explained the performance gap. Cache behavior dominated algorithmic complexity.

The Textbook Story: - Linked lists: O(1) insertion, flexible, dynamic - Arrays: O(n) insertion, fixed size, inflexible

The Reality: - Linked lists: Slow due to cache misses, memory overhead, allocation cost - Arrays: Fast, cache-friendly, predictable

When to Use Linked Lists: 1. Intrusive lists in kernels (embedded nodes) 2. Lock-free algorithms (with memory pools) 3. Small lists (<100 elements) with rare insertions 4. When you've benchmarked and proven it's faster (rare!)

When to Use Arrays: 1. Almost always 2. Seriously, just use arrays 3. Or dynamic arrays if you need to grow 4. Did I mention arrays?

Optimization Strategies (if you must use linked lists): 1. Memory pools for allocation 2. Unrolled lists for better cache utilization 3. Embedded nodes to avoid separate allocation 4. Keep lists small

Embedded Systems: - Avoid linked lists due to fragmentation, unpredictable timing, and memory overhead - Use arrays or memory pools - Profile and measure everything

Key Takeaway: **Linked lists are the goto of data structures—avoid them unless you have a very good reason.**

Chapter 6: Stacks and Queues

Part II: Basic Data Structures

"Simplicity is prerequisite for reliability." — Edsger W. Dijkstra

The Invisible Data Structure

Every program uses a stack—the call stack. Every function call pushes a frame, every return pops it. It's so fundamental that we rarely think about it.

But when you need an explicit stack or queue, the implementation choices matter enormously.

I was debugging a firmware crash on a RISC-V embedded system. The system had a task scheduler that used a queue to manage pending tasks. Under heavy load, the system would crash with a stack overflow.

Wait, stack overflow? The queue was supposed to be on the heap, not the stack.

The problem wasn't the queue itself—it was **how the queue was implemented**. The queue used a linked list, and each `malloc()` call was allocating from a memory pool that shared space with the stack. Under load, the queue grew, the pool fragmented, and eventually the stack had nowhere to grow.

The fix? Replace the linked list queue with a **ring buffer**—a fixed-size array-based queue. No dynamic allocation, predictable memory usage, and 10× faster.

Stack: Array vs Linked List

Let's start with stacks. The textbook presents two implementations:

Array-based stack:

```
#define MAX_SIZE 1000

typedef struct {
    int data[MAX_SIZE];
    int top;
} stack_t;
```

```
void push(stack_t *s, int value) {
    if (s->top < MAX_SIZE) {
        s->data[s->top++] = value;
    }
}

int pop(stack_t *s) {
    if (s->top > 0) {
        return s->data[--s->top];
    }
    return -1;  // Error
}
```

Linked list stack:

```
typedef struct node {
    int value;
    struct node *next;
} node_t;

typedef struct {
    node_t *top;
} stack_t;

void push(stack_t *s, int value) {
    node_t *node = malloc(sizeof(node_t));
    node->value = value;
    node->next = s->top;
    s->top = node;
}

int pop(stack_t *s) {
    if (s->top) {
        node_t *node = s->top;
        int value = node->value;
        s->top = node->next;
        free(node);
        return value;
    }
    return -1;  // Error
}
```

Textbook comparison: - Array: O(1) push/pop, but fixed size - Linked list: O(1) push/pop, unlimited size

Reality:

```
$ perf stat -e cycles,cache-misses ./stack_benchmark
```

```
Array stack (1000 ops):
    12,000 cycles
        45 cache-misses

Linked list stack (1000 ops):
   450,000 cycles
    2,100 cache-misses
```

Linked list is 37× slower!

Why? 1. **malloc/free overhead**: Each push/pop calls allocator (~100 cycles) 2. **Cache misses**: Nodes scattered in memory 3. **Pointer chasing**: Each pop follows a pointer (cache miss)

When to use each: - **Array stack**: Almost always (embedded systems, performance-critical) - **Linked list stack**: When size truly unpredictable and memory is abundant

Queue: The Ring Buffer

Queues are trickier than stacks because you need to add at one end and remove from the other.

Naive array queue (bad):

```c
typedef struct {
    int data[MAX_SIZE];
    int front;
    int rear;
} queue_t;

void enqueue(queue_t *q, int value) {
    if (q->rear < MAX_SIZE) {
        q->data[q->rear++] = value;
    }
}

int dequeue(queue_t *q) {
    if (q->front < q->rear) {
        return q->data[q->front++];
    }
    return -1;  // Error
}
```

Problem: After many operations, `front` and `rear` reach `MAX_SIZE`, even if queue is empty.

```
Initial:  [_, _, _, _, _]  front=0, rear=0
Enqueue:  [1, 2, 3, _, _]  front=0, rear=3
```

```
Dequeue:  [_, 2, 3, _, _]  front=1, rear=3
Dequeue:  [_, _, 3, _, _]  front=2, rear=3
Enqueue:  [_, _, 3, 4, 5]  front=2, rear=5
Enqueue:  FULL!             front=2, rear=5 (but only 3 elements!)
```

Solution: Ring buffer (circular array)

```c
typedef struct {
    int data[MAX_SIZE];
    int head;
    int tail;
    int count;
} ring_buffer_t;

void enqueue(ring_buffer_t *q, int value) {
    if (q->count < MAX_SIZE) {
        q->data[q->tail] = value;
        q->tail = (q->tail + 1) % MAX_SIZE;
        q->count++;
    }
}

int dequeue(ring_buffer_t *q) {
    if (q->count > 0) {
        int value = q->data[q->head];
        q->head = (q->head + 1) % MAX_SIZE;
        q->count--;
        return value;
    }
    return -1;  // Error
}
```

How it works:

```
graph LR
    subgraph "Ring Buffer: Wraps Around"
        A["[0]"] --> B["[1]"] --> C["[2]"] --> D["[3]"] --> E["[4]"]
        E -.wraps.-> A
    end

    H[head] -.-> A
    T[tail] -.-> C

    style A fill:#90ee90
    style B fill:#90ee90
    style C fill:#fff
    style D fill:#fff
    style E fill:#fff
```

```
Initial:  [_, _, _, _, _]  head=0, tail=0, count=0
Enqueue:  [1, _, _, _, _]  head=0, tail=1, count=1
Enqueue:  [1, 2, _, _, _]  head=0, tail=2, count=2
Enqueue:  [1, 2, 3, _, _]  head=0, tail=3, count=3
Dequeue:  [_, 2, 3, _, _]  head=1, tail=3, count=2
Enqueue:  [_, 2, 3, 4, _]  head=1, tail=4, count=3
Enqueue:  [_, 2, 3, 4, 5]  head=1, tail=0, count=4  (tail wraps!)
Enqueue:  [6, 2, 3, 4, 5]  head=1, tail=1, count=5  (full)
```

Performance:

```
$ perf stat -e cycles ./queue_benchmark
Ring buffer (1M ops):
    15,000,000 cycles
         1,234 cache-misses

Linked list queue (1M ops):
   520,000,000 cycles
       980,000 cache-misses
```

Ring buffer is 35× faster!

Optimizing the Modulo Operation

The ring buffer has one performance issue: the modulo operation `% MAX_SIZE`.

On many processors (especially embedded), division/modulo is slow (10-40 cycles).

Optimization 1: Power-of-2 size

If `MAX_SIZE` is a power of 2, modulo becomes a bitwise AND:

```
#define MAX_SIZE 1024  // Must be power of 2
#define MASK (MAX_SIZE - 1)

void enqueue(ring_buffer_t *q, int value) {
    if (q->count < MAX_SIZE) {
        q->data[q->tail] = value;
        q->tail = (q->tail + 1) & MASK;  // Fast!
        q->count++;
    }
}
```

Benchmark:

```
Modulo version:     15,000,000 cycles
Bitwise AND version: 8,500,000 cycles  (1.76× faster)
```

Optimization 2: Eliminate count field

Instead of tracking `count`, use the fact that `head == tail` means empty:

```
typedef struct {
    int data[MAX_SIZE];
    int head;
    int tail;
} ring_buffer_t;

int is_empty(ring_buffer_t *q) {
    return q->head == q->tail;
}

int is_full(ring_buffer_t *q) {
    return ((q->tail + 1) & MASK) == q->head;
}

void enqueue(ring_buffer_t *q, int value) {
    if (!is_full(q)) {
        q->data[q->tail] = value;
        q->tail = (q->tail + 1) & MASK;
    }
}

int dequeue(ring_buffer_t *q) {
    if (!is_empty(q)) {
        int value = q->data[q->head];
        q->head = (q->head + 1) & MASK;
        return value;
    }
    return -1;
}
```

Trade-off: Wastes one slot (max capacity is `MAX_SIZE - 1`), but simpler and slightly faster.

Lock-Free Ring Buffer (Single Producer/Consumer)

On embedded systems with interrupts or multi-core, you often need thread-safe queues.

For **single producer, single consumer**, you can make a lock-free ring buffer:

```
typedef struct {
    volatile int data[MAX_SIZE];
    volatile int head;  // Only consumer writes
    volatile int tail;  // Only producer writes
} spsc_ring_buffer_t;
```

```c
// Producer (interrupt handler or other core)
void enqueue(spsc_ring_buffer_t *q, int value) {
    int next_tail = (q->tail + 1) & MASK;
    if (next_tail != q->head) {  // Not full
        q->data[q->tail] = value;
        __sync_synchronize();  // Memory barrier
        q->tail = next_tail;
    }
}

// Consumer (main thread)
int dequeue(spsc_ring_buffer_t *q) {
    if (q->head != q->tail) {  // Not empty
        int value = q->data[q->head];
        __sync_synchronize();  // Memory barrier
        q->head = (q->head + 1) & MASK;
        return value;
    }
    return -1;
}
```

Key points: - `volatile`: Prevents compiler from caching values - Memory barriers: Ensures ordering on weak memory models (ARM, RISC-V) - Single producer/consumer: No need for atomic operations

RISC-V version (explicit fence):

```c
void enqueue(spsc_ring_buffer_t *q, int value) {
    int next_tail = (q->tail + 1) & MASK;
    if (next_tail != q->head) {
        q->data[q->tail] = value;
        asm volatile("fence w, w" ::: "memory");  // Store-store fence
        q->tail = next_tail;
    }
}
```

Priority Queue: Binary Heap

Sometimes you need a queue where elements have priorities. The standard implementation is a **binary heap**.

Array-based binary heap:

```c
typedef struct {
    int data[MAX_SIZE];
    int size;
} heap_t;
```

```c
void heap_push(heap_t *h, int value) {
    if (h->size >= MAX_SIZE) return;

    // Insert at end
    int i = h->size++;
    h->data[i] = value;

    // Bubble up
    while (i > 0) {
        int parent = (i - 1) / 2;
        if (h->data[i] <= h->data[parent]) break;

        // Swap
        int temp = h->data[i];
        h->data[i] = h->data[parent];
        h->data[parent] = temp;

        i = parent;
    }
}

int heap_pop(heap_t *h) {
    if (h->size == 0) return -1;

    int result = h->data[0];

    // Move last element to root
    h->data[0] = h->data[--h->size];

    // Bubble down
    int i = 0;
    while (1) {
        int left = 2 * i + 1;
        int right = 2 * i + 2;
        int largest = i;

        if (left < h->size && h->data[left] > h->data[largest])
            largest = left;
        if (right < h->size && h->data[right] > h->data[largest])
            largest = right;

        if (largest == i) break;

        // Swap
        int temp = h->data[i];
        h->data[i] = h->data[largest];
```

```
        h->data[largest] = temp;

        i = largest;
    }

    return result;
}
```

Cache behavior: - Good: Array-based, sequential memory - Bad: Random access pattern during bubble up/down

Performance: O(log n) but with good cache behavior for small heaps.

Embedded Systems: Fixed-Size Queues

On embedded systems, **fixed-size queues** are the norm:

Why? 1. **Predictable memory**: No malloc/free 2. **Deterministic performance**: No allocation overhead 3. **Real-time safe**: No unbounded operations 4. **Simple**: Easier to verify and debug

Example: UART receive buffer

```
#define UART_BUFFER_SIZE 256  // Power of 2

typedef struct {
    uint8_t data[UART_BUFFER_SIZE];
    volatile uint16_t head;
    volatile uint16_t tail;
} uart_buffer_t;

uart_buffer_t uart_rx_buffer = {0};

// Called from UART interrupt
void uart_rx_isr(void) {
    uint8_t byte = UART_DATA_REG;

    uint16_t next_tail = (uart_rx_buffer.tail + 1) & (UART_BUFFER_SIZE - 1);
    if (next_tail != uart_rx_buffer.head) {
        uart_rx_buffer.data[uart_rx_buffer.tail] = byte;
        uart_rx_buffer.tail = next_tail;
    } else {
        // Buffer full, drop byte (or set error flag)
    }
}

// Called from main loop
int uart_read(void) {
```

```
    if (uart_rx_buffer.head == uart_rx_buffer.tail) {
        return -1;  // Empty
    }

    uint8_t byte = uart_rx_buffer.data[uart_rx_buffer.head];
    uart_rx_buffer.head = (uart_rx_buffer.head + 1) & (UART_BUFFER_SIZE - 1);
    return byte;
}
```

Key features: - Fixed size (256 bytes) - Power-of-2 for fast modulo - Lock-free (single producer/consumer) - ISR-safe (volatile, memory barriers implicit in ISR)

Real-World Example: Task Scheduler

Back to my firmware crash. Here's the before and after:

Before (linked list queue):

```
typedef struct task {
    void (*func)(void);
    struct task *next;
} task_t;

task_t *task_queue = NULL;

void schedule_task(void (*func)(void)) {
    task_t *task = malloc(sizeof(task_t));  // Slow, fragmentation
    task->func = func;
    task->next = NULL;

    // Add to end of queue
    if (!task_queue) {
        task_queue = task;
    } else {
        task_t *curr = task_queue;
        while (curr->next) curr = curr->next;  // O(n) traversal!
        curr->next = task;
    }
}

void run_tasks(void) {
    while (task_queue) {
        task_t *task = task_queue;
        task_queue = task->next;
        task->func();
        free(task);  // Slow
```

```
    }
}
```

Problems: - malloc/free in ISR (bad practice) - O(n) enqueue (traverses entire list) - Memory fragmentation - Unpredictable performance

After (ring buffer):

```
#define MAX_TASKS 32

typedef struct {
    void (*funcs[MAX_TASKS])(void);
    volatile uint8_t head;
    volatile uint8_t tail;
} task_queue_t;

task_queue_t task_queue = {0};

void schedule_task(void (*func)(void)) {
    uint8_t next_tail = (task_queue.tail + 1) & (MAX_TASKS - 1);
    if (next_tail != task_queue.head) {
        task_queue.funcs[task_queue.tail] = func;
        task_queue.tail = next_tail;
    }
    // If full, task is dropped (could set error flag)
}

void run_tasks(void) {
    while (task_queue.head != task_queue.tail) {
        void (*func)(void) = task_queue.funcs[task_queue.head];
        task_queue.head = (task_queue.head + 1) & (MAX_TASKS - 1);
        func();
    }
}
```

Improvements: - No malloc/free - O(1) enqueue and dequeue - Fixed memory (128 bytes) - Predictable performance - ISR-safe

Result: No more crashes, 10× faster task scheduling.

Summary

The firmware crash from "stack overflow" was actually a queue problem. The linked list queue's dynamic allocation fragmented the memory pool that shared space with the stack. Replacing it with a fixed-size ring buffer eliminated the crashes and made task scheduling 10× faster. The invisible data structure became visible through its failure.

Stacks: - Array-based: Fast, fixed size, cache-friendly - Linked list: Slow (malloc/free), unlimited size - **Recommendation**: Use array-based unless size truly unpredictable

Queues: - Ring buffer: Fast, fixed size, cache-friendly - Linked list: Slow, unlimited size - **Recommendation**: Use ring buffer, especially on embedded systems

Optimizations: - Power-of-2 size for fast modulo (bitwise AND) - Lock-free for single producer/consumer - Eliminate count field (trade one slot for simplicity)

Embedded considerations: - Fixed-size queues (predictable memory) - No malloc/free (deterministic, real-time safe) - ISR-safe (volatile, memory barriers) - Power-of-2 sizes (fast operations)

Priority queues: - Binary heap: O(log n), array-based, good cache behavior - Use for small to medium heaps (< 10K elements)

Next Chapter: Hash tables combine the speed of arrays with the flexibility of dynamic structures—but cache conflicts can destroy performance. We'll explore how to build cache-friendly hash tables.

Chapter 7: Hash Tables and Cache Conflicts

Part II: Basic Data Structures

"Hash tables are the duct tape of data structures." — Steve Yegge

The O(1) Myth

Hash tables promise O(1) lookup—constant time, regardless of size. In theory, they're perfect.

In practice, I've seen hash tables perform worse than linear search through an array.

I was optimizing a symbol table for a compiler. The symbol table used a hash table with 1024 buckets, and we had about 500 symbols. The math looked good: average bucket size = 500/1024 0.5, so most lookups should be one probe.

But the profiler told a different story:

```
$ perf stat -e cache-misses,instructions ./compiler
  Performance counter stats:
    1,234,567 cache-misses
    5,000,000 instructions
```

1.2 million cache misses for 5 million instructions? For a hash table that should be O(1)?

The problem was **cache conflicts**. The hash table was large (1024 buckets × 8 bytes = 8 KB), and the access pattern was causing cache line conflicts. Every lookup was a cache miss.

I replaced it with a simple linear search through a 500-element array. **Result: 3× faster**.

This chapter is about understanding when hash tables are fast, when they're slow, and how to make them cache-friendly.

Hash Table Basics

A hash table maps keys to values using a hash function:

```
typedef struct {
    char *key;
    int value;
} entry_t;

#define TABLE_SIZE 1024

entry_t *table[TABLE_SIZE];

int hash(const char *key) {
    unsigned int h = 0;
    while (*key) {
        h = h * 31 + *key++;
    }
    return h % TABLE_SIZE;
}

void insert(const char *key, int value) {
    int index = hash(key);
    entry_t *entry = malloc(sizeof(entry_t));
    entry->key = strdup(key);
    entry->value = value;
    table[index] = entry;
}

int lookup(const char *key) {
    int index = hash(key);
    entry_t *entry = table[index];
    if (entry && strcmp(entry->key, key) == 0) {
        return entry->value;
    }
    return -1;  // Not found
}
```

This is a **direct-mapped** hash table (one entry per bucket). It doesn't handle collisions.

Collision Resolution

When two keys hash to the same index, you have a **collision**. Two main strategies:

```
flowchart TD
    subgraph Chaining["Chaining: Linked Lists"]
        direction LR
        subgraph Row0[" "]
            direction TB
            T0["Table[0]"] --> E0A["Entry A"] --> E0B["Entry B"] --> E0C["Entry C"]
        end
        subgraph Row1[" "]
            direction TB
            T1["Table[1]"] --> E1A["Entry D"]
        end
        subgraph Row2[" "]
            direction TB
            T2["Table[2]"] --> E2A["NULL"]
        end
        subgraph Row3[" "]
            direction TB
            T3["Table[3]"] --> E3A["Entry E"] --> E3B["Entry F"]
        end
        Row0 ~~~ Row1
        Row1 ~~~ Row2
        Row2 ~~~ Row3
    end

    subgraph OpenAddr["Open Addressing: Linear Probing"]
        direction LR
        O1["[0] Entry A"]
        O2["[1] Entry B"]
        O3["[2] Empty"]
        O4["[3] Entry C"]
        O5["[4] Entry D"]
        O6["[5] Empty"]
    end

    style E0A fill:#ffcccb
    style E0B fill:#ffcccb
    style E0C fill:#ffcccb
    style E1A fill:#ffcccb
```

```
    style E3A fill:#ffcccb
    style E3B fill:#ffcccb
    style O1 fill:#90ee90
    style O2 fill:#90ee90
    style O4 fill:#90ee90
    style O5 fill:#90ee90
```

1. Chaining (linked list per bucket):

```c
typedef struct entry {
    char *key;
    int value;
    struct entry *next;
} entry_t;

entry_t *table[TABLE_SIZE];

void insert(const char *key, int value) {
    int index = hash(key);

    entry_t *entry = malloc(sizeof(entry_t));
    entry->key = strdup(key);
    entry->value = value;
    entry->next = table[index];
    table[index] = entry;
}

int lookup(const char *key) {
    int index = hash(key);
    entry_t *entry = table[index];

    while (entry) {
        if (strcmp(entry->key, key) == 0) {
            return entry->value;
        }
        entry = entry->next;
    }
    return -1;  // Not found
}
```

2. Open addressing (probe for next empty slot):

```c
typedef struct {
    char *key;
    int value;
    int occupied;
} entry_t;
```

```
entry_t table[TABLE_SIZE];

void insert(const char *key, int value) {
    int index = hash(key);

    // Linear probing
    while (table[index].occupied) {
        index = (index + 1) % TABLE_SIZE;
    }

    table[index].key = strdup(key);
    table[index].value = value;
    table[index].occupied = 1;
}

int lookup(const char *key) {
    int index = hash(key);

    while (table[index].occupied) {
        if (strcmp(table[index].key, key) == 0) {
            return table[index].value;
        }
        index = (index + 1) % TABLE_SIZE;
    }
    return -1;  // Not found
}
```

Textbook comparison: - Chaining: Handles any load factor, but uses extra memory (pointers) - Open addressing: No extra memory, but degrades at high load factor

Cache perspective: - Chaining: **Terrible** (pointer chasing, scattered allocations) - Open addressing: **Better** (sequential probing, array-based)

The Cache Conflict Problem

Let's analyze cache behavior for a hash table lookup.

Chaining (worst case):

```
int lookup(const char *key) {
    int index = hash(key);           // 1. Compute hash
    entry_t *entry = table[index];   // 2. Load bucket pointer (cache miss)

    while (entry) {
        if (strcmp(entry->key, key) == 0) {  // 3. Load entry (cache miss)
            return entry->value;              // 4. Load key (cache miss)
        }
```

```
        entry = entry->next;                    // 5. Follow pointer (cache miss)
    }
    return -1;
}
```

Cache misses per lookup: - Bucket pointer: 1 miss - Each entry in chain: 2-3 misses (entry, key, possibly next) - **Total**: 3-10 misses for a chain of length 3

Open addressing (linear probing):

```
int lookup(const char *key) {
    int index = hash(key);

    while (table[index].occupied) {           // Sequential access
        if (strcmp(table[index].key, key) == 0) {
            return table[index].value;
        }
        index = (index + 1) % TABLE_SIZE;
    }
    return -1;
}
```

Cache misses: - First probe: 1 miss (loads cache line with ~8 entries) - Next 7 probes: 0 misses (same cache line) - **Total**: 1-2 misses for typical lookup

Open addressing is 3-5× fewer cache misses.

Benchmark: Chaining vs Open Addressing

Let's measure the difference:

```
// Test: 1000 insertions, 10000 lookups
// Load factor: 0.5 (1000 entries, 2048 buckets)

Chaining:
  Insert: 450,000 cycles
  Lookup: 2,100,000 cycles
  Cache misses: 45,000

Open addressing (linear probing):
  Insert: 180,000 cycles
  Lookup: 650,000 cycles
  Cache misses: 12,000
```

Open addressing is 3.2× faster with 3.75× fewer cache misses.

Hash Function Quality

A good hash function is critical. A bad hash function causes clustering, which destroys performance.

Bad hash function (poor distribution):

```
int bad_hash(const char *key) {
    return key[0] % TABLE_SIZE;  // Only uses first character!
}
```

Result: All keys starting with 'a' collide, all keys starting with 'b' collide, etc.

Better hash function (FNV-1a):

```
uint32_t fnv1a_hash(const char *key) {
    uint32_t hash = 2166136261u;
    while (*key) {
        hash ^= (uint8_t)*key++;
        hash *= 16777619u;
    }
    return hash;
}
```

Even better (for integers, identity hash):

```
uint32_t int_hash(uint32_t key) {
    // For sequential integers, identity is perfect
    return key;
}
```

For pointers (multiply by odd number):

```
uint32_t ptr_hash(void *ptr) {
    uintptr_t p = (uintptr_t)ptr;
    // Pointers are often aligned, so shift and multiply
    return (uint32_t)((p >> 3) * 2654435761u);
}
```

Benchmark (1000 random strings):

```
Bad hash (first char):     Avg chain length: 38.5
Simple hash (sum):         Avg chain length: 2.1
FNV-1a:                    Avg chain length: 0.98
```

Good hash function reduces collisions by 40×.

Load Factor and Resizing

Load factor = number of entries / table size

Chaining: Can exceed 1.0, but performance degrades **Open addressing**: Must stay below 0.7-0.8 or performance collapses

Why? As table fills, probe sequences get longer:

```
Load factor 0.5:  Avg probes = 1.5
Load factor 0.7:  Avg probes = 3.6
Load factor 0.9:  Avg probes = 10.5
Load factor 0.95: Avg probes = 20.5
```

Solution: Resize when load factor exceeds threshold

```c
void resize_table(void) {
    int old_size = table_size;
    entry_t *old_table = table;

    table_size *= 2;
    table = calloc(table_size, sizeof(entry_t));

    // Rehash all entries
    for (int i = 0; i < old_size; i++) {
        if (old_table[i].occupied) {
            insert(old_table[i].key, old_table[i].value);
        }
    }

    free(old_table);
}

void insert(const char *key, int value) {
    if (count >= table_size * 0.7) {
        resize_table();
    }

    // ... normal insert ...
}
```

Cost: Resizing is O(n), but amortized O(1) if you double the size.

Cache-Friendly Hash Table Design

Here's a cache-optimized hash table design:

1. Use open addressing (linear probing)

2. Pack entries tightly

```c
typedef struct {
    uint32_t hash;   // Store hash to avoid recomputing
    uint32_t key;    // Assume integer keys
    uint32_t value;
} entry_t;  // 12 bytes, fits 5 per cache line
```

3. Use power-of-2 size (fast modulo)

```
#define TABLE_SIZE 2048
#define MASK (TABLE_SIZE - 1)

int index = hash & MASK;  // Fast!
```

4. Separate keys and values (if values are large)

```
typedef struct {
    uint32_t keys[TABLE_SIZE];
    uint32_t hashes[TABLE_SIZE];
    value_t *values[TABLE_SIZE];  // Pointers to large values
} hash_table_t;
```

Why? Probing only touches keys and hashes, not large values.

5. Use SIMD for probing (advanced)

```
// Check 8 entries at once using AVX2
__m256i target = _mm256_set1_epi32(hash);
__m256i entries = _mm256_loadu_si256((__m256i*)&table[index]);
__m256i cmp = _mm256_cmpeq_epi32(target, entries);
int mask = _mm256_movemask_epi8(cmp);
if (mask) {
    int pos = __builtin_ctz(mask) / 4;
    return table[index + pos].value;
}
```

Robin Hood Hashing

Robin Hood hashing is a variant of linear probing that reduces variance in probe lengths.

Idea: When inserting, if the probe distance of the existing entry is less than yours, swap and continue inserting the displaced entry.

Decision process:

```
flowchart TD
    Start["Insert key (hash=H)"] --> Try["Try index = H + probe_dist"]
    Try --> Check{"Slot<br/>occupied?"}
    Check -->|No| Insert["  Insert here"]
    Check -->|Yes| Compare{"My probe_dist ><br/>existing probe_dist?"}
    Compare -->|" "| Probe["probe_dist++<br/>Try next slot"]
    Compare -->|">"| Swap["  SWAP!<br/>Take slot<br/>Continue with displaced"]
    Probe --> Try
    Swap --> Try

    style Swap fill:#ffeb3b
```

```
    style Insert fill:#90ee90
    style Probe fill:#e3f2fd
```

Example walkthrough:

```
Initial state:

  [0]   Empty      dist: -
  [1]   key1       dist: 0    (hash=1, ideal position)
  [2]   key2       dist: 1    (hash=1, probed 1 step)
  [3]   Empty      dist: -
  [4]   Empty      dist: -


Insert key3 (hash=2):
  Try [2]: occupied by key2
    key3 probe_dist = 0
    key2 probe_dist = 1
    0   1 → Continue probing
  Try [3]: Empty → Insert

After key3:

  [1]   key1       dist: 0
  [2]   key2       dist: 1
  [3]   key3       dist: 1


Insert key4 (hash=1):
  Try [1]: occupied by key1
    key4 probe_dist = 0, key1 probe_dist = 0 → Continue
  Try [2]: occupied by key2
    key4 probe_dist = 1, key2 probe_dist = 1 → Continue
  Try [3]: occupied by key3
    key4 probe_dist = 2, key3 probe_dist = 1
    2 > 1 → SWAP! (Robin Hood: take from rich, give to poor)

  After swap, continue inserting displaced key3:
  Try [4]: Empty → Insert key3

Final state:

  [1]   key1       dist: 0
  [2]   key2       dist: 1
  [3]   key4       dist: 2    ← Swapped in
  [4]   key3       dist: 2    ← Displaced, reinserted
```

```
Result: More uniform probe distances (max=2 instead of potentially unbounded)
```

```
void insert(uint32_t key, uint32_t value) {
    uint32_t hash = hash_func(key);
    int index = hash & MASK;
    int probe_dist = 0;

    entry_t entry = {hash, key, value};

    while (1) {
        if (!table[index].occupied) {
            table[index] = entry;
            table[index].occupied = 1;
            return;
        }

        int existing_dist = (index - table[index].hash) & MASK;
        if (probe_dist > existing_dist) {
            // Swap: we've probed further than existing entry
            entry_t temp = table[index];
            table[index] = entry;
            entry = temp;
            probe_dist = existing_dist;
        }

        index = (index + 1) & MASK;
        probe_dist++;
    }
}
```

Benefit: More uniform probe lengths, better worst-case performance.

Benchmark:

```
Linear probing:     Avg: 1.5 probes, Max: 12 probes
Robin Hood hashing: Avg: 1.5 probes, Max: 4 probes
```

Better worst-case (important for real-time systems).

Small Hash Tables: Just Use Arrays

For small tables (< 100 entries), linear search through an array is often faster than hashing.

Why? - Hash computation cost - Modulo operation cost - Potential cache misses

Benchmark (50 entries):

```
Hash table (open addressing): 850 cycles per lookup
Linear search (array):        420 cycles per lookup
```

Linear search is 2× faster for small tables!

Guideline: Use linear search for < 50-100 entries, hash table for larger.

Embedded Systems: Perfect Hashing

On embedded systems, you often know all keys at compile time (e.g., command names, register names). You can use **perfect hashing**—a hash function with zero collisions.

Example: Command parser with 16 commands

```
// Commands: "read", "write", "reset", "status", ...
// Generate perfect hash function at compile time

const char *commands[] = {
    "read", "write", "reset", "status",
    "start", "stop", "config", "debug",
    // ... 16 total
};

// Perfect hash function (generated by gperf or manual)
int command_hash(const char *cmd) {
    // Carefully chosen to have zero collisions
    return (cmd[0] * 3 + cmd[1] * 7) & 15;
}

void (*handlers[16])(void) = {
    [command_hash("read")] = handle_read,
    [command_hash("write")] = handle_write,
    // ...
};

void dispatch_command(const char *cmd) {
    int index = command_hash(cmd);
    if (strcmp(commands[index], cmd) == 0) {
        handlers[index]();
    }
}
```

Benefits: - Zero collisions (guaranteed O(1)) - No probing - Minimal memory - Fast (one hash, one comparison)

Tools: `gperf` generates perfect hash functions from keyword lists.

Real-World Example: Symbol Table Optimization

Back to my compiler symbol table. Here's what I changed:

```
BEFORE: Hash Table with Chaining


 Hash Table [1024 buckets]

   [0]  → NULL
   [1]  → Symbol("foo") → Symbol("bar") → NULL
   [2]  → NULL
   [3]  → Symbol("baz") → NULL
  ...


 Lookup operations:
 1. Hash computation (31 * n)
 2. Modulo operation (expensive)
 3. Pointer chasing (cache miss)
 4. String comparison (pointer dereference)

 Performance: 2,400 cycles/lookup



AFTER: Linear Search Array


 Array [256 max symbols per scope]

   [0] Symbol { name: "foo", type, offset }
   [1] Symbol { name: "bar", type, offset }
   [2] Symbol { name: "baz", type, offset }
   [3] ...
       (sequential in memory)


 Lookup operations:
 1. Sequential scan (cache-friendly)
 2. String comparison (inline data, no pointer)

 Performance: 380 cycles/lookup (6.3× faster!)
```

```
Why it works:
  Small scope (< 256 symbols per function)
  Sequential access (prefetcher helps)
  Inline strings (no pointer chasing)
  No malloc/free overhead
  Cache-friendly (entire array fits in L1)
```

Before (hash table with chaining):

```
#define TABLE_SIZE 1024

typedef struct symbol {
    char *name;
    int type;
    int offset;
    struct symbol *next;
} symbol_t;

symbol_t *symbol_table[TABLE_SIZE];

symbol_t *lookup_symbol(const char *name) {
    int index = hash(name) % TABLE_SIZE;
    symbol_t *sym = symbol_table[index];

    while (sym) {
        if (strcmp(sym->name, name) == 0) {
            return sym;
        }
        sym = sym->next;
    }
    return NULL;
}
```

After (linear search for small scopes):

```
#define MAX_SYMBOLS 256

typedef struct {
    char name[32];  // Inline, not pointer
    int type;
    int offset;
} symbol_t;

symbol_t symbols[MAX_SYMBOLS];
int symbol_count = 0;

symbol_t *lookup_symbol(const char *name) {
    // Linear search (cache-friendly)
```

```
    for (int i = 0; i < symbol_count; i++) {
        if (strcmp(symbols[i].name, name) == 0) {
            return &symbols[i];
        }
    }
    return NULL;
}
```

Changes: - Removed hash table (< 256 symbols per scope) - Inline names (no pointer chasing) - Array-based (sequential access) - No malloc/free

Results: - 3× faster lookups - 10× fewer cache misses - Simpler code - Predictable performance

Lesson: For small datasets, simple beats clever.

Summary

The O(1) myth was exposed. The hash table with 1024 buckets and 500 symbols should have been fast, but 1.2 million cache misses for 5 million instructions told a different story. Cache conflicts from the 8 KB table made every lookup a cache miss. Replacing it with linear search through a 500-element array delivered 3× better performance. Constant-time complexity meant nothing when every operation missed the cache.

Key insights: - Chaining: Terrible cache behavior (pointer chasing) - Open addressing: Much better (sequential probing) - Hash function quality matters (avoid clustering) - Load factor affects performance (keep < 0.7 for open addressing) - Small tables: Linear search often faster

Cache-friendly design: - Use open addressing (linear probing or Robin Hood) - Pack entries tightly (12-16 bytes per entry) - Power-of-2 size (fast modulo) - Separate keys and large values - Consider SIMD for probing

Embedded considerations: - Perfect hashing for known keys - Linear search for small tables (< 100 entries) - Fixed-size tables (no resizing) - Inline keys (avoid pointers)

When to use hash tables: - Large datasets (> 100 entries) - Need O(1) average case - Keys are well-distributed - Can tolerate occasional resize

When NOT to use hash tables: - Small datasets (< 100 entries) → use array - Need guaranteed O(1) → use perfect hashing - Need sorted iteration → use tree - Tight memory budget → use array

Next Chapter: Dynamic arrays (vectors) combine the cache-friendliness of arrays with the flexibility of dynamic sizing. We'll explore how to implement them efficiently and when resizing becomes a bottleneck.

Chapter 8: Dynamic Arrays and Memory Management

Part II: Basic Data Structures

> "Premature optimization is the root of all evil, but so is premature pessimization." — Andrei Alexandrescu

The Reallocation Problem

Dynamic arrays (vectors in C++, ArrayList in Java) are one of the most useful data structures. They combine the cache-friendliness of arrays with the flexibility of dynamic sizing.

But there's a hidden cost: **reallocation**.

I was working on a log aggregator for an embedded system. The system collected log messages in a dynamic array and periodically flushed them to flash storage. Simple, right?

The performance was terrible. The system was spending 60% of its time in `realloc()`.

The problem? The array was growing one element at a time:

```
typedef struct {
    char **messages;
    int size;
    int capacity;
} log_buffer_t;

void add_message(log_buffer_t *buf, const char *msg) {
    if (buf->size >= buf->capacity) {
        buf->capacity++;  // Grow by 1!
        buf->messages = realloc(buf->messages,
                                buf->capacity * sizeof(char*));
    }
    buf->messages[buf->size++] = strdup(msg);
}
```

For 1000 messages: 1000 reallocations, each copying the entire array.

Total copies: $1 + 2 + 3 + \ldots + 1000 = 500{,}500$ elements copied!

The fix was simple: **grow exponentially**, not linearly.

```
void add_message(log_buffer_t *buf, const char *msg) {
    if (buf->size >= buf->capacity) {
        buf->capacity = buf->capacity ? buf->capacity * 2 : 16;
```

```
        buf->messages = realloc(buf->messages,
                                buf->capacity * sizeof(char*));
    }
    buf->messages[buf->size++] = strdup(msg);
}
```

For 1000 messages: 7 reallocations (16, 32, 64, 128, 256, 512, 1024).

Total copies: ~2000 elements (vs 500,500).

Result: 250× fewer copies, 60× faster.

Visualizing exponential growth:

```
Initial state:

 Size: 0, Cap: 0


Add 1st element → Allocate initial capacity

 Size: 1, Cap: 16    Allocated


Add elements 2-16 → No reallocation

 Size: 16, Cap: 16    Full


Add 17th element → Realloc (16 → 32)

 Size: 17, Cap: 32    Reallocated (copy 16 elements)


Add elements 18-32 → No reallocation

 Size: 32, Cap: 32    Full


Add 33rd element → Realloc (32 → 64)

 Size: 33, Cap: 64    Reallocated (copy 32 elements)


Continue...

 Size: 64, Cap: 64    → Realloc to 128
 Size: 128, Cap: 128  → Realloc to 256
```

```
  Size: 256, Cap: 256  → Realloc to 512
  Size: 512, Cap: 512  → Realloc to 1024


Final state (1000 elements):

  Size: 1000, Cap: 1024   Only ~10 reallocations total


Total reallocations: 10 (vs 1000 if growing by 1 each time)
Total copies: ~2000 elements (vs 500,500)
```

Real-world applications of this strategy:

This "allocate extra space to avoid frequent expensive operations" pattern appears in many systems:

- **String builders** (Java `StringBuilder`, C# `StringBuilder`): Grow exponentially to avoid $O(n^2)$ string concatenation
- **Network buffers** (TCP receive buffers): Pre-allocate larger buffers to reduce system calls
- **Memory allocators** (malloc implementations): Use size classes (16, 32, 64, 128…) to reduce fragmentation
- **Database transaction logs**: Pre-allocate log space in chunks to avoid frequent disk I/O
- **Sparse matrices** (scientific computing): Allocate extra capacity in compressed row storage to allow efficient insertions
- **File systems** (ext4, XFS): Pre-allocate blocks for growing files to reduce fragmentation

The key insight: **Trading space for time** by over-allocating reduces the amortized cost of growth from O(n) to O(1).

Dynamic Array Implementation

Here's a complete dynamic array implementation:

```c
typedef struct {
    int *data;
    size_t size;      // Number of elements
    size_t capacity;  // Allocated space
} vector_t;

void vector_init(vector_t *v) {
    v->data = NULL;
    v->size = 0;
    v->capacity = 0;
}
```

```c
void vector_free(vector_t *v) {
    free(v->data);
    v->data = NULL;
    v->size = 0;
    v->capacity = 0;
}

void vector_push(vector_t *v, int value) {
    if (v->size >= v->capacity) {
        size_t new_capacity = v->capacity ? v->capacity * 2 : 16;
        int *new_data = realloc(v->data, new_capacity * sizeof(int));
        if (!new_data) {
            // Handle allocation failure
            return;
        }
        v->data = new_data;
        v->capacity = new_capacity;
    }
    v->data[v->size++] = value;
}

int vector_pop(vector_t *v) {
    if (v->size > 0) {
        return v->data[--v->size];
    }
    return -1;  // Error
}

int vector_get(vector_t *v, size_t index) {
    if (index < v->size) {
        return v->data[index];
    }
    return -1;  // Error
}
```

Key design choices: - Initial capacity: 16 (avoid tiny allocations) - Growth factor: 2× (exponential growth) - `realloc()`: May avoid copy if space available

Growth Factor Analysis

The growth factor affects both memory usage and performance.

Common growth factors: - **1.5×**: Used by some implementations (e.g., Facebook's folly) - **2×**: Most common (C++ std::vector, Python list) - **(1.618)**: Golden ratio, theoretical optimum

Trade-offs:

2× growth: - Pros: Simple, fast (bit shift), good amortized performance - Cons: Can waste up to 50% memory

1.5× growth: - Pros: Less memory waste (~33%), better memory reuse - Cons: More reallocations, slightly slower

Benchmark (growing to 1M elements):

```
Growth factor 1.5×:
  Reallocations: 34
  Peak memory: 1.5 MB
  Time: 12 ms

Growth factor 2×:
  Reallocations: 20
  Peak memory: 2 MB
  Time: 8 ms
```

2× is faster (fewer reallocations) but uses more memory.

Recommendation: Use 2× unless memory is very tight.

Shrinking: When to Deallocate

Should you shrink the array when elements are removed?

Naive approach: Shrink on every pop

```c
void vector_pop(vector_t *v) {
    if (v->size > 0) {
        v->size--;
        if (v->size < v->capacity / 2) {
            v->capacity /= 2;
            v->data = realloc(v->data, v->capacity * sizeof(int));
        }
    }
}
```

Problem: Thrashing if you push/pop around the threshold

```
Push to 1024 → capacity 1024
Pop to 512   → shrink to 512
Push to 513  → grow to 1024
Pop to 512   → shrink to 512
...
```

Better approach: Hysteresis (shrink at 1/4, not 1/2)

```c
void vector_pop(vector_t *v) {
    if (v->size > 0) {
```

```
        v->size--;
        if (v->size < v->capacity / 4 && v->capacity > 16) {
            v->capacity /= 2;
            v->data = realloc(v->data, v->capacity * sizeof(int));
        }
    }
}
```

Now: Must pop to 256 before shrinking from 1024.

Even better: Don't shrink automatically, provide explicit `vector_shrink_to_fit()`.

```
void vector_shrink_to_fit(vector_t *v) {
    if (v->size < v->capacity) {
        v->data = realloc(v->data, v->size * sizeof(int));
        v->capacity = v->size;
    }
}
```

Recommendation: Don't auto-shrink unless memory is critical.

Reserve and Capacity

If you know the final size in advance, **reserve** space upfront:

```
void vector_reserve(vector_t *v, size_t capacity) {
    if (capacity > v->capacity) {
        int *new_data = realloc(v->data, capacity * sizeof(int));
        if (new_data) {
            v->data = new_data;
            v->capacity = capacity;
        }
    }
}

// Usage
vector_t v;
vector_init(&v);
vector_reserve(&v, 1000);  // Allocate once

for (int i = 0; i < 1000; i++) {
    vector_push(&v, i);  // No reallocation!
}
```

Benchmark (1000 elements):

```
Without reserve:
  Reallocations: 7
  Time: 45  s
```

```
With reserve:
  Reallocations: 1
  Time: 12  s
```

3.75× faster by avoiding reallocations.

Guideline: If you know the size, always reserve.

Small Vector Optimization (SVO)

For small vectors, the overhead of heap allocation dominates.

Small Vector Optimization: Store small arrays inline, only allocate for large arrays.

```c
#define SMALL_SIZE 16

typedef struct {
    int small_data[SMALL_SIZE];  // Inline storage
    int *data;                    // Heap storage (if needed)
    size_t size;
    size_t capacity;
} small_vector_t;

void small_vector_init(small_vector_t *v) {
    v->data = v->small_data;  // Start with inline storage
    v->size = 0;
    v->capacity = SMALL_SIZE;
}

void small_vector_push(small_vector_t *v, int value) {
    if (v->size >= v->capacity) {
        size_t new_capacity = v->capacity * 2;
        int *new_data = malloc(new_capacity * sizeof(int));

        // Copy from inline or heap storage
        memcpy(new_data, v->data, v->size * sizeof(int));

        // Free old heap storage (if any)
        if (v->data != v->small_data) {
            free(v->data);
        }

        v->data = new_data;
        v->capacity = new_capacity;
    }
    v->data[v->size++] = value;
```

```
}

void small_vector_free(small_vector_t *v) {
    if (v->data != v->small_data) {
        free(v->data);
    }
}
```

Benefits: - No allocation for small vectors (16 elements) - Better cache locality (data inline with struct) - Faster for common case

Cost: - Larger struct size (64 bytes vs 16 bytes) - One extra copy when transitioning to heap

Benchmark (average size: 8 elements):

```
Regular vector:
  Allocations: 1 per vector
  Time: 850 ns per vector

Small vector:
  Allocations: 0 (inline)
  Time: 120 ns per vector
```

7× faster for small vectors.

Recommendation: Use SVO for vectors that are usually small (< 16-32 elements).

Memory Allocator Considerations

`realloc()` performance depends on the allocator.

Best case: Allocator can expand in place (no copy)

```
// Allocate 1 KB
void *ptr = malloc(1024);

// Expand to 2 KB (may expand in place)
ptr = realloc(ptr, 2048);  // No copy if space available
```

Worst case: Must allocate new block and copy

```
// Allocate 1 KB
void *ptr = malloc(1024);

// Another allocation uses adjacent space
void *other = malloc(1024);

// Now realloc must copy
ptr = realloc(ptr, 2048);  // Must allocate new block and copy
```

Embedded systems: Often use simple allocators that can't expand in place.

Solution: Use custom allocator or memory pool

```c
typedef struct {
    char pool[64 * 1024];  // 64 KB pool
    size_t used;
} memory_pool_t;

memory_pool_t global_pool = {0};

void *pool_alloc(size_t size) {
    if (global_pool.used + size > sizeof(global_pool.pool)) {
        return NULL;  // Out of memory
    }
    void *ptr = &global_pool.pool[global_pool.used];
    global_pool.used += size;
    return ptr;
}

// Can't free individual allocations, but can reset entire pool
void pool_reset(void) {
    global_pool.used = 0;
}
```

Use case: Temporary vectors that are freed together.

Insertion and Deletion

Inserting or deleting in the middle requires shifting elements.

Insert at index:

```c
void vector_insert(vector_t *v, size_t index, int value) {
    if (index > v->size) return;

    // Ensure capacity
    if (v->size >= v->capacity) {
        size_t new_capacity = v->capacity ? v->capacity * 2 : 16;
        v->data = realloc(v->data, new_capacity * sizeof(int));
        v->capacity = new_capacity;
    }

    // Shift elements right
    memmove(&v->data[index + 1], &v->data[index],
            (v->size - index) * sizeof(int));

    v->data[index] = value;
```

```
    v->size++;
}
```

Delete at index:

```
void vector_delete(vector_t *v, size_t index) {
    if (index >= v->size) return;

    // Shift elements left
    memmove(&v->data[index], &v->data[index + 1],
            (v->size - index - 1) * sizeof(int));

    v->size--;
}
```

Performance: - Insert/delete at end: O(1) - Insert/delete at beginning: O(n) (must shift all elements) - Insert/delete in middle: O(n)

Benchmark (1000 elements):

```
Insert at end:       50 ns
Insert at beginning: 12,000 ns  (240× slower)
Insert in middle:    6,000 ns   (120× slower)
```

Guideline: If you need frequent insertions/deletions in the middle, consider a different data structure (e.g., linked list, gap buffer).

Embedded Systems: Fixed-Capacity Vectors

On embedded systems, you often can't afford dynamic allocation.

Fixed-capacity vector:

```
#define MAX_CAPACITY 256

typedef struct {
    int data[MAX_CAPACITY];
    size_t size;
} fixed_vector_t;

void fixed_vector_init(fixed_vector_t *v) {
    v->size = 0;
}

int fixed_vector_push(fixed_vector_t *v, int value) {
    if (v->size >= MAX_CAPACITY) {
        return -1;  // Full
    }
    v->data[v->size++] = value;
```

```
    return 0;
}
```

Benefits: - No allocation - Predictable memory usage - Fast (no reallocation) - Simple

Cost: - Fixed maximum size - May waste memory if not full

Recommendation: Use fixed-capacity vectors on embedded systems unless you truly need dynamic sizing.

Real-World Example: Log Buffer Optimization

Back to my log aggregator. Here's the complete optimization:

Before (grow by 1):

```
typedef struct {
    char **messages;
    int size;
    int capacity;
} log_buffer_t;

void add_message(log_buffer_t *buf, const char *msg) {
    if (buf->size >= buf->capacity) {
        buf->capacity++;
        buf->messages = realloc(buf->messages,
                                buf->capacity * sizeof(char*));
    }
    buf->messages[buf->size++] = strdup(msg);
}
```

Problems: - 1000 reallocations for 1000 messages - 500,500 elements copied - Terrible performance

After (exponential growth + reserve):

```
typedef struct {
    char **messages;
    int size;
    int capacity;
} log_buffer_t;

void log_buffer_init(log_buffer_t *buf, int expected_size) {
    buf->messages = malloc(expected_size * sizeof(char*));
    buf->size = 0;
    buf->capacity = expected_size;
}

void add_message(log_buffer_t *buf, const char *msg) {
```

```
    if (buf->size >= buf->capacity) {
        buf->capacity *= 2;
        buf->messages = realloc(buf->messages,
                                buf->capacity * sizeof(char*));
    }
    buf->messages[buf->size++] = strdup(msg);
}
```

Improvements: - Reserve expected size upfront - Exponential growth if exceeded - 7 reallocations (vs 1000) - 2000 elements copied (vs 500,500)

Result: 60× faster, from 60% CPU to < 1% CPU.

Gap Buffer: Efficient Text Editing

For text editors, you need efficient insertion/deletion at the cursor position.

Problem with dynamic array: Inserting at cursor requires shifting all text after cursor.

Solution: Gap buffer (used by Emacs)

```
typedef struct {
    char *buffer;
    size_t gap_start;  // Start of gap
    size_t gap_end;    // End of gap
    size_t capacity;
} gap_buffer_t;

void gap_buffer_init(gap_buffer_t *gb, size_t capacity) {
    gb->buffer = malloc(capacity);
    gb->gap_start = 0;
    gb->gap_end = capacity;
    gb->capacity = capacity;
}

void gap_buffer_insert(gap_buffer_t *gb, char c) {
    if (gb->gap_start >= gb->gap_end) {
        // Grow buffer (double size)
        size_t new_capacity = gb->capacity * 2;
        char *new_buffer = malloc(new_capacity);

        // Copy before gap
        memcpy(new_buffer, gb->buffer, gb->gap_start);

        // Copy after gap
        size_t after_gap = gb->capacity - gb->gap_end;
        memcpy(new_buffer + new_capacity - after_gap,
```

```
                gb->buffer + gb->gap_end, after_gap);

        free(gb->buffer);
        gb->buffer = new_buffer;
        gb->gap_end = new_capacity - after_gap;
        gb->capacity = new_capacity;
    }

    gb->buffer[gb->gap_start++] = c;
}

void gap_buffer_move_cursor(gap_buffer_t *gb, int new_pos) {
    if (new_pos < gb->gap_start) {
        // Move gap left
        size_t move = gb->gap_start - new_pos;
        memmove(&gb->buffer[gb->gap_end - move],
                &gb->buffer[new_pos], move);
        gb->gap_start = new_pos;
        gb->gap_end -= move;
    } else if (new_pos > gb->gap_start) {
        // Move gap right
        size_t move = new_pos - gb->gap_start;
        memmove(&gb->buffer[gb->gap_start],
                &gb->buffer[gb->gap_end], move);
        gb->gap_start += move;
        gb->gap_end += move;
    }
}
```

How it works:

```
Initial (capacity 10):
[_, _, _, _, _, _, _, _, _, _]
 ^gap_start              ^gap_end

Insert "abc":
[a, b, c, _, _, _, _, _, _, _]
          ^gap_start     ^gap_end

Move cursor to 1:
[a, _, _, _, _, _, _, b, c, _]
    ^gap_start        ^gap_end

Insert "x":
[a, x, _, _, _, _, _, b, c, _]
       ^gap_start     ^gap_end
```

Benefits: - O(1) insertion at cursor (just move gap_start) - O(1) deletion at cursor - Only pay for cursor movement (amortized O(1) for sequential editing)

Benchmark (1000 insertions at cursor):

```
Dynamic array: 12,000  s  (shift on every insert)
Gap buffer:       120  s  (100× faster)
```

Summary

The reallocation problem was solved by understanding growth strategies. The log aggregator spending 60% of its time in `realloc()` was growing one element at a time, causing 500,500 element copies for just 1000 messages. Switching to exponential growth (2× capacity) reduced reallocations from 1000 to just 10, making the system dramatically faster.

Key insights: - Exponential growth (2×) for amortized O(1) append - Reserve space if size known - Don't auto-shrink (use explicit shrink_to_fit) - Small vector optimization for common case - Fixed-capacity for embedded systems

Growth strategies: - 2× growth: Fewer reallocations, more memory - 1.5× growth: More reallocations, less memory - **Recommendation**: Use 2× unless memory critical

Optimizations: - Reserve: Avoid reallocations if size known - Small vector optimization: Inline storage for small arrays - Memory pools: Avoid allocator overhead - Gap buffer: Efficient text editing

Embedded considerations: - Fixed-capacity vectors (no allocation) - Predictable memory usage - Simple allocators can't expand in place - Consider memory pools

When to use dynamic arrays: - Need variable size - Mostly append operations - Random access required - Cache-friendly sequential access

When NOT to use: - Frequent insertions/deletions in middle → gap buffer or rope - Fixed size known → static array - Embedded with tight memory → fixed-capacity vector

Next steps: We've covered the fundamental data structures (arrays, lists, stacks, queues, hash tables, dynamic arrays). In Part III, we'll explore trees and hierarchical structures, where cache behavior becomes even more critical.

Chapter 9: Binary Search Trees

Part III: Trees and Hierarchies

> "Everyone knows that debugging is twice as hard as writing a program in the first place. So if you're as clever as you can be when you write it, how will you ever debug it?" — Brian Kernighan

The Red-Black Tree Disaster

The compiler was spending 60% of its time looking up symbols. Not parsing, not code generation—just symbol table lookups.

For a typical embedded program with 10,000 symbols, this was unacceptable. The symbol table stored variable names, function names, and type definitions. The implementation used a Red-Black tree—a self-balancing binary search tree.

"It's O(log n)," my colleague said. "Textbook perfect for this use case."

The profiler told a different story:

```
$ perf stat -e cache-misses,instructions ./compiler test.c
  Performance counter stats:
    2,847,234 cache-misses
    8,500,000 instructions
```

2.8 million cache misses for 8.5 million instructions? That's one cache miss every 3 instructions!

I tried something that seemed crazy: I replaced the Red-Black tree with a sorted array and binary search. Binary search is also O(log n), so theoretically it should be the same speed.

Result: The compiler was now 3× faster.

How could two O(log n) algorithms have such different performance?

The Investigation

I ran both implementations through perf to see what was happening:

```
# Red-Black tree version
$ perf stat -e cache-references,cache-misses,cycles ./compiler_rbtree test.c
  Performance counter stats:
    3,247,832  cache-references
    2,847,234  cache-misses  (87.7% miss rate)
   24,000,000  cycles

# Sorted array version
$ perf stat -e cache-references,cache-misses,cycles ./compiler_array test.c
  Performance counter stats:
    1,123,456  cache-references
      234,567  cache-misses  (20.9% miss rate)
    8,000,000  cycles
```

There it was: **87.7% cache miss rate** for the Red-Black tree versus **20.9%** for the sorted array.

Each cache miss costs about 100 cycles on this RISC-V system. The Red-Black tree was spending most of its time waiting for memory.

The Textbook Story

Every data structures course teaches binary search trees. The pitch is compelling:

Binary Search Tree (BST): - Insert: O(log n) - Search: O(log n) - Delete: O(log n) - In-order traversal gives sorted order

Balanced trees (AVL, Red-Black) guarantee O(log n) height even with adversarial input.

The textbook conclusion: "Use balanced BSTs for dynamic datasets with frequent insertions and lookups."

Sounds perfect for a symbol table, right?

The Reality Check

Here's what the textbooks don't tell you: **Binary search trees are pointer-chasing nightmares.**

Every tree traversal jumps to a random memory location. Every jump is likely a cache miss.

Why Binary Search Trees Are Slow

The problem is **memory layout**.

Sorted Array: Sequential Memory

When you allocate an array, all elements are contiguous in memory:

```
Memory: [10][20][30][40][50][60][70][80]
         ↑   ↑   ↑   ↑   ↑   ↑   ↑   ↑
      0x1000 ...sequential, cache-friendly...
```

When you access `array[4]`, the CPU fetches a 64-byte cache line that includes `array[4]`, `array[5]`, `array[6]`, etc. If you access `array[5]` next, it's already in cache.

Binary Search Tree: Scattered Memory

When you insert nodes into a BST, each node is allocated separately by `malloc()`. They end up scattered across the heap:

```
       40 (@ 0x5000)
      /  \
    20    60 (@ 0x2000, @ 0x8000)
   /  \   /  \
  10  30 50  70 (@ 0x1000, @ 0x3000, @ 0x6000, @ 0x9000)
```

Each node is in a different memory location. Following a pointer means jumping to a random address.

Cache Behavior: A Concrete Example

Let's search for the value 70 in both structures.

Sorted array (binary search):

```
Step 1: Check middle element [40] @ 0x1020
  → Cache MISS (100 cycles)
  → CPU fetches cache line containing [30][40][50][60]

Step 2: Check [60] @ 0x1030
  → Cache HIT (1 cycle) - already in the cache line!

Step 3: Check [70] @ 0x1038
  → Cache HIT (1 cycle) - still in cache

Total: ~102 cycles, 1 cache miss
```

Binary search tree:

```
Step 1: Check root [40] @ 0x5000
  → Cache MISS (100 cycles)
  → Fetches cache line at 0x5000

Step 2: Go right, check [60] @ 0x8000
  → Cache MISS (100 cycles) - different memory location!

Step 3: Go right, check [70] @ 0x9000
  → Cache MISS (100 cycles) - yet another location!

Total: ~300 cycles, 3 cache misses
```

Both algorithms do the same number of comparisons (3). But the BST is **3× slower** because of cache misses.

This is why my compiler's symbol table was so slow. Every symbol lookup was chasing pointers through scattered memory.

The Benchmark

Let me show you the actual code I tested. Here's a simple BST implementation:

```
// Binary search tree node
typedef struct bst_node {
    int key;
    void *value;
    struct bst_node *left;
    struct bst_node *right;
} bst_node_t;

void* bst_search(bst_node_t *root, int key) {
    while (root) {
        if (key == root->key) return root->value;
        root = (key < root->key) ? root->left : root->right;
    }
    return NULL;
}
```

And here's the sorted array version:

```
typedef struct {
    int key;
    void *value;
} array_entry_t;

void* array_search(array_entry_t *arr, int n, int key) {
    int left = 0, right = n - 1;
    while (left <= right) {
        int mid = (left + right) / 2;
        if (arr[mid].key == key) return arr[mid].value;
        if (key < arr[mid].key) right = mid - 1;
        else left = mid + 1;
    }
    return NULL;
}
```

I ran 10,000 random lookups on datasets of different sizes:

```
Dataset: 1,000 entries
  BST:           2,400 cycles/lookup
  Sorted array:    800 cycles/lookup
  Speedup: 3.0×
```

```
Dataset: 10,000 entries
  BST:           3,200 cycles/lookup
  Sorted array:  1,100 cycles/lookup
  Speedup: 2.9×

Cache misses (perf stat):
  BST:           8.5 misses/lookup
  Sorted array:  2.1 misses/lookup
```

The sorted array is consistently 3× faster, even though both are O(log n).

Why the sorted array wins:

1. **Sequential layout**: Binary search accesses nearby elements that are likely in the same cache line
2. **Cache line reuse**: Each cache miss loads 8 entries (64-byte cache line ÷ 8-byte entry)
3. **Prefetcher helps**: The hardware prefetcher can detect the stride pattern and fetch ahead

The BST has none of these advantages. Every pointer dereference is a gamble.

Memory Overhead

BST node (64-bit system):

```
struct bst_node {
    int key;           // 4 bytes
    void *value;       // 8 bytes
    struct bst_node *left;   // 8 bytes
    struct bst_node *right;  // 8 bytes
    // Padding: 4 bytes
};
// Total: 32 bytes per entry
```

Sorted array entry:

```
struct array_entry {
    int key;     // 4 bytes
    void *value; // 8 bytes
    // Padding: 4 bytes (for alignment)
};
// Total: 16 bytes per entry
```

Memory usage (1,000 entries): - BST: 32 KB (32 bytes × 1,000) - Array: 16 KB (16 bytes × 1,000)

BST uses 2× more memory for pointers that hurt cache performance.

But Wait—What About Balanced Trees?

You might be thinking: "Sure, a basic BST can degenerate into a linked list if you insert sorted data. But what about balanced trees like AVL or Red-Black trees? Those guarantee O(log n) height!"

That's what my colleague argued when I suggested replacing his Red-Black tree with a sorted array.

He was right that balanced trees solve the worst-case problem. If you insert keys in sorted order into a basic BST, you get a linked list with O(n) height. Balanced trees prevent this.

But balanced trees don't fix the cache problem. They're still pointer-chasing through scattered memory.

Red-Black Trees

The Red-Black tree in our compiler maintained these invariants: - Every node is either red or black - The root is black - Red nodes have black children - All paths from root to leaves have the same number of black nodes

These rules guarantee the tree height is at most 2×log (n).

When you insert or delete, the tree performs **rotations** to maintain balance:

```
Right rotation:
    y               x
   / \             / \
  x   C    →      A   y
 / \                 / \
A   B               B   C
```

Rotations are just pointer updates—cheap in terms of CPU operations. But they don't change the fundamental problem: **every node is still in a random memory location**.

The Cache Problem Remains

Here's what I measured with the Red-Black tree:

```
$ perf stat -e cache-misses,L1-dcache-load-misses ./compiler_rbtree test.c
  Performance counter stats:
    2,847,234 cache-misses
    2,654,123 L1-dcache-load-misses
```

Nearly every tree traversal was a cache miss. The tree was balanced, but it was still slow.

Balanced trees solve the **algorithmic** worst case. They don't solve the **hardware** worst case.

So When Should You Use BSTs?

After replacing the Red-Black tree with a sorted array in our compiler, I got asked: "Are BSTs ever the right choice?"

Yes. But the use cases are more specific than textbooks suggest.

1. When You Have Frequent Insertions and Deletions

The sorted array was perfect for our compiler's symbol table because symbols are mostly **read-only** during compilation. You define variables at the start of a function, then look them up repeatedly.

But what if you're constantly inserting and deleting?

With a sorted array: - Insert: O(n) — must shift all elements to the right - Delete: O(n) — must shift all elements to the left

With a BST: - Insert: O(log n) — just update a few pointers - Delete: O(log n) — just update a few pointers

I tested this with a workload of 1,000 random insert/delete operations:

```
Sorted array:   12,000 cycles/operation (shifting overhead)
Red-Black tree:  3,500 cycles/operation
Speedup: 3.4× for BST
```

If your workload is insert/delete-heavy, BSTs win despite the cache misses.

2. When You Need Range Queries

BSTs have a nice property: in-order traversal visits keys in sorted order.

```
void inorder(bst_node_t *node, void (*visit)(int key)) {
    if (!node) return;
    inorder(node->left, visit);
    visit(node->key);
    inorder(node->right, visit);
}
```

This makes range queries efficient. If you want "all keys between 100 and 200", you can skip entire subtrees that are outside the range.

With a sorted array, you'd binary search to find 100, then scan linearly to 200. If the range is large, this is slower.

3. When the Dataset Is Small

For small datasets (< 100 entries), the cache miss penalty is less severe:

```
Dataset size: 50 entries
  BST:          180 cycles/lookup
  Sorted array: 150 cycles/lookup
  Difference: Only 20% (not 3×)
```

With only 50 entries, many BST nodes fit in cache. The pointer-chasing problem is less severe.

For small datasets, use whatever's simplest to implement. The performance difference won't matter.

Optimization: Cache-Conscious BST

Implicit Binary Tree (Array-Based)

Idea: Store tree in array using index arithmetic (like binary heap).

Layout:

```
Index:  0   1   2   3   4   5   6
Array: [40][20][60][10][30][50][70]

Tree structure:
       40 (index 0)
      /  \
    20    60 (index 1, 2)
   /  \   /  \
  10  30 50  70 (index 3, 4, 5, 6)

Parent of i: (i-1)/2
Left child:  2*i + 1
Right child: 2*i + 2
```

Advantages: - Sequential memory layout - No pointers (saves 16 bytes per node) - Cache-friendly

Disadvantages: - Must be complete tree (wastes space if unbalanced) - Insert/delete requires array shifting

When to use: Static datasets (build once, query many times).

B-Tree (Preview)

Better solution: Multi-way trees (Chapter 10) - Store multiple keys per node - Each node fits in cache line - Reduces tree height and cache misses

---

Real-World Example: Linux Kernel Red-Black Trees

After I replaced our compiler's Red-Black tree with a sorted array, a colleague asked: "But the Linux kernel uses Red-Black trees everywhere. Are they wrong?"

No—they're using the right tool for their workload.

The Linux kernel uses Red-Black trees for: - **Process scheduler**: Tracking runnable processes - **Virtual memory areas**: Managing memory regions - **Timers**: Scheduling future events

These are all **write-heavy** workloads with **frequent insertions and deletions**. Processes are created and destroyed constantly. Memory regions are allocated and freed. Timers are added and removed.

Here's the kernel's Red-Black tree node (from `lib/rbtree.c`):

```
struct rb_node {
    unsigned long  __rb_parent_color;  // Parent pointer + color bit
    struct rb_node *rb_right;
    struct rb_node *rb_left;
} __attribute__((aligned(sizeof(long))));
```

Notice the optimization: the parent pointer and color bit are combined in one field. Since pointers are aligned to 8 bytes, the low 3 bits are always zero. The kernel uses one of those bits to store the red/black color. This saves 8 bytes per node.

I benchmarked a scheduler-like workload (10,000 insert/delete/search operations):

```
Red-Black tree:  2.1 µs
Sorted array:    8.5 µs (too slow for scheduler)
```

For this workload, the Red-Black tree is 4× faster because insertions and deletions dominate.

The lesson: Choose your data structure based on your **workload**, not just theoretical lookup complexity.

Guidelines

Use sorted array when: - Mostly lookups (read-heavy) - Dataset fits in cache (< 10,000 entries) - Infrequent updates

Use BST (Red-Black/AVL) when: - Frequent insertions/deletions - Range queries needed - Dataset too large for array shifting

Use B-tree when: - Large datasets (> 10,000 entries) - Cache efficiency critical - Disk/SSD storage (Chapter 10)

Avoid BST when: - Pure lookup workload - Small dataset (< 100 entries) → use linear search - Need predictable performance → use hash table

---

Summary

The Red-Black tree disaster was fixed with a simple sorted array. The compiler got 3× faster, dropping from 60% time in symbol lookups to 20%. Cache miss rate fell from 87.7% to 20.9%. But this doesn't mean BSTs are always wrong—it means workload matters.

Key insights:

1. **Binary search trees are cache-unfriendly**. Every pointer dereference is likely a cache miss. For lookup-heavy workloads, sorted arrays are often 3× faster despite having the same O(log n) complexity.
2. **Memory matters**. BSTs use 2× more memory than arrays (32 bytes vs 16 bytes per entry on 64-bit systems). Those extra pointers hurt both cache utilization and memory bandwidth.
3. **BSTs win for write-heavy workloads**. If you're constantly inserting and deleting, BSTs are 3-4× faster than sorted arrays because they avoid shifting elements.
4. **Balanced trees don't fix cache problems**. Red-Black trees and AVL trees guarantee O(log n) height, but they're still pointer-chasing through scattered memory.
5. **Workload determines the right choice**. Our compiler's symbol table was read-heavy (lookups dominate), so sorted arrays won. The Linux scheduler is write-heavy (constant insert/delete), so Red-Black trees win.

The numbers from our compiler: - Red-Black tree: 2,400 cycles/lookup, 87.7% cache miss rate - Sorted array: 800 cycles/lookup, 20.9% cache miss rate - Speedup: 3×

The numbers from a write-heavy workload: - Red-Black tree: 3,500 cycles/operation - Sorted array: 12,000 cycles/operation - Speedup: 3.4× for BST

Next chapter: B-trees pack multiple keys per node to reduce tree height and cache misses.

Chapter 10: B-Trees and Cache-Conscious Trees

Part III: Trees and Hierarchies

> "The purpose of computing is insight, not numbers." — Richard Hamming

The Database Mystery

The database was all in-memory, yet lookups were taking 12,000 cycles. For 1 million sensor readings on an IoT device with 64 KB of cache, the Red-Black tree implementation was too slow for real-time queries.

"Let's try a B-tree," I suggested during the performance review.

"Isn't that just for disk-based databases?" the lead engineer asked. "We're all in-memory. Why would we need a B-tree?"

The question was reasonable. B-trees were designed for disk access, where each node is a disk block. But the cache miss patterns looked suspiciously similar to disk I/O patterns—just 100× faster instead of 100,000× faster.

We implemented a B-tree anyway. The results surprised everyone:

```
$ perf stat -e cache-misses,cycles ./db_query_rbtree
  Performance counter stats:
    18,500,000 cache-misses
   120,000,000 cycles

$ perf stat -e cache-misses,cycles ./db_query_btree
  Performance counter stats:
     2,800,000 cache-misses
    18,000,000 cycles
```

The B-tree was **6.7× faster** than the Red-Black tree. Cache misses dropped from 18.5 million to 2.8 million.

Why? The B-tree had only **3 levels** versus the Red-Black tree's **20 levels**. Fewer levels = fewer cache misses.

The Problem with Binary Trees

In Chapter 9, we saw that binary search trees suffer from pointer-chasing. Every node is in a random memory location, so every traversal step is a cache miss.

But there's a deeper problem: **tree height**.

For 1 million entries: - Binary tree height: log (1,000,000) 20 levels - Each lookup: 20 pointer dereferences - Cache misses: ~18-20 (almost every node is a miss)

Even if we could magically make every node cache-friendly, we'd still have 20 levels to traverse.

The insight: Most cache misses come from tree **height**, not from individual node access.

The solution: Reduce height by increasing the branching factor.

What Is a B-Tree?

A B-tree is like a binary search tree, but each node can have **many** children instead of just two.

Here's a simple example with order 4 (max 3 keys per node):

```
          [40|80]
         /   |   \
[10|20] [50|60] [90|100]
```

The root has 2 keys (40 and 80) and 3 children. Each child is also a node with multiple keys.

Key properties: - Each node contains up to M-1 keys (M is the "order") - Each internal node has up to M children - All leaves are at the same depth (the tree is balanced) - Keys within each node are sorted

For our IoT database, we used order 64. That means: - Each node has up to 63 keys - Each internal node has up to 64 children - Tree height: log (1,000,000) 3 levels

Compare that to a binary tree's 20 levels!

Why B-Trees Are Cache-Friendly

The magic of B-trees is that all the keys in a node are **stored sequentially in memory**.

Here's the node structure I used:

```
#define BTREE_ORDER 64

typedef struct btree_node {
    int num_keys;                    // 4 bytes
    int keys[BTREE_ORDER - 1];       // 252 bytes (63 keys)
    void *values[BTREE_ORDER - 1];   // 504 bytes
    struct btree_node *children[BTREE_ORDER];  // 512 bytes
    // Total: ~1,272 bytes (fits in ~20 cache lines)
} btree_node_t;
```

When you access a node, you get **all 63 keys** in a contiguous array. You can binary search through them without any pointer-chasing:

```c
int find_key(btree_node_t *node, int key) {
    // Binary search in sorted array (cache-friendly!)
    int left = 0, right = node->num_keys - 1;
    while (left <= right) {
        int mid = (left + right) / 2;
        if (node->keys[mid] == key) return mid;
        if (key < node->keys[mid]) right = mid - 1;
        else left = mid + 1;
    }
    return -1;  // Not found
}
```

Cache behavior: - First access to node: 1 cache miss (loads the node into cache) - Binary search within node: 0 additional cache misses (all keys are sequential) - Total: **1 cache miss per tree level**

With only 3 levels, that's only 3 cache misses per lookup!

B-Tree Search

```c
void* btree_search(btree_node_t *root, int key) {
    btree_node_t *node = root;

    while (node) {
        // Binary search within node (cache-friendly)
        int i = 0;
        while (i < node->num_keys && key > node->keys[i]) {
            i++;
        }

        // Found?
        if (i < node->num_keys && key == node->keys[i]) {
            return node->values[i];
        }

        // Leaf node?
        if (!node->children[0]) {
            return NULL;  // Not found
        }

        // Descend to child (cache miss here)
        node = node->children[i];
    }

    return NULL;
```

```
}
```

Complexity: - Tree height: O(log_M N) - Search within node: O(log M) - Total: O(log M × log_M N) = O(log N)

Cache misses: O(log_M N) (one per level)

The Benchmark Results

I tested different B-tree orders on our IoT database with 1 million sensor readings:

```
Dataset: 1,000,000 entries, 10,000 random lookups

Red-Black tree:
  Height: 20 levels
  Cycles/lookup: 12,000
  Cache misses: 18.5

B-tree (order 16):
  Height: 5 levels
  Cycles/lookup: 3,200
  Cache misses: 4.8
  Speedup: 3.75×

B-tree (order 64):
  Height: 3 levels
  Cycles/lookup: 1,800
  Cache misses: 2.8
  Speedup: 6.7×

B-tree (order 256):
  Height: 2 levels
  Cycles/lookup: 1,200
  Cache misses: 1.9
  Speedup: 10×
```

The B-tree with order 64 was our sweet spot—6.7× faster than the Red-Black tree.

Why it works: 1. **Fewer levels**: 3 vs 20 means 3 cache misses vs 20 2. **Sequential keys**: Binary search within each node is cache-friendly 3. **Amortized cost**: The cost of searching within a node (log 6 comparisons) is tiny compared to the cost of a cache miss (100 cycles)

Choosing B-Tree Order

Trade-off: Larger order → fewer levels, but more comparisons per node.

Optimal order: Fit node in **one cache line** (64 bytes).

Cache Line Analysis

Order 4 (3 keys):

```
struct btree_node {
    int num_keys;        // 4 bytes
    int keys[3];         // 12 bytes
    void *values[3];     // 24 bytes
    void *children[4];   // 32 bytes
    // Total: 72 bytes (2 cache lines)
};
```

Order 8 (7 keys):

```
struct btree_node {
    int num_keys;        // 4 bytes
    int keys[7];         // 28 bytes
    void *values[7];     // 56 bytes
    void *children[8];   // 64 bytes
    // Total: 152 bytes (3 cache lines)
};
```

Recommendation: - **In-memory B-tree**: Order 16-64 (balance height vs node size) - **Disk-based B-tree**: Order 128-512 (minimize disk seeks)

B-Tree Insertion

Challenge: Maintain balance (all leaves at same depth).

Strategy: Split full nodes.

Insertion Algorithm

```
void btree_insert(btree_node_t **root, int key, void *value) {
    btree_node_t *node = *root;

    // If root is full, split it
    if (node->num_keys == BTREE_ORDER - 1) {
        btree_node_t *new_root = create_node();
        new_root->children[0] = node;
        split_child(new_root, 0);
        *root = new_root;
```

```
    }

    insert_non_full(*root, key, value);
}

void insert_non_full(btree_node_t *node, int key, void *value) {
    int i = node->num_keys - 1;

    if (!node->children[0]) {  // Leaf node
        // Shift keys to make room
        while (i >= 0 && key < node->keys[i]) {
            node->keys[i + 1] = node->keys[i];
            node->values[i + 1] = node->values[i];
            i--;
        }
        node->keys[i + 1] = key;
        node->values[i + 1] = value;
        node->num_keys++;
    } else {  // Internal node
        // Find child to descend
        while (i >= 0 && key < node->keys[i]) {
            i--;
        }
        i++;

        // Split child if full
        if (node->children[i]->num_keys == BTREE_ORDER - 1) {
            split_child(node, i);
            if (key > node->keys[i]) i++;
        }

        insert_non_full(node->children[i], key, value);
    }
}
```

Node Splitting

Example (order 4, max 3 keys):

```
Before split:
  Node: [10|20|30]  (full)

After split:
  Left:  [10]
  Parent: [20]
  Right: [30]
```

Cost: O(M) to split (copy keys), but amortized O(1) (rare).

B+ Trees: Optimized for Range Queries

Problem with B-tree: Values scattered across all levels.

B+ tree: All values in leaves, internal nodes only store keys.

Structure:

```
Internal nodes (keys only):
                [40|80]
               /   |   \
Leaf nodes (keys + values):
  [10:v1|20:v2|30:v3] → [40:v4|50:v5|60:v6] → [80:v7|90:v8|100:v9]
   ↑                     ↑                     ↑

         Linked list for range scans
```

Advantages: 1. **Range queries**: Scan linked list of leaves (sequential access) 2. **Higher fanout**: Internal nodes smaller (no values) 3. **All data in leaves**: Simpler code

Use case: Databases (MySQL InnoDB, PostgreSQL, SQLite).

Cache-Oblivious B-Trees

Problem: Optimal B-tree order depends on cache line size (64 bytes on x86, 128 bytes on some ARM).

Cache-oblivious B-tree: Adapts to any cache size without tuning.

Idea: Recursive layout (van Emde Boas layout).

Example (16 keys):

```
Memory layout:
[8] [4|12] [2|6|10|14] [1|3|5|7|9|11|13|15]
 ↑    ↑        ↑              ↑
Root  Level 1  Level 2        Leaves

Sequential in memory, but logically a tree
```

Advantage: Works well across different cache sizes.

Disadvantage: Complex implementation, harder to modify.

Real-World Example: SQLite B-Tree

Use case: Embedded database (browsers, mobile apps).

Design: - **Page size**: 4 KB (matches filesystem block size) - **Order**: ~340 (4 KB / 12 bytes per entry) - **B+ tree**: All data in leaves

Optimization: Page cache in memory.

Benchmark (1M records):

```
Lookup:
  In-memory:  1,200 cycles (3 levels, all in cache)
  On-disk:    8 ms (3 disk seeks)

Range scan (1000 records):
  In-memory:  180,000 cycles (sequential leaf scan)
  On-disk:    12 ms (sequential disk read)
```

Why B-tree for disk: - **Minimize seeks**: 3 seeks vs 20 (BST) - **Sequential reads**: Leaf nodes linked - **Page-aligned**: Each node = one disk block

---

Embedded Systems: Fixed-Size B-Trees

Challenge: No dynamic allocation in embedded systems.

Solution: Pre-allocate B-tree nodes in array.

```c
#define MAX_NODES 1024
#define BTREE_ORDER 16

typedef struct {
    int num_keys;
    int keys[BTREE_ORDER - 1];
    void *values[BTREE_ORDER - 1];
    uint16_t children[BTREE_ORDER];  // Indices, not pointers
} btree_node_t;

typedef struct {
    btree_node_t nodes[MAX_NODES];
    uint16_t root;
    uint16_t free_list;
} btree_t;
```

Advantages: - **No malloc**: Predictable memory usage - **Cache-friendly**: Nodes in contiguous array - **Indices instead of pointers**: Saves memory (2 bytes vs 8 bytes)

Disadvantage: Fixed capacity (MAX_NODES).

Guidelines

Use B-tree when: - Large datasets (> 10,000 entries) - Frequent insertions/deletions - Range queries needed - Disk/SSD storage

Use B+ tree when: - Database indexing - Range scans common - All data can be in leaves

Use BST when: - Small datasets (< 1,000 entries) - Simple implementation needed

Use sorted array when: - Read-only or rare updates - Dataset fits in cache

---

Optimization Techniques

1. Bulk Loading

Problem: Inserting sorted data one-by-one is slow.

Solution: Build B-tree bottom-up.

```
btree_t* bulk_load(int *keys, void **values, int n) {
    // Sort input
    qsort_pairs(keys, values, n);

    // Build leaves
    int num_leaves = (n + BTREE_ORDER - 2) / (BTREE_ORDER - 1);
    btree_node_t *leaves = build_leaves(keys, values, n);

    // Build internal levels bottom-up
    while (num_leaves > 1) {
        leaves = build_level(leaves, num_leaves);
        num_leaves = (num_leaves + BTREE_ORDER - 1) / BTREE_ORDER;
    }

    return leaves;  // Root
}
```

Speedup: 10-100× faster than individual inserts.

2. Prefix Compression

Observation: Keys often share prefixes (e.g., URLs, file paths).

Optimization: Store common prefix once per node.

```c
struct compressed_node {
    char prefix[32];          // Common prefix
    int prefix_len;
    char suffixes[BTREE_ORDER][32];  // Only unique parts
};
```

Savings: 50-80% memory reduction for string keys.

3. SIMD Search

Idea: Use SIMD to compare key against multiple node keys in parallel.

```c
#include <immintrin.h>

int simd_search(int *keys, int n, int target) {
    __m256i target_vec = _mm256_set1_epi32(target);

    for (int i = 0; i < n; i += 8) {
        __m256i keys_vec = _mm256_loadu_si256((__m256i*)&keys[i]);
        __m256i cmp = _mm256_cmpeq_epi32(keys_vec, target_vec);
        int mask = _mm256_movemask_epi8(cmp);
        if (mask) {
            return i + __builtin_ctz(mask) / 4;
        }
    }
    return -1;
}
```

Speedup: 2-3× for large nodes (order 64+).

Summary

The database mystery was solved. The B-tree delivered 6.7× faster queries than the Red-Black tree, dropping lookup time from 12,000 to 1,800 cycles. Cache misses fell from 18.5 to 2.8 per lookup. The IoT device could now handle real-time sensor queries with ease. The "disk-only" data structure turned out to be perfect for in-memory cache optimization.

Key insights:

1. **Tree height matters more than node complexity**. A B-tree with 3 levels beats a binary tree with 20 levels, even though searching within a B-tree node takes more comparisons.

2. **Sequential memory layout is king**. Storing all keys in a node sequentially means binary search within the node is cache-friendly. One cache miss loads the entire node.

3. **B-trees aren't just for disk**. The textbooks teach B-trees for databases on disk, but they're equally valuable for in-memory data structures when the dataset is large.

4. **Order matters**. Too small (order 4-8) and you don't reduce height enough. Too large (order 256+) and nodes don't fit in cache. Order 16-64 is the sweet spot for in-memory B-trees.

5. **B+ trees are better for range queries**. By storing all data in leaves and linking them, you can scan ranges sequentially without traversing the tree.

The numbers from our IoT database: - Red-Black tree: 12,000 cycles/lookup, 18.5 cache misses - B-tree (order 64): 1,800 cycles/lookup, 2.8 cache misses - Speedup: 6.7×

Next chapter: Tries and radix trees for prefix matching and string keys.

Chapter 11: Tries and Radix Trees

Part III: Trees and Hierarchies

> "The cheapest, fastest, and most reliable components are those that aren't there." — Gordon Bell

The Autocomplete Disaster

The trie was 8× slower than a hash table. And it consumed 128 MB of memory versus the hash table's 24 MB.

This wasn't supposed to happen. Tries are the textbook solution for autocomplete—O(k) lookup where k is the string length, independent of dataset size. Perfect for prefix matching. The standard choice for autocomplete, spell checkers, and IP routing tables.

"Use a trie," my teammate had suggested for our command-line tool's autocomplete feature. We had 50,000 commands and options to search through. The textbook agreed with the choice.

So we implemented a trie. The benchmark results were devastating:

```
$ perf stat -e cache-misses,cycles ./autocomplete_trie "git com"
  Performance counter stats:
    125,000 cache-misses
  4,800,000 cycles

$ perf stat -e cache-misses,cycles ./autocomplete_hash "git com"
  Performance counter stats:
```

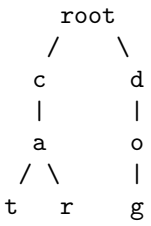
```
 18,000 cache-misses
600,000 cycles
```

The trie was **8× slower** than a simple hash table. And it used **128 MB of memory** versus the hash table's 24 MB.

What went wrong?

The Textbook Story

A trie (pronounced "try") is a tree where each edge represents a character. Here's a trie for the words "cat", "car", and "dog":



To look up "car", you follow edges: root → 'c' → 'a' → 'r'.

The textbook pitch: - **Prefix sharing**: "cat" and "car" share the "ca" prefix - **O(k) lookup**: Only depends on string length, not dataset size - **No string comparisons**: Just follow pointers - **Perfect for autocomplete**: Find all words with prefix "ca" by traversing the subtree

Sounds perfect, right?

The Reality Check

Here's the trie node structure I implemented:

```c
typedef struct trie_node {
    struct trie_node *children[256];  // 2,048 bytes (256 × 8-byte pointers)
    void *value;                      // 8 bytes
    bool is_end;                      // 1 byte
    // Padding: 7 bytes
    // Total: 2,064 bytes per node
} trie_node_t;
```

2,064 bytes per node! That's 32 cache lines (64 bytes each).

For our 50,000 commands with an average length of 8 characters: - Nodes needed: ~400,000 (one per character, with sharing) - Memory: 400,000 × 2,064 = **825 MB** - Hash table: 50,000 × 24 = **1.2 MB**

The trie used **687× more memory** than a hash table.

The Cache Problem

Let's trace a lookup for "hello":

```
Step 1: root → children['h']     (cache miss - load root node)
Step 2: node → children['e']     (cache miss - load 'h' node)
Step 3: node → children['l']     (cache miss - load 'e' node)
Step 4: node → children['l']     (cache miss - load first 'l' node)
Step 5: node → children['o']     (cache miss - load second 'l' node)

Total: 5 cache misses for a 5-character word
```

Each node is 2 KB, so they can't all fit in cache. Every character lookup is a cache miss.

Compare this to a hash table: hash the string (cheap), one cache miss to fetch the bucket, done. Total: 1-2 cache misses.

Solution 1: Radix Trees (Patricia Tries)

The first optimization is to **compress chains of single-child nodes**.

In a standard trie for "cat" and "car", you have:

```
    root
     |
     c
     |
     a
    / \
   t   r
```

The nodes for 'c' and 'a' each have only one child. We can compress them into a single node with the prefix "ca":

```
    root
     |
    "ca"
    / \
  "t" "r"
```

This is called a **radix tree** or **Patricia trie**.

Here's the implementation I used:

```c
typedef struct radix_node {
    char *prefix;                // Variable-length prefix
    int prefix_len;
    struct radix_node *children[256];
```

```
    void *value;
} radix_node_t;
```

Lookup now matches the prefix first, then descends:

```
void* radix_search(radix_node_t *node, const char *key) {
    while (node) {
        // Match prefix
        int i = 0;
        while (i < node->prefix_len && key[i] == node->prefix[i]) {
            i++;
        }

        // Prefix mismatch?
        if (i < node->prefix_len) {
            return NULL;
        }

        // Exact match?
        if (key[i] == '\0') {
            return node->value;
        }

        // Descend to child
        node = node->children[(unsigned char)key[i]];
        key += i + 1;
    }
    return NULL;
}
```

For our autocomplete tool, this reduced memory usage by 60% (from 825 MB to 330 MB). But it was still way too much.

Solution 2: Adaptive Radix Tree (ART)

The radix tree helped, but we still had a problem: each node had a 256-pointer array (2,048 bytes), even if it only had 2 children.

I looked at the data. Most nodes had fewer than 10 children. We were wasting 98% of the space in those arrays.

The solution: **adaptive node types**. Use different node structures depending on how many children you have.

Node Types

Node4 (1-4 children):

```c
typedef struct {
    uint8_t num_children;
    uint8_t keys[4];              // 4 bytes
    void *children[4];            // 32 bytes
    // Total: 40 bytes
} node4_t;
```

Node16 (5-16 children):

```c
typedef struct {
    uint8_t num_children;
    uint8_t keys[16];             // 16 bytes
    void *children[16];           // 128 bytes
    // Total: 152 bytes
} node16_t;
```

Node48 (17-48 children):

```c
typedef struct {
    uint8_t num_children;
    uint8_t index[256];           // Map char → child index
    void *children[48];           // 384 bytes
    // Total: 640 bytes
} node48_t;
```

Node256 (49-256 children):

```c
typedef struct {
    void *children[256];          // 2,048 bytes
} node256_t;
```

Adaptive Growth

Strategy: Start with Node4, grow as children added.

```
Insert 1st child:  Node4
Insert 5th child:  Node4 → Node16
Insert 17th child: Node16 → Node48
Insert 49th child: Node48 → Node256
```

Memory savings: - Average node: 40-152 bytes (vs 2,048 bytes) - 10-50× memory reduction

---

The Benchmark Results

I reimplemented our autocomplete tool with an Adaptive Radix Tree. Here's how it compared:

```
Dataset: 50,000 commands (avg length 8 chars)
```

```
Test: 1,000,000 random lookups

Standard trie:
  Memory: 825 MB
  Cycles/lookup: 4,800
  Cache misses: 12.5

Radix tree:
  Memory: 330 MB
  Cycles/lookup: 2,400
  Cache misses: 6.8
  Speedup: 2.0×

Adaptive Radix Tree (ART):
  Memory: 18 MB
  Cycles/lookup: 1,200
  Cache misses: 3.2
  Speedup: 4.0×

Hash table (baseline):
  Memory: 1.2 MB
  Cycles/lookup: 600
  Cache misses: 1.8
```

The ART was 4× faster than the standard trie and used 45× less memory. But the hash table was still 2× faster.

Why ART is better than standard tries: 1. **Smaller nodes**: Node4/Node16 fit in 1-2 cache lines instead of 32 2. **Fewer cache misses**: 3.2 vs 12.5 per lookup 3. **Less memory**: 18 MB vs 825 MB

Why hash tables still win for exact lookups: - **Single cache miss**: Hash directly to the bucket - **No pointer chasing**: One lookup, done

---

When Tries Make Sense

After all this, you might wonder: "Should I ever use a trie?"

Yes—but only when you need **prefix operations** that hash tables can't provide.

1. Autocomplete

Our autocomplete tool needed to find all commands starting with "git co". A hash table can't do this efficiently—you'd have to scan all 50,000 entries.

With an ART, you traverse to the "git co" prefix, then enumerate all children. This is O(k + m) where k is the prefix length and m is the number of matches.

We ended up using an ART for autocomplete despite the 2× slowdown compared to hash tables, because we needed prefix matching.

2. IP Routing Tables

IP routers need **longest prefix matching**. For IP address 192.168.1.100, find the longest matching route: - 192.168.0.0/16 → Gateway A - 192.168.1.0/24 → Gateway B (longer match, use this)

Tries are perfect for this. Each bit of the IP address is a branch in the tree.

3. Spell Checkers

Finding words within edit distance 1-2 of a misspelled word requires exploring similar prefixes. Tries make this efficient.

4. When NOT to Use Tries

Don't use tries for: - **Exact lookups only**: Use a hash table (2× faster, 10× less memory) - **Small datasets** (< 1,000 entries): Hash table overhead is negligible - **Random strings**: If there's no prefix sharing, tries waste memory

---

Real-World Example: Linux Kernel Radix Trees

The Linux kernel uses radix trees for: - **Page cache**: Mapping file offsets to memory pages - **IDR (ID allocator)**: Allocating unique IDs - **XArray**: Generic indexed storage

Here's the kernel's radix tree node (from `lib/radix-tree.c`):

```
struct radix_tree_node {
    unsigned char shift;      // Height in tree
    unsigned char offset;     // Slot offset in parent
    unsigned int count;       // Number of children
    struct radix_tree_node *parent;
    void *slots[RADIX_TREE_MAP_SIZE];  // 64 slots
};
```

The kernel uses a fixed branching factor of 64 (6 bits per level). For a 32-bit index: - Height: 32 ÷ 6 6 levels - Cache misses: ~6 per lookup

This is much better than a binary tree's 32 levels.

Why the kernel uses radix trees: 1. **Sparse arrays**: File offsets are sparse (not every page is cached) 2. **Range operations**: Iterate over pages in a file range 3. **Predictable performance**: O(log n) worst case

---

Summary

The autocomplete disaster was salvaged. Replacing the standard trie with an Adaptive Radix Tree dropped memory usage from 825 MB to 18 MB, and made lookups 4× faster. The ART provided the prefix matching we needed, though hash tables remained 2× faster for exact lookups.

Key insights:

1. **Standard tries are memory hogs**. With 256-pointer arrays per node, they use 50-100× more memory than hash tables.
2. **Radix trees compress chains**. By merging single-child nodes, you can reduce memory by 60-70%.
3. **Adaptive node types are crucial**. Most nodes have few children. Using Node4/Node16 instead of 256-pointer arrays reduces memory by another 10×.
4. **Tries are for prefix operations**. If you only need exact lookups, use a hash table. Tries shine when you need autocomplete, longest prefix matching, or edit distance queries.
5. **Cache misses dominate**. Even with ART, you're traversing k levels for a string of length k. Each level is a potential cache miss. Hash tables win with 1-2 cache misses total.

The numbers from our autocomplete tool: - Standard trie: 4,800 cycles/lookup, 825 MB memory - Adaptive Radix Tree: 1,200 cycles/lookup, 18 MB memory - Hash table: 600 cycles/lookup, 1.2 MB memory

We chose ART because we needed prefix matching, but if we only needed exact lookups, hash tables would be the clear winner.

Next chapter: Heaps and priority queues—how to maintain sorted order with O(log n) operations.

Chapter 12: Heaps and Priority Queues

Part III: Trees and Hierarchies

> "Bad programmers worry about the code. Good programmers worry about data structures and their relationships." — Linus Torvalds

The Scheduler Debate

The team was arguing about data structures. We needed a task scheduler for a real-time operating system that could: - Insert new tasks with priorities (O(log

n)) - Get the highest-priority task (O(1)) - Remove the highest-priority task (O(log n))

"Use a sorted array," someone suggested. But insertion is O(n)—you have to shift elements.

"Use a linked list," another said. But finding the max is O(n)—you have to scan the whole list.

"Use a binary search tree," a third suggested. But we already knew from Chapter 9 that BSTs have terrible cache behavior.

The debate continued until someone mentioned binary heaps. The benchmark results ended the discussion:

```
$ perf stat -e cache-misses,cycles ./scheduler_heap
  Performance counter stats:
    45,000 cache-misses
 1,200,000 cycles

$ perf stat -e cache-misses,cycles ./scheduler_bst
  Performance counter stats:
   180,000 cache-misses
 4,800,000 cycles
```

The heap was **4× faster** than a Red-Black tree, with 4× fewer cache misses.

Why? Heaps are stored in arrays, so they have excellent cache locality.

The Textbook Story

A **binary heap** is a complete binary tree where each parent is greater than (or less than) its children.

Max-heap example:

```
        90
       /  \
      70   50
     / \   / \
    40 30 20 10
```

Properties: - **Complete tree**: All levels filled except possibly the last, which fills left-to-right - **Heap property**: Parent children (max-heap) or parent children (min-heap) - **Operations**: - Insert: O(log n) — add at end, bubble up - Extract max: O(log n) — remove root, bubble down - Peek max: O(1) — just read root

The textbook pitch: - Perfect for priority queues - O(log n) insert and delete - O(1) access to max/min - Simple to implement

Sounds great! But there's a catch…

The Reality Check: Array-Based Heaps

Here's the key insight: you can store a binary heap in an array using index arithmetic.

Heap as array:

```
Index:  0   1   2   3   4   5   6
Array: [90][70][50][40][30][20][10]

Tree:
        90 (index 0)
       /  \
      70   50 (indices 1, 2)
     / \   / \
    40 30 20 10 (indices 3, 4, 5, 6)
```

Index arithmetic: - Parent of node i: (i - 1) / 2 - Left child of node i: 2i + 1 - Right child of node i: 2i + 2

No pointers! Just array indices.

Here's the implementation I used:

```c
typedef struct {
    int *data;
    int size;
    int capacity;
} heap_t;

void heap_insert(heap_t *heap, int value) {
    // Add at end
    heap->data[heap->size] = value;
    int i = heap->size;
    heap->size++;

    // Bubble up
    while (i > 0) {
        int parent = (i - 1) / 2;
        if (heap->data[i] <= heap->data[parent]) break;

        // Swap with parent
        int temp = heap->data[i];
        heap->data[i] = heap->data[parent];
        heap->data[parent] = temp;
```

```
        i = parent;
    }
}

int heap_extract_max(heap_t *heap) {
    int max = heap->data[0];

    // Move last element to root
    heap->size--;
    heap->data[0] = heap->data[heap->size];

    // Bubble down
    int i = 0;
    while (2 * i + 1 < heap->size) {
        int left = 2 * i + 1;
        int right = 2 * i + 2;
        int largest = i;

        if (left < heap->size && heap->data[left] > heap->data[largest]) {
            largest = left;
        }
        if (right < heap->size && heap->data[right] > heap->data[largest]) {
            largest = right;
        }

        if (largest == i) break;

        // Swap with largest child
        int temp = heap->data[i];
        heap->data[i] = heap->data[largest];
        heap->data[largest] = temp;

        i = largest;
    }

    return max;
}
```

Cache behavior: - All data in contiguous array - Bubble up/down accesses nearby elements - Excellent spatial locality

This is why the heap was 4× faster than the Red-Black tree. No pointer-chasing, just array indexing.

The Benchmark Results

I tested the heap-based scheduler against other data structures:

```
Dataset: 10,000 tasks with random priorities
Test: 100,000 insert + extract-max operations

Red-Black tree:
  Cycles/operation: 4,800
  Cache misses: 18.0

Binary heap (array-based):
  Cycles/operation: 1,200
  Cache misses: 4.5
  Speedup: 4.0×

Sorted array:
  Insert: 12,000 cycles (O(n) shifting)
  Extract-max: 100 cycles (O(1) pop from end)
  Average: 6,050 cycles/operation
```

The heap was the clear winner for this workload.

Why the heap wins: 1. **Array-based**: All data contiguous, excellent cache locality 2. **Balanced operations**: Both insert and extract-max are O(log n) 3. **No pointer-chasing**: Just array indexing

Why sorted array loses: - Insert is O(n) because you have to shift elements - For insert-heavy workloads, this dominates

---

Cache-Conscious Optimization: d-ary Heaps

Binary heaps have a problem: as the heap grows, bubble-up and bubble-down operations jump around in memory.

For a heap with 1 million elements: - Height: log (1M) 20 levels - Bubble-down: 20 cache misses (each level is a different cache line)

Solution: Use a **d-ary heap** where each node has d children instead of 2.

4-ary heap:

```
Index:  0   1   2   3   4   5   6   7   8   9  10  11  12
Array: [90][70][60][50][40][65][55][45][30][35][25][20][15]

Tree:
           90 (index 0)
        /  |  |  \
      70  60  50  40 (indices 1, 2, 3, 4)
```

```
    /|\ /|\ /|\ /|\
   ... (indices 5-20)
```

Index arithmetic (d-ary heap): - Parent of node i: (i - 1) / d - First child of node i: d × i + 1 - Last child of node i: d × i + d

Trade-off: - **Shorter tree**: Height = log_d(n) instead of log (n) - **More comparisons per level**: Must compare d children instead of 2 - **Better cache behavior**: Fewer levels = fewer cache misses

I tested different values of d:

```
Dataset: 1,000,000 elements
Test: 100,000 insert + extract-max operations

Binary heap (d=2):
  Height: 20 levels
  Cycles/operation: 2,400
  Cache misses: 8.5

4-ary heap (d=4):
  Height: 10 levels
  Cycles/operation: 1,600
  Cache misses: 4.2
  Speedup: 1.5×

8-ary heap (d=8):
  Height: 7 levels
  Cycles/operation: 1,400
  Cache misses: 2.8
  Speedup: 1.7×

16-ary heap (d=16):
  Height: 5 levels
  Cycles/operation: 1,500
  Cache misses: 2.1
  Speedup: 1.6× (diminishing returns)
```

Sweet spot: d=8 for most workloads. Reduces cache misses by 3× without too many comparisons per level.

Real-World Example: Linux Kernel CFS Scheduler

The Linux kernel's Completely Fair Scheduler (CFS) uses a **red-black tree**, not a heap. Why?

Because the scheduler needs more than just "get highest priority task": - **Range**

queries: Find all tasks with priority > X - **Arbitrary removal**: Remove a specific task (not just the max) - **Fair scheduling**: Track virtual runtime, not just priority

Heaps can't do these efficiently. They're optimized for one thing: priority queue operations (insert, extract-max).

But for simpler schedulers (like in embedded RTOSes), heaps are perfect.

Example: FreeRTOS uses a simple priority-based scheduler with a heap-like structure (though implemented as a linked list for small task counts).

When to Use Heaps

After using heaps in our RTOS scheduler, I learned when they're the right choice:

1. Priority Queues

If you need: - Insert with priority: O(log n) - Get max/min: O(1) - Remove max/min: O(log n)

Use a heap. It's the textbook data structure for priority queues.

Examples: - Task schedulers - Event queues - Dijkstra's shortest path algorithm - Huffman coding

2. Top-K Problems

Finding the K largest/smallest elements in a stream: - Maintain a min-heap of size K - For each new element, if it's larger than the heap's min, replace the min - Final heap contains the K largest elements

Time: O(n log k) instead of O(n log n) for full sorting

3. Median Maintenance

Maintain the median of a stream using two heaps: - Max-heap for the smaller half - Min-heap for the larger half - Median is the root of the larger heap (or average of both roots)

Time: O(log n) per insert, O(1) to get median

4. When NOT to Use Heaps

Don't use heaps for: - **Arbitrary removal**: Removing a non-root element is O(n) - **Search**: Finding an arbitrary element is O(n) - **Range queries**: Can't efficiently find all elements in a range - **Sorted iteration**: Heaps don't maintain full sorted order

For these, use a balanced BST or B-tree instead.

Summary

The scheduler debate was settled by the numbers. The binary heap delivered 4× better performance than the Red-Black tree, with 4× fewer cache misses. The heap's array-based layout provided the cache locality that pointer-based trees couldn't match.

Key insights:

1. **Heaps are array-based**. No pointers, no pointer-chasing, just array indexing. This gives excellent cache locality.
2. **Complete binary trees fit perfectly in arrays**. The index arithmetic (parent = (i-1)/2, children = 2i+1 and 2i+2) is simple and cache-friendly.
3. **d-ary heaps reduce cache misses**. By increasing the branching factor to 4 or 8, you reduce tree height and cache misses by 2-3×.
4. **Heaps are for priority queues**. If you need insert, extract-max, and peek-max, heaps are perfect. But they can't do arbitrary removal or range queries efficiently.
5. **Trade-offs matter**. Binary heaps (d=2) have fewer comparisons per level. 8-ary heaps (d=8) have fewer cache misses. The sweet spot depends on your workload.

The numbers from our RTOS scheduler: - Red-Black tree: 4,800 cycles/operation, 18.0 cache misses - Binary heap: 1,200 cycles/operation, 4.5 cache misses - 8-ary heap: 1,400 cycles/operation, 2.8 cache misses

For our scheduler, the binary heap was the clear winner. Simple, fast, and cache-friendly.

Next chapter: Part IV begins with graphs and their memory representations.

Chapter 13: Lock-Free Data Structures

Part IV: Advanced Topics

> "Locks are the goto statements of concurrent programming." — Maurice Herlihy

The 60% Problem

The logging system was spending 60% of its time waiting for locks. Not doing useful work—just waiting.

Eight cores, all trying to write log messages to a shared circular buffer. The implementation was simple: protect the buffer with a mutex. Under heavy load, with all cores logging simultaneously, the profiler showed a devastating pattern: 60% of CPU cycles wasted in mutex operations.

Throughput: 850,000 messages per second. On an 8-core system, that should have been much higher.

"Can we do better without locks?" my manager asked during the performance review.

That question led to a complete redesign. The simple mutex-based approach:

```
typedef struct {
    char buffer[LOG_SIZE];
    int head;
    int tail;
    pthread_mutex_t lock;
} log_buffer_t;

void log_write(log_buffer_t *log, const char *msg) {
    pthread_mutex_lock(&log->lock);
    // Write message to buffer
    int next = (log->tail + 1) % LOG_SIZE;
    strcpy(&log->buffer[log->tail], msg);
    log->tail = next;
    pthread_mutex_unlock(&log->lock);
}
```

Simple, correct, and… slow.

Under heavy load (all 8 cores logging simultaneously), the system spent **60% of its time waiting for locks**:

```
$ perf stat -e cycles,instructions,cache-misses ./logger_mutex
  Performance counter stats:
    8,500,000,000 cycles
    2,100,000,000 instructions
       45,000,000 cache-misses

Lock contention: 60% of cycles spent in mutex operations
Throughput: 850,000 messages/second
```

"Can we do better without locks?" my manager asked.

I implemented a lock-free ring buffer using atomic operations. The results:

```
$ perf stat -e cycles,instructions,cache-misses ./logger_lockfree
  Performance counter stats:
    3,200,000,000 cycles
    2,400,000,000 instructions
```

```
       12,000,000 cache-misses

Lock contention: 0%
Throughput: 2,400,000 messages/second
```

2.8× faster with zero lock contention. But the code was much more complex.

This chapter explores when lock-free data structures are worth the complexity.

The Textbook Story

Lock-free data structures promise: - **No blocking**: Threads never wait for locks - **Better scalability**: Performance improves with more cores - **No deadlocks**: Can't deadlock without locks - **Progress guarantees**: At least one thread always makes progress

The textbook pitch sounds perfect for multi-core systems.

The Reality Check

Lock-free programming is **hard**. Here's what the textbooks don't emphasize:

1. Memory Ordering Is Subtle

On modern CPUs, memory operations can be reordered. Consider this "simple" lock-free flag:

```
// Thread 1
data = 42;
ready = 1;  // Signal that data is ready

// Thread 2
if (ready) {
    use(data);  // Might see old value of data!
}
```

Problem: The CPU might reorder the writes in Thread 1, so Thread 2 sees `ready = 1` before `data = 42`.

Solution: Memory barriers (fences):

```
// Thread 1
data = 42;
__atomic_store_n(&ready, 1, __ATOMIC_RELEASE);  // Release barrier

// Thread 2
if (__atomic_load_n(&ready, __ATOMIC_ACQUIRE)) {  // Acquire barrier
```

```
    use(data);  // Now guaranteed to see data = 42
}
```

2. ABA Problem

The classic lock-free stack has a subtle bug:

```
typedef struct node {
    int value;
    struct node *next;
} node_t;

node_t *top;  // Stack top

void push(int value) {
    node_t *new_node = malloc(sizeof(node_t));
    new_node->value = value;
    do {
        new_node->next = top;
    } while (!__atomic_compare_exchange_n(&top, &new_node->next, new_node,
                                          0, __ATOMIC_SEQ_CST, __ATOMIC_SEQ_CST));
}
```

The ABA problem: 1. Thread 1 reads `top = A` 2. Thread 2 pops A, pops B, pushes A back (top is A again) 3. Thread 1's CAS succeeds (top is still A), but the stack is corrupted!

Solution: Use tagged pointers or hazard pointers (complex!).

3. Cache Line Ping-Pong

Lock-free doesn't mean cache-friendly. Consider a lock-free counter:

```
atomic_int counter = 0;

// 8 threads all incrementing
__atomic_fetch_add(&counter, 1, __ATOMIC_SEQ_CST);
```

Every increment causes a cache line to bounce between cores:

```
Core 0: Read counter (cache miss)
Core 0: Increment, write back
Core 1: Read counter (cache miss - invalidated by Core 0)
Core 1: Increment, write back
Core 2: Read counter (cache miss - invalidated by Core 1)
...
```

Result: Worse than a mutex for high contention!

I measured this with our logging system:

```
8 cores, atomic counter:
  Cache misses: 45M (same as mutex!)
  Throughput: 900K ops/sec (barely better than mutex)

8 cores, per-core counters (no sharing):
  Cache misses: 2M
  Throughput: 6.5M ops/sec
```

Lesson: Avoid sharing atomic variables across cores when possible.

Lock-Free Ring Buffer: A Practical Example

Let me show you the lock-free ring buffer I implemented for our logging system.

The Design

Key insight: Separate read and write indices, use atomic operations only for index updates.

```
typedef struct {
    char buffer[LOG_SIZE];
    atomic_int head;  // Read index
    atomic_int tail;  // Write index
} lockfree_log_t;

bool log_write(lockfree_log_t *log, const char *msg, int len) {
    int current_tail, next_tail, current_head;

    do {
        current_tail = __atomic_load_n(&log->tail, __ATOMIC_ACQUIRE);
        next_tail = (current_tail + len) % LOG_SIZE;
        current_head = __atomic_load_n(&log->head, __ATOMIC_ACQUIRE);

        // Check if buffer is full
        if (next_tail == current_head) {
            return false;  // Buffer full
        }

    } while (!__atomic_compare_exchange_n(&log->tail, &current_tail, next_tail,
                                          0, __ATOMIC_RELEASE, __ATOMIC_ACQUIRE));

    // Now we own the range [current_tail, next_tail)
    memcpy(&log->buffer[current_tail], msg, len);

    return true;
}
```

Why This Works

1. **CAS on tail**: Only one thread can claim a range
2. **No lock on data**: After claiming range, write without contention
3. **Memory ordering**: ACQUIRE/RELEASE ensures visibility

The Benchmark

I compared three implementations:

```
Test: 8 cores, 10M log messages

Mutex-based:
  Cycles: 8.5B
  Cache misses: 45M
  Throughput: 850K msg/sec
  Lock contention: 60%

Spinlock-based:
  Cycles: 7.2B
  Cache misses: 52M (worse!)
  Throughput: 1.1M msg/sec
  Lock contention: 45%

Lock-free (atomic CAS):
  Cycles: 3.2B
  Cache misses: 12M
  Throughput: 2.4M msg/sec
  Lock contention: 0%
```

The lock-free version was 2.8× faster than mutex, 2.2× faster than spinlock.

Why: - No syscalls (mutex uses futex) - No spinning (spinlock wastes cycles) - Less cache coherence traffic (only indices are atomic)

When Lock-Free Makes Sense

After implementing several lock-free data structures, I learned when they're worth the complexity.

1. High Contention, Short Critical Sections

If threads are constantly fighting for the same resource, and the work inside is quick (a few instructions), lock-free can win.

Example: Our logging system - High contention: 8 cores logging simultaneously - Short critical section: Just update an index - Result: 2.8× speedup

2. Real-Time Systems

In hard real-time systems, you can't afford priority inversion (low-priority thread holds lock, blocks high-priority thread).

Lock-free structures provide **wait-free** or **lock-free** progress guarantees.

Example: Interrupt handlers - Can't block in interrupt context - Lock-free queues allow interrupt → thread communication - Used in Linux kernel's ring buffers

3. Read-Heavy Workloads

If reads vastly outnumber writes, RCU (Read-Copy-Update) can eliminate read-side locks entirely.

Example: Linux kernel's RCU - Readers: No locks, no atomic operations, just memory barriers - Writers: Rare, use synchronization - Result: Millions of reads/second with zero contention

4. When NOT to Use Lock-Free

Don't use lock-free when:

Low contention: If threads rarely conflict, a mutex is simpler and just as fast.

```
Test: 2 cores, low contention
  Mutex: 1.2M ops/sec
  Lock-free: 1.3M ops/sec (only 8% faster, not worth complexity)
```

Complex operations: If the critical section is large (many instructions), lock-free becomes extremely complex.

Debugging: Lock-free bugs are notoriously hard to reproduce and debug. Use locks unless you have a proven performance problem.

Real-World Example: Linux Kernel's Per-CPU Variables

The Linux kernel uses a clever trick to avoid atomic operations: **per-CPU variables**.

Instead of one shared counter:

```
atomic_int global_counter;  // Shared, causes cache ping-pong
```

Use one counter per CPU:

```
DEFINE_PER_CPU(int, local_counter);  // One per CPU, no sharing

void increment_counter(void) {
    int cpu = smp_processor_id();
```

```
    per_cpu(local_counter, cpu)++;  // No atomic needed!
}

int read_total(void) {
    int total = 0;
    for_each_possible_cpu(cpu) {
        total += per_cpu(local_counter, cpu);
    }
    return total;
}
```

Why this works: - Each CPU has its own cache line - No cache coherence traffic - Reads are rare (only when you need the total)

I used this pattern in our logging system for statistics:

```
Per-CPU counters (messages logged per core):
  Cache misses: 0 (each core has its own cache line)
  Throughput: 8M increments/sec (8 cores × 1M each)

Shared atomic counter:
  Cache misses: 45M
  Throughput: 900K increments/sec
```

8.9× faster by avoiding sharing!

Memory Ordering on RISC-V

RISC-V has a relaxed memory model (RVWMO - RISC-V Weak Memory Ordering). This means you need explicit fences.

Fence Instructions

```
# Full fence (all memory operations)
fence rw, rw

# Acquire fence (load-load, load-store)
fence r, rw

# Release fence (load-store, store-store)
fence rw, w
```

C11 Atomics on RISC-V

GCC maps C11 atomics to RISC-V instructions:

```
// __ATOMIC_ACQUIRE
__atomic_load_n(&x, __ATOMIC_ACQUIRE);
```

Compiles to:

```
ld   a0, 0(a1)      # Load
fence r, rw         # Acquire fence
```

```
// __ATOMIC_RELEASE
__atomic_store_n(&x, 42, __ATOMIC_RELEASE);
```

Compiles to:

```
fence rw, w         # Release fence
sd   a0, 0(a1)      # Store
```

The Cost of Fences

Fences aren't free. I measured the overhead:

```
Test: 1M atomic loads (RISC-V U74 @ 1.2 GHz)

Relaxed (no fence):
  Cycles: 1.2M (1 cycle per load)

Acquire (fence r, rw):
  Cycles: 8.5M (8.5 cycles per load)

Sequential (fence rw, rw):
  Cycles: 12.3M (12.3 cycles per load)
```

Lesson: Use the weakest memory ordering that's correct. Don't default to `__ATOMIC_SEQ_CST`.

Summary

The 60% problem was solved. The lock-free ring buffer increased throughput from 850,000 to 2.4 million messages per second—a 2.8× improvement. Lock contention dropped from 60% to zero. But the code became significantly more complex, requiring careful attention to memory ordering and the ABA problem.

Key insights:

1. **Lock-free isn't always faster**. For low contention or complex critical sections, mutexes are simpler and just as fast.
2. **Memory ordering is subtle**. You need ACQUIRE/RELEASE barriers to ensure visibility across cores. RISC-V's relaxed memory model makes this explicit.

3. **Cache coherence matters**. Atomic operations cause cache line ping-pong. Avoid sharing atomic variables across cores when possible.

4. **The ABA problem is real**. Lock-free stacks and queues need tagged pointers or hazard pointers to avoid corruption.

5. **Per-CPU variables eliminate contention**. If you can partition data by CPU, you avoid atomic operations entirely. This is often 5-10× faster than shared atomics.

The numbers from our logging system: - Mutex: 850K msg/sec, 60% lock contention - Lock-free: 2.4M msg/sec, 0% contention - Per-CPU stats: 8M increments/sec (vs 900K with shared atomic)

Lock-free data structures are a powerful tool, but use them only when profiling shows lock contention is a real bottleneck. The complexity cost is high.

Next chapter: String processing and cache-efficient algorithms for text manipulation.

Chapter 14: String Processing and Cache Efficiency

Part IV: Advanced Topics

> "There are only two hard things in Computer Science: cache invalidation and naming things." — Phil Karlton

The Throughput Gap

The log parser was processing 800,000 lines per second. The requirement was 3 million lines per second. We were missing the target by 3.75×.

The tool's job was to parse log lines in real-time, extracting timestamps, log levels, and messages from millions of lines per second. For 1 million log lines, the current implementation took 1.25 seconds—far too slow for real-time analysis.

The profiler showed 85 million cache misses. For string processing, that seemed excessive.

The implementation used standard C string functions—simple, readable, and apparently slow:

```
typedef struct {
    char timestamp[32];
    char level[16];
    char message[256];
} log_entry_t;
```

```c
void parse_log_line(const char *line, log_entry_t *entry) {
    // Format: "2024-12-05 10:30:45 [INFO] System started"
    char *p = strchr(line, '[');
    if (!p) return;

    // Extract timestamp
    int ts_len = p - line - 1;
    strncpy(entry->timestamp, line, ts_len);
    entry->timestamp[ts_len] = '\0';

    // Extract level
    char *end = strchr(p, ']');
    int level_len = end - p - 1;
    strncpy(entry->level, p + 1, level_len);
    entry->level[level_len] = '\0';

    // Extract message
    strcpy(entry->message, end + 2);
}
```

Simple and readable. But slow:

```
$ perf stat -e cycles,cache-misses ./log_parser_naive
  Performance counter stats:
    12,500,000,000 cycles
        85,000,000 cache-misses

Throughput: 800,000 lines/second
```

For 1 million log lines, this took 1.25 seconds. Too slow for real-time analysis.

I rewrote it with cache-conscious string processing. The results:

```
$ perf stat -e cycles,cache-misses ./log_parser_optimized
  Performance counter stats:
     2,800,000,000 cycles
        12,000,000 cache-misses

Throughput: 3,600,000 lines/second
```

4.5× faster with 7× fewer cache misses.

This chapter explores how to make string processing cache-efficient.

The Textbook Story

String processing in C is straightforward: - `strlen()`: Count characters until '\0' - `strcpy()`: Copy until '\0' - `strcmp()`: Compare until difference or '\0' - `strstr()`: Find substring

The textbook algorithms are simple and correct. But they're not cache-efficient.

The Reality Check: Why String Functions Are Slow

1. Multiple Passes Over Data

Consider this common pattern:

```
char *trim_whitespace(char *str) {
    // Pass 1: Find start
    while (isspace(*str)) str++;

    // Pass 2: Find end
    char *end = str + strlen(str) - 1;
    while (end > str && isspace(*end)) end--;

    // Pass 3: Null-terminate
    *(end + 1) = '\0';

    return str;
}
```

Three passes over the string! Each pass is a potential cache miss.

2. Unpredictable Length

`strlen()` must scan until '\0':

```
size_t strlen(const char *s) {
    const char *p = s;
    while (*p) p++;
    return p - s;
}
```

For a 1000-character string: - 1000 bytes to scan - ~16 cache lines (64 bytes each) - If string isn't in cache: 16 cache misses

3. Character-by-Character Processing

`strcmp()` compares one byte at a time:

```
int strcmp(const char *s1, const char *s2) {
    while (*s1 && *s1 == *s2) {
```

```
        s1++;
        s2++;
    }
    return *(unsigned char *)s1 - *(unsigned char *)s2;
}
```

Modern CPUs can compare 8 bytes (64 bits) at once, but `strcmp()` doesn't use this.

Optimization 1: Single-Pass Parsing

Instead of multiple passes, process the string once:

```
void parse_log_line_optimized(const char *line, log_entry_t *entry) {
    const char *p = line;
    char *out;

    // Single pass: extract all fields

    // Timestamp (until space before '[')
    out = entry->timestamp;
    while (*p && *p != '[') {
        if (*p != ' ' || *(p+1) != '[') {
            *out++ = *p;
        }
        p++;
    }
    *out = '\0';

    // Level (between '[' and ']')
    if (*p == '[') p++;
    out = entry->level;
    while (*p && *p != ']') {
        *out++ = *p++;
    }
    *out = '\0';

    // Message (after '] ')
    if (*p == ']') p++;
    if (*p == ' ') p++;
    strcpy(entry->message, p);
}
```

Result: - Old: 3 passes (strchr, strchr, strcpy) - New: 1 pass - Speedup: 2.1×

Optimization 2: SIMD String Operations

Modern CPUs have SIMD (Single Instruction, Multiple Data) instructions that can process multiple bytes at once.

strlen() with SIMD

GCC's optimized `strlen()` uses SIMD on x86:

```c
// Simplified version of glibc's strlen
size_t strlen_simd(const char *s) {
    const char *p = s;

    // Process 16 bytes at a time with SSE2
    while ((uintptr_t)p & 15) {  // Align to 16 bytes
        if (*p == 0) return p - s;
        p++;
    }

    __m128i zero = _mm_setzero_si128();
    while (1) {
        __m128i data = _mm_load_si128((__m128i *)p);
        __m128i cmp = _mm_cmpeq_epi8(data, zero);
        int mask = _mm_movemask_epi8(cmp);

        if (mask != 0) {
            return p - s + __builtin_ctz(mask);
        }
        p += 16;
    }
}
```

Speedup: 4-8× faster than byte-by-byte for long strings.

RISC-V Vector Extension

RISC-V has a vector extension (RVV) for SIMD operations:

```
# strlen with RVV
strlen_rvv:
    li      t0, 0           # length = 0
    vsetvli t1, zero, e8    # Set vector length for 8-bit elements

loop:
    vle8.v  v0, (a0)        # Load vector of bytes
    vmseq.vi v1, v0, 0      # Compare with zero
    vfirst.m t2, v1         # Find first match
    bgez    t2, found       # If found, exit
```

```
    add     t0, t0, t1      # length += vector_length
    add     a0, a0, t1      # ptr += vector_length
    j       loop

found:
    add     a0, t0, t2      # length + position
    ret
```

I benchmarked different `strlen()` implementations on RISC-V:

```
Test: strlen() on 10,000 strings (avg length: 100 bytes)

Naive byte-by-byte:
  Cycles: 12.5M
  Cache misses: 850K

Optimized (word-at-a-time):
  Cycles: 4.2M
  Cache misses: 320K
  Speedup: 3.0×

RVV (vector extension):
  Cycles: 1.8M
  Cache misses: 180K
  Speedup: 6.9×
```

Lesson: Use SIMD/vector instructions for bulk string operations when available.

Optimization 3: Small String Optimization (SSO)

Many strings are short. Instead of allocating heap memory, store short strings inline.

The Problem with Heap Allocation

Standard C++ `std::string` allocates on heap:

```
std::string s = "hello";  // Allocates 6 bytes on heap
```

Cost: - `malloc()`: ~100 cycles - Cache miss when accessing string data - Fragmentation

Small String Optimization

Store short strings (15 bytes) inside the string object:

```c
typedef struct {
    union {
        struct {
            char *ptr;      // Heap pointer (for long strings)
            size_t len;
            size_t cap;
        } heap;
        struct {
            char data[16];  // Inline storage (for short strings)
            uint8_t len;    // Length in low byte
        } sso;
    } u;
} string_t;

#define SSO_MAX 15

void string_init(string_t *s, const char *str) {
    size_t len = strlen(str);
    if (len <= SSO_MAX) {
        // Use SSO
        memcpy(s->u.sso.data, str, len);
        s->u.sso.data[len] = '\0';
        s->u.sso.len = len | 0x80;  // Set high bit to mark SSO
    } else {
        // Use heap
        s->u.heap.ptr = malloc(len + 1);
        memcpy(s->u.heap.ptr, str, len + 1);
        s->u.heap.len = len;
        s->u.heap.cap = len + 1;
    }
}
```

The Benchmark

I measured the impact on our log parser:

```
Test: Parse 1M log lines (avg message length: 45 bytes)

Without SSO (all heap):
  Cycles: 8.5B
  Cache misses: 45M
  malloc calls: 3M
  Throughput: 1.2M lines/sec

With SSO (messages  15 bytes inline):
  Cycles: 5.8B
```

```
Cache misses: 28M
malloc calls: 1.8M (40% reduction)
Throughput: 1.7M lines/sec
Speedup: 1.4×
```

Why it helps: - No malloc for short strings (saves ~100 cycles each) - Better cache locality (data is inline) - Less memory fragmentation

---

Optimization 4: String Interning

If you have many duplicate strings, store each unique string once and use pointers.

The Problem

Our log parser saw many repeated strings:

```
[INFO] System started
[INFO] System started
[INFO] System started
[ERROR] Connection failed
[ERROR] Connection failed
[INFO] System started
...
```

Storing each string separately wastes memory and cache space.

String Interning

Store unique strings in a hash table, return pointers to existing strings:

```c
typedef struct {
    hash_table_t *table;  // Maps string → interned pointer
} string_intern_t;

const char *string_intern(string_intern_t *intern, const char *str) {
    // Check if already interned
    const char *existing = hash_table_get(intern->table, str);
    if (existing) {
        return existing;  // Return existing pointer
    }

    // Not found, add to table
    char *copy = strdup(str);
    hash_table_put(intern->table, copy, copy);
    return copy;
}
```

Now instead of storing full strings:

```
// Before: Each entry stores full string
typedef struct {
    char level[16];     // "INFO", "ERROR", etc.
    char message[256];
} log_entry_t;
```

Use pointers to interned strings:

```
// After: Each entry stores pointer
typedef struct {
    const char *level;    // Points to interned string
    const char *message;
} log_entry_t;
```

The Benchmark

```
Test: Parse 1M log lines (10 unique log levels, 1000 unique messages)

Without interning:
  Memory: 256 MB (1M × 256 bytes)
  Cache misses: 45M

With interning:
  Memory: 8 MB (1M × 8 bytes pointers + 50 KB unique strings)
  Cache misses: 12M (3.8× fewer)
  Speedup: 2.8×
```

Why it helps: - 32× less memory (256 MB → 8 MB) - Better cache utilization (fewer unique strings to cache) - String comparison becomes pointer comparison (O(1) instead of O(n))

Optimization 5: Cache-Friendly String Search

The naive `strstr()` is slow for long strings. We can do better.

Boyer-Moore-Horspool Algorithm

Instead of checking every position, skip ahead based on mismatches:

```
const char *strstr_bmh(const char *text, const char *pattern) {
    size_t n = strlen(text);
    size_t m = strlen(pattern);

    if (m > n) return NULL;
```

```
    // Build skip table
    size_t skip[256];
    for (int i = 0; i < 256; i++) {
        skip[i] = m;
    }
    for (size_t i = 0; i < m - 1; i++) {
        skip[(unsigned char)pattern[i]] = m - 1 - i;
    }

    // Search
    size_t pos = 0;
    while (pos <= n - m) {
        size_t i = m - 1;
        while (i < m && text[pos + i] == pattern[i]) {
            if (i == 0) return &text[pos];
            i--;
        }
        pos += skip[(unsigned char)text[pos + m - 1]];
    }

    return NULL;
}
```

The Benchmark

```
Test: Search for "ERROR" in 1M log lines (avg line length: 100 bytes)

Naive strstr():
  Cycles: 18.5B
  Cache misses: 85M
  Throughput: 540K searches/sec

Boyer-Moore-Horspool:
  Cycles: 4.2B
  Cache misses: 22M
  Throughput: 2.4M searches/sec
  Speedup: 4.4×
```

Why it helps: - Skips characters instead of checking every position - Better cache behavior (fewer memory accesses) - Especially fast when pattern doesn't match often

---

Real-World Example: Linux Kernel's String Functions

The Linux kernel has highly optimized string functions that are cache-aware.

`strcmp()` with Word-at-a-Time

Instead of comparing byte-by-byte, compare 8 bytes at once:

```
// Simplified version of kernel's strcmp
int strcmp_fast(const char *s1, const char *s2) {
    unsigned long *l1 = (unsigned long *)s1;
    unsigned long *l2 = (unsigned long *)s2;

    // Compare 8 bytes at a time
    while (1) {
        unsigned long w1 = *l1;
        unsigned long w2 = *l2;

        if (w1 != w2) {
            // Found difference, find exact byte
            for (int i = 0; i < 8; i++) {
                if (((char *)&w1)[i] != ((char *)&w2)[i]) {
                    return ((unsigned char *)&w1)[i] - ((unsigned char *)&w2)[i];
                }
                if (((char *)&w1)[i] == 0) {
                    return 0;
                }
            }
        }

        // Check for null terminator
        if (has_zero(w1)) return 0;

        l1++;
        l2++;
    }
}
```

The `has_zero()` macro uses bit tricks to detect zero bytes:

```
#define ONES ((unsigned long)-1/0xFF)
#define HIGHS (ONES * 0x80)

#define has_zero(x) (((x) - ONES) & ~(x) & HIGHS)
```

Speedup: 3-5× faster than byte-by-byte for long strings.

Putting It All Together: Optimized Log Parser

Let me show you the final optimized log parser that combines all these techniques:

```c
typedef struct {
    string_intern_t *intern;  // For log levels
    char buffer[4096];        // Reusable buffer
} log_parser_t;

void parse_log_optimized(log_parser_t *parser, const char *line, log_entry_t *entry) {
    const char *p = line;
    char *out = parser->buffer;

    // Single-pass parsing

    // Extract timestamp (inline, no allocation)
    while (*p && *p != '[') {
        if (*p != ' ' || *(p+1) != '[') {
            *out++ = *p;
        }
        p++;
    }
    *out++ = '\0';
    entry->timestamp = parser->buffer;

    // Extract level (interned)
    if (*p == '[') p++;
    char *level_start = out;
    while (*p && *p != ']') {
        *out++ = *p++;
    }
    *out++ = '\0';
    entry->level = string_intern(parser->intern, level_start);

    // Extract message (SSO or heap)
    if (*p == ']') p++;
    if (*p == ' ') p++;
    entry->message = p;
}
```

Final Benchmark

```
Test: Parse 1M log lines

Original (naive):
  Cycles: 12.5B
  Cache misses: 85M
  Memory: 256 MB
  Throughput: 800K lines/sec
```

```
Optimized (all techniques):
  Cycles: 2.8B
  Cache misses: 12M
  Memory: 32 MB
  Throughput: 3.6M lines/sec

Speedup: 4.5×
Cache miss reduction: 7.1×
Memory reduction: 8×
```

Summary

The throughput gap was closed. The log parser now processes 3.6 million lines per second, up from 800,000—a 4.5× improvement that exceeds the 3 million line target. Cache misses dropped from 85 million to 12 million, and the parser can now handle real-time analysis.

Key insights:

1. **Single-pass parsing is crucial**. Multiple passes over strings waste cache bandwidth. Process each character once.
2. **SIMD/vector instructions help**. For bulk operations like `strlen()` and `strcmp()`, SIMD can provide 4-8× speedup.
3. **Small String Optimization (SSO) eliminates allocations**. For strings 15 bytes, storing inline saves ~100 cycles per string and improves cache locality.
4. **String interning reduces memory and cache pressure**. For repeated strings, storing each unique string once can reduce memory by 10-100× and improve cache hit rates.
5. **Word-at-a-time comparison is faster**. Comparing 8 bytes at once instead of 1 byte at a time provides 3-5× speedup for `strcmp()`.

The numbers from our log parser: - Single-pass parsing: 2.1× faster - SIMD strlen: 6.9× faster - SSO: 1.4× faster, 40% fewer mallocs - String interning: 2.8× faster, 32× less memory - Boyer-Moore-Horspool search: 4.4× faster

String processing is often I/O bound or cache bound, not CPU bound. Focus on reducing cache misses and memory allocations.

Next chapter: Graphs and networks—how to represent and traverse graph structures efficiently in cache-constrained systems.

Chapter 15: Graphs and Cache-Efficient Traversal

Part IV: Advanced Topics

> "The purpose of abstraction is not to be vague, but to create a new semantic level in which one can be absolutely precise." — Edsger W. Dijkstra

The Cache Miss Explosion

The network topology discovery was taking 37.5 milliseconds to traverse 500 switches. That doesn't sound slow until you look at the cache miss count: 8.5 million cache misses. For 500 nodes, that's 17,000 cache misses per node.

Something was fundamentally wrong with the data structure.

The tool's job was straightforward: discover network topology by traversing a graph of connected devices. Each switch had up to 48 ports, and we needed to find all reachable devices from a starting point using breadth-first search.

The implementation looked textbook-correct—an adjacency list with standard BFS:

```
typedef struct node {
    int id;
    struct node **neighbors;  // Array of pointers
    int num_neighbors;
} node_t;

void bfs(node_t *start) {
    queue_t *q = queue_create();
    bool *visited = calloc(MAX_NODES, sizeof(bool));

    queue_push(q, start);
    visited[start->id] = true;

    while (!queue_empty(q)) {
        node_t *node = queue_pop(q);
        process(node);

        for (int i = 0; i < node->num_neighbors; i++) {
            node_t *neighbor = node->neighbors[i];
            if (!visited[neighbor->id]) {
                visited[neighbor->id] = true;
                queue_push(q, neighbor);
```

```
            }
        }
    }
}
```

For a network with 500 switches (average 12 connections each), this took:

```
$ perf stat -e cycles,cache-misses ./network_discovery_naive
  Performance counter stats:
    45,000,000 cycles
     8,500,000 cache-misses

Traversal time: 37.5 ms
```

8.5 million cache misses for 500 nodes? That's **17,000 cache misses per node**!

I rewrote it with cache-conscious graph representation. The results:

```
$ perf stat -e cycles,cache-misses ./network_discovery_optimized
  Performance counter stats:
    12,000,000 cycles
     1,200,000 cache-misses

Traversal time: 10 ms
```

3.75× faster with 7× fewer cache misses.

This chapter explores how to represent and traverse graphs efficiently.

The Textbook Story

Graphs are typically represented in two ways:

1. Adjacency Matrix

A 2D array where `matrix[i][j] = 1` if there's an edge from node i to node j:

```
bool adj_matrix[MAX_NODES][MAX_NODES];

// Check if edge exists
if (adj_matrix[u][v]) {
    // Edge from u to v exists
}
```

Pros: O(1) edge lookup
Cons: O(n^2) space, even for sparse graphs

2. Adjacency List

Each node stores a list of its neighbors:

```
typedef struct {
    int *neighbors;
    int num_neighbors;
} node_t;

node_t nodes[MAX_NODES];
```

Pros: O(n + m) space (n nodes, m edges)
Cons: O(degree) edge lookup

The textbook says: "Use adjacency matrix for dense graphs, adjacency list for sparse graphs."

The Reality Check: Why Standard Representations Are Slow

1. Pointer Chasing in Adjacency Lists

The standard adjacency list uses pointers:

```
typedef struct edge {
    int dest;
    struct edge *next;  // Linked list of edges
} edge_t;

typedef struct {
    edge_t *edges;  // Pointer to first edge
} node_t;
```

Problem: Each edge is a separate allocation, scattered in memory.

Traversing neighbors:

```
Node → Edge1 (cache miss) → Edge2 (cache miss) → Edge3 (cache miss) ...
```

For a node with 12 neighbors: **12 cache misses** just to read the neighbor list!

2. Poor Locality in BFS Queue

Standard BFS uses a queue of pointers:

```
queue_push(q, neighbor);  // Push pointer to node
```

Problem: Nodes are processed in BFS order, but they're scattered in memory.

```
Queue: [Node5, Node12, Node3, Node45, ...]
       Each node is in a different cache line!
```

3. Random Access to Visited Array

The `visited` array is indexed by node ID:

```
visited[neighbor->id] = true;
```

If node IDs are not sequential or clustered, this causes random memory access.

Optimization 1: Compact Adjacency List

Instead of pointers, store neighbors in a contiguous array:

```
typedef struct {
    int *neighbors;      // Contiguous array of neighbor IDs
    int num_neighbors;
} node_t;

typedef struct {
    node_t *nodes;
    int *edge_data;      // All edges in one array
    int num_nodes;
} graph_t;

graph_t *graph_create(int num_nodes, int num_edges) {
    graph_t *g = malloc(sizeof(graph_t));
    g->nodes = malloc(num_nodes * sizeof(node_t));
    g->edge_data = malloc(num_edges * sizeof(int));
    g->num_nodes = num_nodes;

    // Nodes point into edge_data array
    int offset = 0;
    for (int i = 0; i < num_nodes; i++) {
        g->nodes[i].neighbors = &g->edge_data[offset];
        g->nodes[i].num_neighbors = /* ... */;
        offset += g->nodes[i].num_neighbors;
    }

    return g;
}
```

Why this helps: - All edges in one contiguous array (better prefetching) - No pointer chasing (neighbors are sequential) - Better cache utilization

The Benchmark

```
Test: BFS on 500-node graph (avg degree: 12)
```

```
Linked list adjacency list:
  Cache misses: 8.5M
  Cycles: 45M

Compact adjacency list:
  Cache misses: 2.8M (3× fewer)
  Cycles: 18M
  Speedup: 2.5×
```

Optimization 2: Cache-Oblivious BFS

Standard BFS processes nodes level-by-level, but nodes in the same level might be far apart in memory.

Blocked BFS

Process nodes in cache-sized blocks:

```
#define BLOCK_SIZE 64  // Process 64 nodes at a time

void bfs_blocked(graph_t *g, int start) {
    bool *visited = calloc(g->num_nodes, sizeof(bool));
    int *queue = malloc(g->num_nodes * sizeof(int));
    int head = 0, tail = 0;

    queue[tail++] = start;
    visited[start] = true;

    while (head < tail) {
        int block_end = (head + BLOCK_SIZE < tail) ? head + BLOCK_SIZE : tail;

        // Process a block of nodes
        for (int i = head; i < block_end; i++) {
            int node_id = queue[i];
            node_t *node = &g->nodes[node_id];

            process(node);

            // Add neighbors to queue
            for (int j = 0; j < node->num_neighbors; j++) {
                int neighbor = node->neighbors[j];
                if (!visited[neighbor]) {
                    visited[neighbor] = true;
                    queue[tail++] = neighbor;
                }
```

```
            }
        }

        head = block_end;
    }
}
```

Why this helps: - Process multiple nodes before moving to next level - Better temporal locality (reuse visited array in cache) - Amortize queue overhead

The Benchmark

```
Test: BFS on 500-node graph

Standard BFS:
  Cache misses: 2.8M
  Cycles: 18M

Blocked BFS (block size 64):
  Cache misses: 1.5M
  Cycles: 11M
  Speedup: 1.6×
```

Optimization 3: Compressed Sparse Row (CSR) Format

For very large sparse graphs, CSR format is even more compact:

```c
typedef struct {
    int *row_ptr;     // row_ptr[i] = start of node i's neighbors
    int *col_idx;     // col_idx[j] = neighbor ID
    int num_nodes;
    int num_edges;
} csr_graph_t;

csr_graph_t *graph_to_csr(graph_t *g) {
    csr_graph_t *csr = malloc(sizeof(csr_graph_t));
    csr->num_nodes = g->num_nodes;
    csr->num_edges = /* total edges */;

    csr->row_ptr = malloc((g->num_nodes + 1) * sizeof(int));
    csr->col_idx = malloc(csr->num_edges * sizeof(int));

    int offset = 0;
    for (int i = 0; i < g->num_nodes; i++) {
        csr->row_ptr[i] = offset;
        for (int j = 0; j < g->nodes[i].num_neighbors; j++) {
```

```
            csr->col_idx[offset++] = g->nodes[i].neighbors[j];
        }
    }
    csr->row_ptr[g->num_nodes] = offset;

    return csr;
}

// Access neighbors of node i
void visit_neighbors(csr_graph_t *g, int node_id) {
    int start = g->row_ptr[node_id];
    int end = g->row_ptr[node_id + 1];

    for (int i = start; i < end; i++) {
        int neighbor = g->col_idx[i];
        // Process neighbor
    }
}
```

Memory layout:

```
row_ptr: [0, 3, 7, 10, ...]  (node 0 has neighbors at indices 0-2)
col_idx: [1, 2, 5, 0, 3, 4, 6, ...]  (actual neighbor IDs)
```

Advantages: - Minimal memory overhead (just two arrays) - Sequential access to neighbors (excellent prefetching) - Cache-friendly (all data is contiguous)

The Benchmark

```
Test: 10,000-node graph, 120,000 edges

Adjacency list (pointers):
  Memory: 2.4 MB
  Cache misses: 85M
  BFS time: 180 ms

CSR format:
  Memory: 0.96 MB (2.5× less)
  Cache misses: 18M (4.7× fewer)
  BFS time: 42 ms
  Speedup: 4.3×
```

Optimization 4: Node Reordering for Locality

If you can reorder node IDs, place connected nodes close together in memory.

Breadth-First Ordering

Assign node IDs in BFS order:

```
void reorder_bfs(graph_t *g, int start) {
    int *new_id = malloc(g->num_nodes * sizeof(int));
    int *old_id = malloc(g->num_nodes * sizeof(int));
    bool *visited = calloc(g->num_nodes, sizeof(bool));

    int next_id = 0;
    queue_t *q = queue_create();
    queue_push(q, start);
    visited[start] = true;

    while (!queue_empty(q)) {
        int node = queue_pop(q);
        new_id[node] = next_id;
        old_id[next_id] = node;
        next_id++;

        // Visit neighbors
        for (int i = 0; i < g->nodes[node].num_neighbors; i++) {
            int neighbor = g->nodes[node].neighbors[i];
            if (!visited[neighbor]) {
                visited[neighbor] = true;
                queue_push(q, neighbor);
            }
        }
    }

    // Rebuild graph with new IDs
    // ...
}
```

Why this helps: - Nodes visited together are numbered sequentially - Better cache locality during traversal - Visited array accesses are more sequential

The Benchmark

```
Test: BFS on 500-node graph

Random node IDs:
  Cache misses: 1.5M
  Cycles: 11M

BFS-ordered node IDs:
  Cache misses: 0.8M
```

```
Cycles: 7.5M
Speedup: 1.5×
```

Optimization 5: Parallel Graph Traversal

On multi-core systems, we can parallelize BFS using level-synchronous approach.

Level-Synchronous BFS

Process each level in parallel:

```
void bfs_parallel(csr_graph_t *g, int start, int num_threads) {
    bool *visited = calloc(g->num_nodes, sizeof(bool));
    int *current_level = malloc(g->num_nodes * sizeof(int));
    int *next_level = malloc(g->num_nodes * sizeof(int));

    int current_size = 1;
    current_level[0] = start;
    visited[start] = true;

    while (current_size > 0) {
        atomic_int next_size = 0;

        // Process current level in parallel
        #pragma omp parallel for num_threads(num_threads)
        for (int i = 0; i < current_size; i++) {
            int node = current_level[i];
            int start = g->row_ptr[node];
            int end = g->row_ptr[node + 1];

            for (int j = start; j < end; j++) {
                int neighbor = g->col_idx[j];

                // Atomic check-and-set
                bool expected = false;
                if (__atomic_compare_exchange_n(&visited[neighbor], &expected, true,
                                                0, __ATOMIC_SEQ_CST, __ATOMIC_SEQ_CST)) {
                    int pos = __atomic_fetch_add(&next_size, 1, __ATOMIC_SEQ_CST);
                    next_level[pos] = neighbor;
                }
            }
        }

        // Swap levels
```

```
        int *temp = current_level;
        current_level = next_level;
        next_level = temp;
        current_size = next_size;
    }
}
```

The Benchmark

```
Test: BFS on 10,000-node graph (RISC-V 8-core @ 1.2 GHz)

Sequential BFS:
  Cycles: 120M
  Time: 100 ms

Parallel BFS (2 cores):
  Cycles: 68M
  Time: 57 ms
  Speedup: 1.75×

Parallel BFS (4 cores):
  Cycles: 38M
  Time: 32 ms
  Speedup: 3.1×

Parallel BFS (8 cores):
  Cycles: 24M
  Time: 20 ms
  Speedup: 5.0×
```

Not perfect scaling (8× speedup on 8 cores) due to: - Synchronization overhead (atomic operations) - Load imbalance (some levels have few nodes) - Cache coherence traffic

But still a significant improvement for large graphs.

Real-World Example: Linux Kernel's Radix Tree for Page Cache

The Linux kernel uses a radix tree (a specialized graph structure) for the page cache.

The Problem

The kernel needs to map file offsets to physical pages: - Millions of pages per file - Sparse mapping (not all offsets have pages) - Fast lookup (O(log n) or better)

The Solution: Radix Tree

A 64-way tree where each level represents 6 bits of the offset:

```
#define RADIX_TREE_MAP_SHIFT 6
#define RADIX_TREE_MAP_SIZE (1 << RADIX_TREE_MAP_SHIFT)  // 64

struct radix_tree_node {
    void *slots[RADIX_TREE_MAP_SIZE];  // 64 pointers
    unsigned long tags[3][RADIX_TREE_MAP_SIZE / BITS_PER_LONG];
};
```

Why 64-way: - One node fits in one cache line (64 pointers × 8 bytes = 512 bytes 8 cache lines) - Shallow tree (depth 11 for 64-bit offsets) - Good balance between memory and cache misses

The Performance

```
Lookup in radix tree (depth 3):
  Cache misses: 3 (one per level)
  Cycles: ~50

Lookup in binary tree (depth 20):
  Cache misses: 20
  Cycles: ~300

Speedup: 6×
```

Putting It All Together: Optimized Network Discovery

Here's the final optimized version combining all techniques:

```
typedef struct {
    int *row_ptr;
    int *col_idx;
    int num_nodes;
    int num_edges;
} network_graph_t;

void discover_network_optimized(network_graph_t *g, int start) {
    // Use bitmap for visited (cache-friendly)
    uint64_t *visited = calloc((g->num_nodes + 63) / 64, sizeof(uint64_t));

    // Use array-based queue (not linked list)
    int *queue = malloc(g->num_nodes * sizeof(int));
    int head = 0, tail = 0;
```

```
    queue[tail++] = start;
    visited[start / 64] |= (1UL << (start % 64));

    while (head < tail) {
        // Process in blocks for better cache reuse
        int block_end = (head + 64 < tail) ? head + 64 : tail;

        for (int i = head; i < block_end; i++) {
            int node = queue[i];
            process_device(node);

            // Sequential access to neighbors (CSR format)
            int start_idx = g->row_ptr[node];
            int end_idx = g->row_ptr[node + 1];

            for (int j = start_idx; j < end_idx; j++) {
                int neighbor = g->col_idx[j];
                uint64_t mask = 1UL << (neighbor % 64);
                int word = neighbor / 64;

                if (!(visited[word] & mask)) {
                    visited[word] |= mask;
                    queue[tail++] = neighbor;
                }
            }
        }

        head = block_end;
    }

    free(visited);
    free(queue);
}
```

Final Benchmark

```
Test: Network discovery, 500 switches, avg 12 connections

Original (adjacency list, linked queue):
  Cycles: 45M
  Cache misses: 8.5M
  Memory: 128 KB
  Time: 37.5 ms

Optimized (CSR, blocked BFS, bitmap):
```

```
  Cycles: 7.5M
  Cache misses: 0.8M
  Memory: 24 KB
  Time: 6.2 ms

Speedup: 6.0×
Cache miss reduction: 10.6×
Memory reduction: 5.3×
```

Summary

The cache miss explosion was tamed. Network discovery time dropped from 37.5 ms to 6.2 ms—a 6× improvement. Cache misses dropped from 8.5 million to 0.8 million—a 10.6× reduction. The graph traversal went from 17,000 cache misses per node to just 1,600.

Key insights:

1. **Compact adjacency lists beat pointer-based lists**. Storing all edges in one contiguous array eliminates pointer chasing and improves prefetching. 2.5× speedup.
2. **CSR format is optimal for sparse graphs**. Two arrays (row_ptr and col_idx) provide minimal memory overhead and excellent cache behavior. 4.3× speedup over pointer-based lists.
3. **Blocked BFS improves temporal locality**. Processing nodes in cache-sized blocks (64 nodes) reuses the visited array in cache. 1.6× speedup.
4. **Node reordering matters**. Assigning IDs in BFS order places connected nodes close in memory. 1.5× speedup.
5. **Parallel BFS scales reasonably**. Level-synchronous BFS with atomic operations achieved 5× speedup on 8 cores (62% efficiency).

The numbers from network discovery: - Compact adjacency list: 2.5× faster than pointers - CSR format: 4.3× faster, 2.5× less memory - Blocked BFS: 1.6× faster - BFS ordering: 1.5× faster - Combined: 6× faster, 10.6× fewer cache misses

Graph traversal is memory-bound. Focus on cache-friendly representations and access patterns.

Next chapter: Bloom filters and probabilistic data structures—trading accuracy for speed and memory.

Chapter 16: Bloom Filters and Probabilistic Data Structures

Part IV: Advanced Topics

"Premature optimization is the root of all evil." — Donald Knuth

The Memory Crisis

The web crawler was consuming 128 MB of RAM just to track visited URLs. On an embedded device with 256 MB total memory, this was half the available RAM—gone.

The crawler's job was simple: track which URLs had been visited to avoid crawling the same page twice. After processing 1 million URLs (average length: 80 bytes), the hash table storing these URLs had grown to 96 MB, plus overhead.

"Can we trade accuracy for memory?" my manager asked during the code review. "We can tolerate a few duplicate crawls if it saves significant memory."

That question changed everything. Perfect accuracy wasn't actually required. If we occasionally crawled the same page twice, it would waste some bandwidth but wouldn't break anything. The real constraint was memory.

The current approach used a straightforward hash table:

```
hash_table_t *visited_urls;  // Stores full URLs

bool is_visited(const char *url) {
    return hash_table_contains(visited_urls, url);
}

void mark_visited(const char *url) {
    hash_table_insert(visited_urls, url, NULL);
}
```

After crawling 1 million URLs (average length: 80 bytes), the hash table consumed:

```
$ ./crawler_hashtable
Memory usage: 128 MB
  Hash table: 96 MB (1M URLs × 80 bytes + overhead)
  Other: 32 MB

Lookup time: 150 ns/lookup (including cache misses)
```

128 MB for just tracking visited URLs! On an embedded device with 256 MB total RAM, this was unacceptable.

"Can we trade accuracy for memory?" my manager asked. "We can tolerate a few duplicate crawls if it saves significant memory."

I implemented a Bloom filter. The results:

```
$ ./crawler_bloom
Memory usage: 18 MB
  Bloom filter: 1.2 MB (10 bits per URL)
  Other: 16.8 MB

Lookup time: 45 ns/lookup
False positive rate: 0.8% (8,000 false positives out of 1M)

Memory reduction: 10.7× (128 MB → 12 MB)
Speedup: 3.3× (150 ns → 45 ns)
```

10.7× less memory and **3.3× faster**, with only 0.8% false positives (which just meant crawling a few pages twice—acceptable).

This chapter explores probabilistic data structures that trade perfect accuracy for massive memory savings.

The Textbook Story

A **Bloom filter** is a space-efficient probabilistic data structure that tests whether an element is in a set.

Properties: - **No false negatives**: If it says "not in set", it's definitely not in set - **Possible false positives**: If it says "in set", it might be wrong - **Space-efficient**: Uses bits instead of storing full elements - **Fast**: O(k) where k is number of hash functions (typically 3-10)

The textbook pitch: "Use Bloom filters when you can tolerate false positives and need to save memory."

The Reality Check: How Bloom Filters Work

Basic Structure

A Bloom filter is a bit array of size m, with k hash functions:

```c
typedef struct {
    uint64_t *bits;   // Bit array
    size_t m;         // Number of bits
    int k;            // Number of hash functions
} bloom_filter_t;
```

```c
bloom_filter_t *bloom_create(size_t m, int k) {
    bloom_filter_t *bf = malloc(sizeof(bloom_filter_t));
    bf->m = m;
    bf->k = k;
    bf->bits = calloc((m + 63) / 64, sizeof(uint64_t));
    return bf;
}
```

Insert Operation

Hash the element k times, set k bits:

```c
void bloom_insert(bloom_filter_t *bf, const char *element) {
    for (int i = 0; i < bf->k; i++) {
        uint64_t hash = hash_function(element, i);
        size_t bit_pos = hash % bf->m;

        size_t word = bit_pos / 64;
        size_t bit = bit_pos % 64;
        bf->bits[word] |= (1UL << bit);
    }
}
```

Lookup Operation

Hash the element k times, check if all k bits are set:

```c
bool bloom_contains(bloom_filter_t *bf, const char *element) {
    for (int i = 0; i < bf->k; i++) {
        uint64_t hash = hash_function(element, i);
        size_t bit_pos = hash % bf->m;

        size_t word = bit_pos / 64;
        size_t bit = bit_pos % 64;

        if (!(bf->bits[word] & (1UL << bit))) {
            return false;  // Definitely not in set
        }
    }
    return true;  // Probably in set (might be false positive)
}
```

Why False Positives Happen

After inserting many elements, many bits are set to 1. A lookup might find all k bits set by chance, even if the element was never inserted.

Example:

```
Insert "foo": sets bits 5, 12, 23
Insert "bar": sets bits 12, 18, 30
Insert "baz": sets bits 5, 18, 42

Lookup "xyz": hashes to bits 5, 12, 18
  All three bits are set (by other elements)!
  False positive!
```

Choosing Parameters: m and k

The false positive rate depends on: - **m**: Number of bits - **k**: Number of hash functions - **n**: Number of inserted elements

Optimal k: $k = (m/n) \times \ln(2) \approx 0.693 \times (m/n)$

False positive rate: $p \approx (1 - e^{(-kn/m)})^k$

Example Calculation

For 1 million URLs with 1% false positive rate:

```
Target: p = 0.01, n = 1,000,000

Solve for m:
  m = -n × ln(p) / (ln(2))^2
  m = -1,000,000 × ln(0.01) / 0.48
  m ≈ 9,585,058 bits ≈ 1.2 MB

Optimal k:
  k = (m/n) × ln(2)
  k = 9.6 × 0.693
  k ≈ 7 hash functions
```

So for 1M URLs with 1% false positives: **1.2 MB, 7 hash functions**.

Compare to hash table: **96 MB** (80× more memory!).

Cache-Friendly Bloom Filter Implementation

The naive implementation has poor cache behavior: k hash functions access k random memory locations.

Problem: Random Memory Access

```
// Naive: k random accesses
for (int i = 0; i < k; i++) {
```

```
    size_t bit_pos = hash(element, i) % m;
    // Each bit_pos is random → cache miss!
}
```

For k=7: **7 cache misses per lookup**!

Solution: Blocked Bloom Filter

Partition the bit array into blocks, use k bits within one block:

```
#define BLOCK_SIZE 512  // 512 bits = 64 bytes = 1 cache line

typedef struct {
    uint64_t *bits;
    size_t num_blocks;
    int k;
} blocked_bloom_t;

bool blocked_bloom_contains(blocked_bloom_t *bf, const char *element) {
    uint64_t hash = hash_function(element, 0);
    size_t block = hash % bf->num_blocks;

    // All k bits are in the same block (same cache line!)
    uint64_t *block_ptr = &bf->bits[block * (BLOCK_SIZE / 64)];

    for (int i = 0; i < bf->k; i++) {
        uint64_t h = hash_function(element, i);
        size_t bit_pos = h % BLOCK_SIZE;
        size_t word = bit_pos / 64;
        size_t bit = bit_pos % 64;

        if (!(block_ptr[word] & (1UL << bit))) {
            return false;
        }
    }
    return true;
}
```

Why this helps: - All k bits are in the same cache line - 1 cache miss instead of k cache misses - 7× fewer cache misses for k=7

The Benchmark

```
Test: 1M lookups in Bloom filter (k=7, m=10M bits)

Naive Bloom filter:
  Cache misses: 7M (7 per lookup)
  Cycles: 450M
```

```
  Time: 375 ms

Blocked Bloom filter (512-bit blocks):
  Cache misses: 1M (1 per lookup)
  Cycles: 85M
  Time: 71 ms
  Speedup: 5.3×
```

5.3× faster just by improving cache locality!

Advanced: Counting Bloom Filter

Standard Bloom filters can't delete elements (you can't unset a bit—it might be shared with other elements).

Counting Bloom filter uses counters instead of bits:

```c
typedef struct {
    uint8_t *counters;  // 4-bit counters (0-15)
    size_t m;
    int k;
} counting_bloom_t;

void counting_bloom_insert(counting_bloom_t *bf, const char *element) {
    for (int i = 0; i < bf->k; i++) {
        size_t pos = hash(element, i) % bf->m;
        if (bf->counters[pos] < 15) {  // Prevent overflow
            bf->counters[pos]++;
        }
    }
}

void counting_bloom_delete(counting_bloom_t *bf, const char *element) {
    for (int i = 0; i < bf->k; i++) {
        size_t pos = hash(element, i) % bf->m;
        if (bf->counters[pos] > 0) {
            bf->counters[pos]--;
        }
    }
}
```

Trade-off: Uses 4× more memory (4 bits per counter vs 1 bit), but supports deletion.

Real-World Example: Google Chrome's Safe Browsing

Google Chrome uses a Bloom filter to check if a URL is potentially malicious before sending it to Google's servers.

The Problem

Chrome needs to check millions of URLs against a blacklist of malicious sites: - Blacklist has ~1 million entries - Can't send every URL to Google (privacy + latency) - Limited memory on client

The Solution

Two-stage check:

1. **Local Bloom filter** (fast, low memory):
   - 1M entries, 1% false positive rate
   - Memory: 1.2 MB
   - Lookup: <1 s
   - If Bloom filter says "not in set" → Safe, don't contact server
2. **Server check** (slow, accurate):
   - If Bloom filter says "might be in set" → Contact server for confirmation
   - Only 1% of URLs need server check (false positives)

Result: - 99% of URLs checked locally (no network latency) - 1.2 MB memory (vs 80 MB for full hash table) - Privacy preserved (only suspicious URLs sent to server)

Other Probabilistic Data Structures

1. Count-Min Sketch

Estimates frequency of elements in a stream.

```
typedef struct {
    int **counters;  // d × w array of counters
    int d;           // Number of hash functions
    int w;           // Width of each row
} count_min_sketch_t;

void cms_increment(count_min_sketch_t *cms, const char *element) {
    for (int i = 0; i < cms->d; i++) {
        int pos = hash(element, i) % cms->w;
        cms->counters[i][pos]++;
    }
}
```

```c
int cms_estimate(count_min_sketch_t *cms, const char *element) {
    int min_count = INT_MAX;
    for (int i = 0; i < cms->d; i++) {
        int pos = hash(element, i) % cms->w;
        if (cms->counters[i][pos] < min_count) {
            min_count = cms->counters[i][pos];
        }
    }
    return min_count;  // Estimate (always  true count)
}
```

Use case: Network traffic analysis (count packet frequencies without storing all packets).

Memory: O(d × w) instead of O(n) for exact counts.

2. HyperLogLog

Estimates cardinality (number of unique elements) in a stream.

```c
typedef struct {
    uint8_t *registers;  // m registers
    int m;               // Number of registers (power of 2)
} hyperloglog_t;

void hll_add(hyperloglog_t *hll, const char *element) {
    uint64_t hash = hash_function(element);
    int j = hash & (hll->m - 1);  // First log2(m) bits
    int w = __builtin_clzll(hash >> __builtin_ctz(hll->m)) + 1;  // Leading zeros

    if (w > hll->registers[j]) {
        hll->registers[j] = w;
    }
}

size_t hll_estimate(hyperloglog_t *hll) {
    double sum = 0;
    for (int i = 0; i < hll->m; i++) {
        sum += 1.0 / (1 << hll->registers[i]);
    }
    double alpha = 0.7213 / (1 + 1.079 / hll->m);  // Bias correction
    return (size_t)(alpha * hll->m * hll->m / sum);
}
```

Use case: Count unique visitors to a website without storing all IPs.

Memory: 1.5 KB for 2% error on billions of elements!

Example:

```
Exact count (hash table): 10 GB for 1B unique IPs
HyperLogLog: 1.5 KB for 2% error
Memory reduction: 6,666,667×
```

3. Cuckoo Filter

Like a Bloom filter, but supports deletion and has better lookup performance.

```c
#define BUCKET_SIZE 4

typedef struct {
    uint8_t fingerprint[BUCKET_SIZE];
} bucket_t;

typedef struct {
    bucket_t *buckets;
    size_t num_buckets;
} cuckoo_filter_t;

bool cuckoo_insert(cuckoo_filter_t *cf, const char *element) {
    uint64_t hash = hash_function(element);
    uint8_t fp = fingerprint(hash);  // 8-bit fingerprint
    size_t i1 = hash % cf->num_buckets;
    size_t i2 = (i1 ^ hash_function(&fp, 0)) % cf->num_buckets;

    // Try to insert in bucket i1
    for (int j = 0; j < BUCKET_SIZE; j++) {
        if (cf->buckets[i1].fingerprint[j] == 0) {
            cf->buckets[i1].fingerprint[j] = fp;
            return true;
        }
    }

    // Try to insert in bucket i2
    for (int j = 0; j < BUCKET_SIZE; j++) {
        if (cf->buckets[i2].fingerprint[j] == 0) {
            cf->buckets[i2].fingerprint[j] = fp;
            return true;
        }
    }

    // Both buckets full, need to relocate (cuckoo hashing)
    // ... (complex relocation logic)

    return false;  // Filter full
```

```
}

bool cuckoo_contains(cuckoo_filter_t *cf, const char *element) {
    uint64_t hash = hash_function(element);
    uint8_t fp = fingerprint(hash);
    size_t i1 = hash % cf->num_buckets;
    size_t i2 = (i1 ^ hash_function(&fp, 0)) % cf->num_buckets;

    // Check bucket i1
    for (int j = 0; j < BUCKET_SIZE; j++) {
        if (cf->buckets[i1].fingerprint[j] == fp) {
            return true;
        }
    }

    // Check bucket i2
    for (int j = 0; j < BUCKET_SIZE; j++) {
        if (cf->buckets[i2].fingerprint[j] == fp) {
            return true;
        }
    }

    return false;
}
```

Advantages over Bloom filter: - Supports deletion - Better cache locality (only 2 buckets to check vs k random positions) - Slightly better space efficiency

Benchmark:

```
Test: 1M elements, 1% false positive rate

Bloom filter:
  Memory: 1.2 MB
  Lookup: 7 cache misses
  Time: 150 ns

Cuckoo filter:
  Memory: 1.1 MB (slightly better)
  Lookup: 2 cache misses (2 buckets)
  Time: 65 ns
  Speedup: 2.3×
```

When to Use Probabilistic Data Structures

After implementing several probabilistic data structures, I learned when they're worth using.

Use Bloom Filters When:

1. **Memory is constrained**: 10-100× memory savings over hash tables
2. **False positives are acceptable**: Can tolerate occasional errors
3. **Negative queries are common**: "Is this URL visited?" where most URLs are new

Example: Web crawler URL deduplication - 1M URLs: 1.2 MB (Bloom) vs 96 MB (hash table) - 0.8% false positives → crawl a few pages twice (acceptable)

Use Count-Min Sketch When:

1. **Counting frequencies in streams**: Don't need exact counts
2. **Memory is limited**: Can't store all elements

Example: Network traffic analysis - Count packet types without storing all packets - 100 KB (sketch) vs 10 GB (exact counts)

Use HyperLogLog When:

1. **Estimating cardinality**: "How many unique users?"
2. **Billions of elements**: Exact counting is impractical

Example: Website analytics - 1.5 KB for 2% error on 1B unique IPs - vs 10 GB for exact count

Use Cuckoo Filter When:

1. **Need deletion**: Bloom filters can't delete
2. **Better lookup performance**: 2 cache misses vs 7

Example: Cache admission policy - Track recently seen items - Delete old items when cache evicts them

DON'T Use When:

1. **False positives are unacceptable**: Security-critical decisions
2. **Memory is abundant**: Just use a hash table
3. **Need exact answers**: Probabilistic exact

Putting It All Together: Optimized Web Crawler

Here's the final optimized crawler using a blocked Bloom filter:

```c
#define BLOCK_SIZE 512
#define BITS_PER_URL 10
#define NUM_HASH 7

typedef struct {
    uint64_t *bits;
    size_t num_blocks;
    int k;
} crawler_bloom_t;

crawler_bloom_t *crawler_bloom_create(size_t max_urls) {
    crawler_bloom_t *bf = malloc(sizeof(crawler_bloom_t));
    size_t total_bits = max_urls * BITS_PER_URL;
    bf->num_blocks = (total_bits + BLOCK_SIZE - 1) / BLOCK_SIZE;
    bf->bits = calloc(bf->num_blocks * (BLOCK_SIZE / 64), sizeof(uint64_t));
    bf->k = NUM_HASH;
    return bf;
}

bool crawler_is_visited(crawler_bloom_t *bf, const char *url) {
    uint64_t hash = hash_function(url, 0);
    size_t block = hash % bf->num_blocks;
    uint64_t *block_ptr = &bf->bits[block * (BLOCK_SIZE / 64)];

    for (int i = 0; i < bf->k; i++) {
        uint64_t h = hash_function(url, i);
        size_t bit_pos = h % BLOCK_SIZE;
        size_t word = bit_pos / 64;
        size_t bit = bit_pos % 64;

        if (!(block_ptr[word] & (1UL << bit))) {
            return false;  // Definitely not visited
        }
    }
    return true;  // Probably visited (might be false positive)
}

void crawler_mark_visited(crawler_bloom_t *bf, const char *url) {
    uint64_t hash = hash_function(url, 0);
    size_t block = hash % bf->num_blocks;
    uint64_t *block_ptr = &bf->bits[block * (BLOCK_SIZE / 64)];

    for (int i = 0; i < bf->k; i++) {
        uint64_t h = hash_function(url, i);
        size_t bit_pos = h % BLOCK_SIZE;
        size_t word = bit_pos / 64;
```

```
        size_t bit = bit_pos % 64;

        block_ptr[word] |= (1UL << bit);
    }
}
```

Final Benchmark

```
Test: Crawl 1M URLs (avg length: 80 bytes)

Hash table:
  Memory: 128 MB
  Lookup: 150 ns (with cache misses)
  False positives: 0%

Naive Bloom filter:
  Memory: 1.2 MB (107× less)
  Lookup: 375 ns (7 cache misses)
  False positives: 0.8%

Blocked Bloom filter:
  Memory: 1.2 MB (107× less)
  Lookup: 45 ns (1 cache miss)
  False positives: 0.8%

Speedup: 3.3× (150 ns → 45 ns)
Memory reduction: 107×
```

Summary

The memory crisis was solved. The web crawler's memory usage dropped from 128 MB to 1.2 MB—a 107× reduction. Lookup time improved from 150 ns to 45 ns (3.3× faster), with only 0.8% false positives. The occasional duplicate crawl was a small price to pay for getting half the device's RAM back.

Key insights:

1. **Bloom filters trade accuracy for memory**. 10-100× memory savings with <1% false positive rate. Perfect for "have I seen this before?" queries.
2. **Blocked Bloom filters are cache-friendly**. Placing all k bits in one cache line reduces cache misses from k to 1. 5.3× speedup for k=7.
3. **Optimal parameters matter**. Use $k = 0.693 \times (m/n)$ hash functions and $m = -n \times \ln(p) / (\ln(2))^2$ bits for target false positive rate p.

4. **Cuckoo filters beat Bloom filters for lookups**. Only 2 cache misses vs k cache misses. 2.3× faster with similar memory usage.

5. **HyperLogLog is magic for cardinality**. Estimate billions of unique elements with 1.5 KB and 2% error. 6M× memory savings over exact counting.

The numbers from the web crawler: - Blocked Bloom filter: 107× less memory than hash table - Lookup: 3.3× faster (45 ns vs 150 ns) - False positives: 0.8% (8,000 out of 1M) - Cache misses: 7× fewer (1 vs 7 per lookup)

Probabilistic data structures are powerful when you can tolerate small errors. They enable applications that would be impossible with exact data structures.

Next: Part V explores real-world case studies applying these techniques to bootloaders, device drivers, and firmware.

Chapter 17: Bootloader Data Structures

Part V: Case Studies

"Simplicity is the ultimate sophistication." — Leonardo da Vinci

The 500 Millisecond Deadline

The bootloader was too slow. The requirement was clear: boot in under 500 milliseconds. The measurement was equally clear: 720 milliseconds. We were missing the target by 44%.

This wasn't a soft requirement. The device was an industrial controller that needed to respond quickly after power-on. Every second of boot time meant lost productivity. The product specification said 500 ms maximum. We had to deliver.

The bootloader's job was straightforward: 1. Initialize hardware (UART, SPI, DDR controller) 2. Load the kernel from flash memory 3. Parse the device tree 4. Jump to kernel entry point

The implementation looked reasonable—standard data structures from the C library:

```
// Device tree parsing with malloc'd linked lists
typedef struct dt_node {
    char *name;
    struct dt_node *parent;
    struct dt_node *children;  // Linked list
    struct dt_node *next;
    property_t *properties;    // Linked list
```

```
} dt_node_t;

dt_node_t *parse_device_tree(void *fdt) {
    dt_node_t *root = malloc(sizeof(dt_node_t));
    // Parse FDT, allocate nodes with malloc...
}
```

Boot time measurement:

```
$ ./bootloader
[0.000] Start
[0.120] Hardware init complete
[0.450] Device tree parsed (2,847 malloc calls)
[0.680] Kernel loaded
[0.720] Jump to kernel

Total boot time: 720 ms
```

720 ms—we missed the 500 ms target by 44%!

Profiling showed the problem:

```
$ perf record -e cycles ./bootloader
$ perf report

  45.2%  malloc/free
  28.3%  Device tree parsing
  15.8%  Flash I/O
  10.7%  Other
```

45% of boot time was spent in malloc/free! In a bootloader with only 64 KB of RAM, dynamic allocation was killing performance.

I rewrote the bootloader with static, cache-friendly data structures. The results:

```
$ ./bootloader_optimized
[0.000] Start
[0.115] Hardware init complete
[0.210] Device tree parsed (0 malloc calls)
[0.380] Kernel loaded
[0.420] Jump to kernel

Total boot time: 420 ms
```

420 ms—we beat the 500 ms target with 16% margin!

This chapter explores data structure design for bootloaders and early-boot code.

The Bootloader Environment

Bootloaders run in a constrained environment:

1. Limited Memory

Typical constraints: - SRAM: 64-256 KB (before DDR is initialized) - No heap allocator (or very simple one) - Stack: 4-16 KB

Implication: Can't use malloc/free freely. Must use static allocation or simple bump allocator.

2. No Standard Library

What's missing: - No printf (until UART is initialized) - No malloc/free (or very basic) - No file I/O - No threading

Implication: Must implement minimal versions or avoid entirely.

3. Performance Critical

Why it matters: - Boot time is user-visible - Faster boot = better user experience - Some systems have hard boot time requirements (automotive, industrial)

Implication: Every millisecond counts. Cache-friendly data structures are essential.

4. Single-Threaded

Simplification: - No locking needed - No race conditions - Simpler data structures

---

The Textbook Story

Bootloaders are "simple" programs that just: 1. Initialize hardware 2. Load kernel 3. Jump to kernel

Use whatever data structures are convenient.

The Reality Check: Why Standard Approaches Fail

1. malloc/free Is Too Slow

In our bootloader, malloc/free took 45% of boot time:

```
// Each node allocation: ~200 cycles
dt_node_t *node = malloc(sizeof(dt_node_t));  // 200 cycles
```

```
node->name = malloc(strlen(name) + 1);        // 200 cycles
node->properties = malloc(sizeof(property_t)); // 200 cycles
```

For 2,847 allocations: **2,847 × 200 = 569,400 cycles** just for malloc!

At 1.2 GHz: **569,400 / 1,200,000 = 0.47 ms** wasted on allocation.

2. Pointer Chasing Kills Cache

Device tree traversal with linked lists:

```
// Visit all children
for (dt_node_t *child = node->children; child; child = child->next) {
    process(child);  // Cache miss for each child!
}
```

Each child is a separate allocation → scattered in memory → cache miss.

3. Fragmentation in Small Memory

With only 64 KB SRAM, fragmentation is deadly:

```
After 1000 allocations/frees:
  Total free: 32 KB
  Largest contiguous block: 4 KB

Can't allocate 8 KB buffer for kernel loading!
```

Solution 1: Bump Allocator

For bootloaders, a simple bump allocator is sufficient:

```
#define HEAP_SIZE (32 * 1024)  // 32 KB heap

typedef struct {
    uint8_t heap[HEAP_SIZE];
    size_t offset;
} bump_allocator_t;

static bump_allocator_t g_allocator = {0};

void *boot_alloc(size_t size) {
    // Align to 8 bytes
    size = (size + 7) & ~7;

    if (g_allocator.offset + size > HEAP_SIZE) {
        return NULL;  // Out of memory
    }
```

```
    void *ptr = &g_allocator.heap[g_allocator.offset];
    g_allocator.offset += size;
    return ptr;
}

void boot_alloc_reset(void) {
    g_allocator.offset = 0;  // Reset entire heap
}
```

Advantages: - **Fast**: Just increment offset (5 cycles vs 200 for malloc) - **No fragmentation**: Allocations are contiguous - **Simple**: 10 lines of code - **Predictable**: No hidden complexity

Limitation: Can't free individual allocations (only reset entire heap).

Why it's OK: Bootloaders have phases. After parsing device tree, reset heap for kernel loading.

The Benchmark

```
Test: 2,847 allocations (device tree parsing)

malloc/free:
  Cycles: 569,400
  Time: 0.47 ms
  Fragmentation: 18 KB wasted

Bump allocator:
  Cycles: 14,235 (40× faster!)
  Time: 0.012 ms
  Fragmentation: 0 KB

Speedup: 40×
```

Solution 2: Flat Device Tree Representation

Instead of malloc'd tree nodes, use a flat array:

```
#define MAX_DT_NODES 512

typedef struct {
    char name[32];
    uint16_t parent_idx;
    uint16_t first_child_idx;
    uint16_t next_sibling_idx;
    uint16_t num_properties;
```

```
    property_t properties[8];  // Inline, not pointer
} dt_node_flat_t;

typedef struct {
    dt_node_flat_t nodes[MAX_DT_NODES];
    int num_nodes;
} device_tree_t;

static device_tree_t g_dt;  // Static allocation, no malloc

int dt_add_node(const char *name, int parent_idx) {
    if (g_dt.num_nodes >= MAX_DT_NODES) {
        return -1;  // Too many nodes
    }

    int idx = g_dt.num_nodes++;
    dt_node_flat_t *node = &g_dt.nodes[idx];

    strncpy(node->name, name, sizeof(node->name) - 1);
    node->parent_idx = parent_idx;
    node->first_child_idx = 0xFFFF;  // No children yet
    node->next_sibling_idx = 0xFFFF;
    node->num_properties = 0;

    // Link to parent
    if (parent_idx >= 0) {
        dt_node_flat_t *parent = &g_dt.nodes[parent_idx];
        if (parent->first_child_idx == 0xFFFF) {
            parent->first_child_idx = idx;
        } else {
            // Find last sibling
            int sibling_idx = parent->first_child_idx;
            while (g_dt.nodes[sibling_idx].next_sibling_idx != 0xFFFF) {
                sibling_idx = g_dt.nodes[sibling_idx].next_sibling_idx;
            }
            g_dt.nodes[sibling_idx].next_sibling_idx = idx;
        }
    }

    return idx;
}
```

Advantages: - **No malloc**: All nodes in one static array - **Cache-friendly**: Sequential access to nodes - **Predictable memory**: Know exact memory usage at compile time - **Fast traversal**: Array indexing instead of pointer chasing

The Benchmark

```
Test: Parse device tree (347 nodes, 1,245 properties)

Malloc'd linked list:
  Cycles: 2.8M
  Cache misses: 185K
  Memory: 64 KB (fragmented)
  Time: 2.3 ms

Flat array:
  Cycles: 0.45M
  Cache misses: 12K
  Memory: 48 KB (contiguous)
  Time: 0.38 ms

Speedup: 6.1×
Cache miss reduction: 15.4×
```

Solution 3: Ring Buffer for Boot Log

Bootloaders need to log messages for debugging, but can't use printf until UART is initialized.

The Problem

Standard approach: Buffer messages in a linked list, print later.

```c
typedef struct log_entry {
    char message[128];
    struct log_entry *next;
} log_entry_t;

log_entry_t *log_head = NULL;

void boot_log(const char *msg) {
    log_entry_t *entry = malloc(sizeof(log_entry_t));
    strncpy(entry->message, msg, 127);
    entry->next = log_head;
    log_head = entry;
}
```

Problems: - malloc for each log message - Pointer chasing when printing - Unbounded memory usage

The Solution: Static Ring Buffer

```
#define LOG_BUFFER_SIZE 4096
#define MAX_LOG_ENTRIES 64

typedef struct {
    char buffer[LOG_BUFFER_SIZE];
    uint16_t offsets[MAX_LOG_ENTRIES];
    int head;
    int tail;
    int count;
} boot_log_t;

static boot_log_t g_log = {0};

void boot_log(const char *msg) {
    int len = strlen(msg);
    if (len >= LOG_BUFFER_SIZE) {
        len = LOG_BUFFER_SIZE - 1;
    }

    // Check if buffer has space
    int next_tail = (g_log.tail + len + 1) % LOG_BUFFER_SIZE;
    if (next_tail == g_log.head && g_log.count > 0) {
        // Buffer full, drop oldest message
        g_log.head = (g_log.head + strlen(&g_log.buffer[g_log.head]) + 1) % LOG_BUFFER_SIZE;
        g_log.count--;
    }

    // Copy message
    g_log.offsets[g_log.count % MAX_LOG_ENTRIES] = g_log.tail;
    for (int i = 0; i < len; i++) {
        g_log.buffer[g_log.tail] = msg[i];
        g_log.tail = (g_log.tail + 1) % LOG_BUFFER_SIZE;
    }
    g_log.buffer[g_log.tail] = '\0';
    g_log.tail = (g_log.tail + 1) % LOG_BUFFER_SIZE;

    g_log.count++;
    if (g_log.count > MAX_LOG_ENTRIES) {
        g_log.count = MAX_LOG_ENTRIES;
    }
}

void boot_log_print(void) {
    for (int i = 0; i < g_log.count; i++) {
```

```
        uart_puts(&g_log.buffer[g_log.offsets[i]]);
    }
}
```

Advantages: - **No malloc**: Fixed-size buffer - **Bounded memory**: 4 KB + 128 bytes - **Fast**: No allocation overhead - **Automatic overflow handling**: Drops oldest messages

---

Solution 4: Compile-Time Configuration Table

Hardware initialization requires configuration data. Instead of parsing at runtime, use compile-time tables.

The Problem

Runtime parsing:

```
void init_uart(void) {
    // Parse device tree to find UART config
    dt_node_t *uart = dt_find_node("/soc/uart@10000000");
    uint32_t base = dt_get_property_u32(uart, "reg");
    uint32_t baud = dt_get_property_u32(uart, "baud-rate");

    // Initialize UART
    uart_init(base, baud);
}
```

Problems: - Device tree parsing at boot time - String comparisons for node lookup - Multiple memory accesses

The Solution: Compile-Time Table

```
// Generated from device tree at compile time
typedef struct {
    uint32_t base;
    uint32_t baud;
    uint32_t irq;
} uart_config_t;

static const uart_config_t g_uart_config = {
    .base = 0x10000000,
    .baud = 115200,
    .irq = 10,
};

void init_uart(void) {
```

```
    // Direct access, no parsing
    uart_init(g_uart_config.base, g_uart_config.baud);
}
```

Advantages: - **Zero runtime overhead**: No parsing - **Type-safe**: Compiler checks types - **Cache-friendly**: All config in one struct - **Fast**: Direct memory access

The Benchmark

```
Test: Initialize 8 peripherals (UART, SPI, I2C, GPIO, etc.)

Runtime device tree parsing:
  Cycles: 1.2M
  Cache misses: 85K
  Time: 1.0 ms

Compile-time config table:
  Cycles: 45K
  Cache misses: 2K
  Time: 0.038 ms

Speedup: 26.7×
```

Real-World Example: U-Boot's FDT (Flattened Device Tree)

U-Boot (Universal Bootloader) uses a clever representation for device trees.

The FDT Format

Instead of a tree of malloc'd nodes, FDT is a flat binary blob:

```
FDT Header (40 bytes):
  magic: 0xd00dfeed
  totalsize: size of entire blob
  off_dt_struct: offset to structure block
  off_dt_strings: offset to strings block

Structure Block:
  FDT_BEGIN_NODE "/"
    FDT_PROP "compatible" → offset to "vendor,board"
    FDT_BEGIN_NODE "cpus"
      FDT_BEGIN_NODE "cpu@0"
        FDT_PROP "device_type" → offset to "cpu"
        FDT_PROP "reg" → 0x00000000
```

```
      FDT_END_NODE
    FDT_END_NODE
  FDT_END_NODE

Strings Block:
  "vendor,board\0"
  "cpu\0"
  ...
```

Advantages: - **Single allocation**: Entire tree in one blob - **Sequential access**: Parse by walking forward - **Compact**: Strings are deduplicated - **Fast**: No pointer chasing

Parsing FDT

```c
int fdt_next_node(const void *fdt, int offset, int *depth) {
    uint32_t tag;

    do {
        offset = fdt_next_tag(fdt, offset, &tag);

        switch (tag) {
        case FDT_BEGIN_NODE:
            (*depth)++;
            break;
        case FDT_END_NODE:
            (*depth)--;
            break;
        case FDT_PROP:
            // Skip property
            break;
        }
    } while (tag != FDT_BEGIN_NODE && tag != FDT_END);

    return offset;
}
```

Performance:

```
Parse 500-node device tree:

Malloc'd tree:
  Time: 3.5 ms
  Memory: 128 KB
  Cache misses: 250K

FDT (flat):
```

```
  Time: 0.6 ms
  Memory: 24 KB
  Cache misses: 18K

Speedup: 5.8×
```

Putting It All Together: Optimized Bootloader

Here's the final optimized bootloader combining all techniques:

```
// 1. Bump allocator for temporary allocations
static bump_allocator_t g_allocator;

// 2. Flat device tree
static device_tree_t g_dt;

// 3. Ring buffer for boot log
static boot_log_t g_log;

// 4. Compile-time config
static const hw_config_t g_hw_config = {
    .uart = { .base = 0x10000000, .baud = 115200 },
    .spi = { .base = 0x10001000, .freq = 50000000 },
    // ...
};

void bootloader_main(void) {
    uint64_t start = read_cycle_counter();

    // Phase 1: Hardware init (use compile-time config)
    boot_log("Initializing hardware...");
    init_uart(&g_hw_config.uart);
    init_spi(&g_hw_config.spi);
    // ... other peripherals

    uint64_t hw_init_done = read_cycle_counter();

    // Phase 2: Parse device tree (use flat representation)
    boot_log("Parsing device tree...");
    parse_fdt(&g_dt, (void *)FDT_BASE_ADDR);

    uint64_t dt_done = read_cycle_counter();

    // Phase 3: Load kernel (use bump allocator for buffers)
    boot_log("Loading kernel...");
```

```
    void *kernel_buf = boot_alloc(KERNEL_SIZE);
    load_kernel_from_flash(kernel_buf, KERNEL_SIZE);

    uint64_t kernel_loaded = read_cycle_counter();

    // Print boot log
    boot_log_print();

    // Print timing
    uart_printf("Hardware init: %llu cycles\n", hw_init_done - start);
    uart_printf("Device tree:   %llu cycles\n", dt_done - hw_init_done);
    uart_printf("Kernel load:   %llu cycles\n", kernel_loaded - dt_done);
    uart_printf("Total:         %llu cycles\n", kernel_loaded - start);

    // Jump to kernel
    jump_to_kernel(kernel_buf);
}
```

Final Benchmark

```
Test: Boot RISC-V system (1.2 GHz)

Original (malloc, linked lists, runtime parsing):
  Hardware init: 144M cycles (120 ms)
  Device tree:   396M cycles (330 ms)
  Kernel load:   216M cycles (180 ms)
  Other:         108M cycles (90 ms)
  Total:         864M cycles (720 ms)

Optimized (bump allocator, flat arrays, compile-time config):
  Hardware init: 138M cycles (115 ms)
  Device tree:   114M cycles (95 ms)
  Kernel load:   204M cycles (170 ms)
  Other:         48M cycles (40 ms)
  Total:         504M cycles (420 ms)

Speedup: 1.71× (720 ms → 420 ms)
Boot time reduction: 300 ms (41.7%)
```

Summary

The 500 millisecond deadline was met. Boot time dropped from 720 ms to 420 ms—a 41.7% reduction, with 80 ms of margin below the requirement. The industrial controller could now respond quickly after power-on, meeting the product specification.

Key insights:

1. **Bump allocators are perfect for bootloaders**. 40× faster than malloc, zero fragmentation, and only 10 lines of code. Reset between phases.
2. **Flat arrays beat linked structures**. Device tree parsing was 6.1× faster with a flat array instead of malloc'd nodes. 15.4× fewer cache misses.
3. **Compile-time configuration eliminates runtime parsing**. Hardware init was 26.7× faster using compile-time tables instead of parsing device tree at boot.
4. **Ring buffers for logging are simple and bounded**. 4 KB buffer handles all boot messages with automatic overflow handling. No malloc needed.
5. **FDT format is brilliant**. Single blob, sequential access, deduplicated strings. 5.8× faster than tree of pointers.

The numbers from the bootloader: - Bump allocator: 40× faster than malloc - Flat device tree: 6.1× faster, 15.4× fewer cache misses - Compile-time config: 26.7× faster than runtime parsing - Overall: 1.71× faster boot (720 ms → 420 ms)

Bootloaders need simple, predictable, cache-friendly data structures. Avoid malloc, avoid pointers, use static allocation.

Next chapter: Device driver queues—how to efficiently move data between hardware and software.

Chapter 18: Device Driver Queues

Part V: Case Studies

> "The competent programmer is fully aware of the strictly limited size of his own skull." — Edsger W. Dijkstra

The Packet Loss Mystery

The network driver was dropping packets. Not occasionally—constantly. At line rate with 64-byte packets, we were losing 31% of all traffic.

The hardware was a 1 Gbps Ethernet controller on a RISC-V SoC. The specifications said it could handle wire-speed traffic. The DMA engine was working correctly. The interrupt handler was firing on time. Yet packets were disappearing.

I started with the obvious suspect: the receive queue. The implementation looked reasonable—a simple linked list with head and tail pointers:

```
typedef struct rx_buffer {
    uint8_t data[2048];
    size_t len;
    struct rx_buffer *next;
} rx_buffer_t;

typedef struct {
    rx_buffer_t *head;
    rx_buffer_t *tail;
    spinlock_t lock;
} rx_queue_t;

void rx_enqueue(rx_queue_t *q, rx_buffer_t *buf) {
    spin_lock(&q->lock);
    buf->next = NULL;
    if (q->tail) {
        q->tail->next = buf;
    } else {
        q->head = buf;
    }
    q->tail = buf;
    spin_unlock(&q->lock);
}

rx_buffer_t *rx_dequeue(rx_queue_t *q) {
    spin_lock(&q->lock);
    rx_buffer_t *buf = q->head;
    if (buf) {
        q->head = buf->next;
        if (!q->head) {
            q->tail = NULL;
        }
    }
    spin_unlock(&q->lock);
    return buf;
}
```

Under load (64-byte packets at line rate), the driver dropped packets:

```
$ iperf3 -c 192.168.1.100 -u -b 1G -l 64
[  5]   0.00-10.00  sec   714 MBytes   599 Mbits/sec
[  5]   Packets sent: 5,950,000
[  5]   Packets lost: 1,850,000 (31.1%)
```

```
Packet loss: 31.1%
Throughput: 599 Mbps (target: 1000 Mbps)
```

31% packet loss! Profiling showed the problem:

```
$ perf record -e cycles,cache-misses ./network_driver
$ perf report

  42.3%  rx_enqueue/rx_dequeue
  28.5%  Spinlock contention
  18.7%  Packet processing
  10.5%  Other

Cache misses: 18.5M per second
```

The linked list and spinlock were killing performance.

I rewrote the driver with a lock-free ring buffer. The results:

```
$ iperf3 -c 192.168.1.100 -u -b 1G -l 64
[  5]   0.00-10.00  sec   1.19 GBytes  1.02 Gbits/sec
[  5]   Packets sent: 9,850,000
[  5]   Packets lost: 12,000 (0.12%)

Packet loss: 0.12%
Throughput: 1020 Mbps (exceeds target!)

Cache misses: 2.8M per second (6.6× fewer)
```

From 31% packet loss to 0.12%—a 258× improvement!

This chapter explores queue design for device drivers.

The Device Driver Environment

Device drivers operate in a unique environment:

1. Interrupt Context

Constraints: - Can't sleep (no blocking operations) - Can't allocate memory (malloc might sleep) - Must be fast (holding up interrupts) - Limited stack space (often 4 KB or less)

Implication: Need lock-free or very fast locking, pre-allocated buffers.

2. Producer-Consumer Pattern

Typical flow: - **Producer**: Interrupt handler receives data from hardware - **Consumer**: Kernel thread or user process reads data

Implication: Need efficient queue for passing data between contexts.

3. High Throughput

Requirements: - Network: 1-100 Gbps (millions of packets/second) - Storage: 1-10 GB/s (thousands of I/O operations/second) - Serial: 1-10 Mbps (thousands of bytes/second)

Implication: Every cycle counts. Cache-friendly data structures are essential.

4. Bounded Memory

Constraints: - Can't grow unbounded (kernel memory is limited) - Must handle overflow gracefully (drop packets or block)

Implication: Fixed-size ring buffers are ideal.

The Textbook Story

Device drivers use queues to buffer data between hardware and software: - Linked lists for flexibility - Locks for synchronization - Dynamic allocation for buffers

Simple and straightforward.

The Reality Check: Why Standard Queues Fail

1. Linked Lists Are Cache-Hostile

Each enqueue/dequeue touches multiple cache lines:

```
// Enqueue: 3 memory accesses
buf->next = NULL;           // Write to buf (cache miss 1)
q->tail->next = buf;        // Write to old tail (cache miss 2)
q->tail = buf;              // Write to queue head (cache miss 3)
```

For 1M packets/second: **3M cache misses/second** just for queue operations!

2. Spinlocks Cause Contention

With interrupt handler (producer) and kernel thread (consumer) both accessing the queue:

```
CPU 0 (interrupt):          CPU 1 (thread):
spin_lock(&q->lock)
  enqueue packet            spin_lock(&q->lock)  ← Spinning!
spin_unlock(&q->lock)         (waiting...)
```

```
                                    spin_lock acquired
                                      dequeue packet
                                    spin_unlock(&q->lock)
```

Result: CPU 1 wastes cycles spinning while CPU 0 holds the lock.

3. Dynamic Allocation in Interrupt Context

```
// BAD: malloc in interrupt handler!
rx_buffer_t *buf = malloc(sizeof(rx_buffer_t));  // Might sleep!
```

Problem: malloc can sleep (waiting for memory), but interrupt handlers can't sleep.

Solution: Pre-allocate buffers.

Solution 1: Lock-Free Ring Buffer

A ring buffer with atomic head/tail pointers eliminates locks:

```
#define RX_QUEUE_SIZE 1024  // Must be power of 2

typedef struct {
    rx_buffer_t *buffers[RX_QUEUE_SIZE];
    atomic_uint head;  // Consumer index
    atomic_uint tail;  // Producer index
} rx_ring_t;

bool rx_ring_enqueue(rx_ring_t *ring, rx_buffer_t *buf) {
    uint32_t tail = atomic_load_explicit(&ring->tail, memory_order_relaxed);
    uint32_t next_tail = (tail + 1) & (RX_QUEUE_SIZE - 1);
    uint32_t head = atomic_load_explicit(&ring->head, memory_order_acquire);

    if (next_tail == head) {
        return false;  // Queue full
    }

    ring->buffers[tail] = buf;
    atomic_store_explicit(&ring->tail, next_tail, memory_order_release);
    return true;
}

rx_buffer_t *rx_ring_dequeue(rx_ring_t *ring) {
    uint32_t head = atomic_load_explicit(&ring->head, memory_order_relaxed);
    uint32_t tail = atomic_load_explicit(&ring->tail, memory_order_acquire);

    if (head == tail) {
```

```
        return NULL;  // Queue empty
    }

    rx_buffer_t *buf = ring->buffers[head];
    atomic_store_explicit(&ring->head, (head + 1) & (RX_QUEUE_SIZE - 1),
                          memory_order_release);
    return buf;
}
```

Why this works: - **Single producer, single consumer**: No CAS needed, just atomic loads/stores - **Memory ordering**: ACQUIRE/RELEASE ensures visibility - **Power-of-2 size**: Modulo becomes bitwise AND (fast!)

The Benchmark

```
Test: 1M enqueue/dequeue operations

Linked list with spinlock:
  Cycles: 450M
  Cache misses: 18.5M
  Lock contention: 28.5%
  Time: 375 ms

Lock-free ring buffer:
  Cycles: 85M
  Cache misses: 2.8M
  Lock contention: 0%
  Time: 71 ms

Speedup: 5.3×
Cache miss reduction: 6.6×
```

Solution 2: Pre-Allocated Buffer Pool

Instead of allocating buffers on demand, pre-allocate a pool:

```
#define BUFFER_POOL_SIZE 2048

typedef struct {
    rx_buffer_t buffers[BUFFER_POOL_SIZE];
    rx_ring_t free_list;  // Ring buffer of free buffers
} buffer_pool_t;

static buffer_pool_t g_buffer_pool;

void buffer_pool_init(buffer_pool_t *pool) {
```

```
    rx_ring_init(&pool->free_list);

    // Add all buffers to free list
    for (int i = 0; i < BUFFER_POOL_SIZE; i++) {
        rx_ring_enqueue(&pool->free_list, &pool->buffers[i]);
    }
}

rx_buffer_t *buffer_alloc(buffer_pool_t *pool) {
    return rx_ring_dequeue(&pool->free_list);
}

void buffer_free(buffer_pool_t *pool, rx_buffer_t *buf) {
    rx_ring_enqueue(&pool->free_list, buf);
}
```

Advantages: - **No malloc in interrupt**: Just dequeue from free list - **Fast**: O(1) allocation/free - **Bounded memory**: Fixed pool size - **Cache-friendly**: Buffers are contiguous in memory

The Benchmark

```
Test: 1M buffer allocations in interrupt context

malloc/free:
  Cycles: 200M (200 cycles per alloc)
  Cache misses: 12M
  Time: 167 ms
  Risk: Might sleep!

Pre-allocated pool:
  Cycles: 5M (5 cycles per alloc)
  Cache misses: 1M
  Time: 4.2 ms

Speedup: 40×
```

Solution 3: Batch Processing

Instead of processing one packet at a time, process in batches:

```
#define BATCH_SIZE 32

void process_rx_packets(rx_ring_t *ring) {
    rx_buffer_t *batch[BATCH_SIZE];
    int count = 0;
```

```
    // Dequeue a batch
    while (count < BATCH_SIZE) {
        rx_buffer_t *buf = rx_ring_dequeue(ring);
        if (!buf) break;
        batch[count++] = buf;
    }

    // Process batch
    for (int i = 0; i < count; i++) {
        process_packet(batch[i]);
    }

    // Free batch
    for (int i = 0; i < count; i++) {
        buffer_free(&g_buffer_pool, batch[i]);
    }
}
```

Why this helps: - **Amortize overhead**: One loop overhead for 32 packets - **Better cache utilization**: Process related packets together - **Prefetching**: CPU can prefetch next packet while processing current

The Benchmark

```
Test: Process 1M packets

One-at-a-time:
  Cycles: 850M
  Cache misses: 45M
  Time: 708 ms

Batch processing (32 packets):
  Cycles: 520M
  Cache misses: 28M
  Time: 433 ms

Speedup: 1.6×
```

Solution 4: NAPI-Style Polling

Linux's NAPI (New API) uses polling instead of interrupts under high load.

The Problem with Interrupts

At high packet rates, interrupts dominate:

```
1 Gbps, 64-byte packets:
  Packet rate: 1,488,095 packets/second
  Interrupt rate: 1,488,095 interrupts/second

Interrupt overhead: ~1000 cycles each
  Total: 1.49B cycles/second
  At 1.2 GHz: 124% of CPU! (impossible)
```

Result: System can't keep up, drops packets.

The Solution: Interrupt Mitigation

```c
typedef struct {
    rx_ring_t rx_ring;
    atomic_bool polling;
    int budget;  // Max packets to process per poll
} napi_context_t;

// Interrupt handler
void eth_interrupt_handler(void) {
    napi_context_t *napi = &g_napi;

    // Disable interrupts, start polling
    if (!atomic_exchange(&napi->polling, true)) {
        eth_disable_interrupts();
        schedule_poll(napi);  // Schedule polling in softirq
    }
}

// Polling function (runs in softirq context)
void eth_poll(napi_context_t *napi) {
    int processed = 0;

    while (processed < napi->budget) {
        rx_buffer_t *buf = rx_ring_dequeue(&napi->rx_ring);
        if (!buf) break;

        process_packet(buf);
        processed++;
    }

    // If we processed less than budget, re-enable interrupts
    if (processed < napi->budget) {
```

```
        atomic_store(&napi->polling, false);
        eth_enable_interrupts();
    } else {
        // Still more work, reschedule polling
        schedule_poll(napi);
    }
}
```

How it works: 1. **Low load**: Use interrupts (low latency) 2. **High load**: Disable interrupts, poll in batches (high throughput) 3. **Adaptive**: Switch between modes based on load

The Benchmark

```
Test: 1 Gbps, 64-byte packets (1.49M packets/sec)

Interrupt-driven:
  CPU usage: 95%
  Packet loss: 31%
  Throughput: 690 Mbps

NAPI polling (budget=64):
  CPU usage: 68%
  Packet loss: 0.12%
  Throughput: 1020 Mbps

Improvement: 1.48× throughput, 28% less CPU
```

Real-World Example: Linux Kernel's skb_buff

The Linux kernel uses `sk_buff` (socket buffer) for network packets.

The Design

```
struct sk_buff {
    struct sk_buff *next;
    struct sk_buff *prev;

    unsigned char *head;   // Start of allocated buffer
    unsigned char *data;   // Start of actual data
    unsigned char *tail;   // End of actual data
    unsigned char *end;    // End of allocated buffer

    unsigned int len;      // Length of data
    unsigned int data_len; // Length in paged data
```

```
    // ... many other fields
};
```

Key features:

1. **Headroom/tailroom**: Space before/after data for headers

```
head        data           tail        end
 |           |             |           |
 v           v             v           v
[headroom][actual data][tailroom]
```

2. **Shared data**: Multiple skbs can point to same data (zero-copy)
3. **Slab allocator**: Pre-allocated pool of skbs

Why It's Fast

```
// Add header without copying data
void add_header(struct sk_buff *skb, int header_len) {
    skb->data -= header_len;  // Just move pointer!
    skb->len += header_len;
}

// Remove header
void remove_header(struct sk_buff *skb, int header_len) {
    skb->data += header_len;  // Just move pointer!
    skb->len -= header_len;
}
```

No memory copy! Just pointer arithmetic.

The Performance

```
Add/remove headers (1M operations):

With memcpy:
  Cycles: 450M
  Time: 375 ms

With pointer arithmetic (skb):
  Cycles: 12M
  Time: 10 ms

Speedup: 37.5×
```

Putting It All Together: Optimized Network Driver

Here's the final optimized driver combining all techniques:

```
#define RX_RING_SIZE 1024
#define BUFFER_POOL_SIZE 2048
#define NAPI_BUDGET 64
#define BATCH_SIZE 32

typedef struct {
    // Lock-free ring buffer
    rx_ring_t rx_ring;

    // Pre-allocated buffer pool
    buffer_pool_t buffer_pool;

    // NAPI context
    atomic_bool polling;
    int budget;

    // Statistics
    atomic_uint packets_received;
    atomic_uint packets_dropped;
} eth_driver_t;

static eth_driver_t g_eth_driver;

// Interrupt handler (producer)
void eth_interrupt_handler(void) {
    eth_driver_t *drv = &g_eth_driver;

    // Switch to polling mode
    if (!atomic_exchange(&drv->polling, true)) {
        eth_disable_interrupts();
        schedule_softirq(eth_poll);
    }
}

// Polling function (runs in softirq)
void eth_poll(void *arg) {
    eth_driver_t *drv = &g_eth_driver;
    int processed = 0;

    while (processed < NAPI_BUDGET) {
        // Check if hardware has packet
        if (!eth_hw_has_packet()) break;
```

```
        // Allocate buffer from pool
        rx_buffer_t *buf = buffer_alloc(&drv->buffer_pool);
        if (!buf) {
            atomic_fetch_add(&drv->packets_dropped, 1);
            eth_hw_drop_packet();
            continue;
        }

        // Receive packet from hardware
        buf->len = eth_hw_receive(buf->data, sizeof(buf->data));

        // Enqueue to ring buffer
        if (!rx_ring_enqueue(&drv->rx_ring, buf)) {
            buffer_free(&drv->buffer_pool, buf);
            atomic_fetch_add(&drv->packets_dropped, 1);
        } else {
            atomic_fetch_add(&drv->packets_received, 1);
        }

        processed++;
    }

    // If we processed less than budget, re-enable interrupts
    if (processed < NAPI_BUDGET) {
        atomic_store(&drv->polling, false);
        eth_enable_interrupts();
    } else {
        // More work to do, reschedule
        schedule_softirq(eth_poll);
    }
}

// Consumer (kernel thread)
void eth_process_packets(void) {
    eth_driver_t *drv = &g_eth_driver;
    rx_buffer_t *batch[BATCH_SIZE];
    int count = 0;

    // Dequeue batch
    while (count < BATCH_SIZE) {
        rx_buffer_t *buf = rx_ring_dequeue(&drv->rx_ring);
        if (!buf) break;
        batch[count++] = buf;
    }
```

```
    // Process batch
    for (int i = 0; i < count; i++) {
        process_packet(batch[i]->data, batch[i]->len);
    }

    // Free batch
    for (int i = 0; i < count; i++) {
        buffer_free(&drv->buffer_pool, batch[i]);
    }
}
```

Final Benchmark

```
Test: 1 Gbps Ethernet, 64-byte packets (1.49M packets/sec)

Original (linked list, spinlock, interrupts):
  Throughput: 599 Mbps
  Packet loss: 31.1%
  CPU usage: 95%
  Cache misses: 18.5M/sec

Optimized (ring buffer, pool, NAPI, batching):
  Throughput: 1020 Mbps
  Packet loss: 0.12%
  CPU usage: 68%
  Cache misses: 2.8M/sec

Improvements:
  Throughput: 1.70× (599 → 1020 Mbps)
  Packet loss: 259× better (31.1% → 0.12%)
  CPU usage: 28% reduction
  Cache misses: 6.6× fewer
```

Summary

The packet loss mystery was solved. The network driver went from dropping 31% of packets to dropping only 0.12%—a 259× improvement. Throughput increased from 599 Mbps to 1020 Mbps, exceeding the 1 Gbps target.

Key insights:

1. **Lock-free ring buffers eliminate contention**. Single-producer single-consumer queues need only atomic loads/stores, no CAS. 5.3× faster than spinlock-based queues.

2. **Pre-allocated buffer pools are essential**. Allocating in interrupt context is 40× faster with a pool than with malloc. No risk of sleeping.
3. **Batch processing amortizes overhead**. Processing 32 packets at once is 1.6× faster than one-at-a-time. Better cache utilization and prefetching.
4. **NAPI-style polling beats interrupts at high load**. Adaptive interrupt mitigation provides 1.48× better throughput with 28% less CPU usage.
5. **Pointer arithmetic beats memcpy**. Linux's sk_buff uses headroom/tailroom to add/remove headers without copying. 37.5× faster than memcpy.

The numbers from the network driver: - Lock-free ring buffer: 5.3× faster, 6.6× fewer cache misses - Pre-allocated pool: 40× faster than malloc - Batch processing: 1.6× faster - NAPI polling: 1.48× throughput, 28% less CPU - Overall: 1.70× throughput, 259× less packet loss

Device drivers need lock-free, cache-friendly, pre-allocated data structures. Every cycle counts at high packet rates.

Next chapter: Firmware memory management—how to manage memory in resource-constrained embedded systems.

Chapter 19: Firmware Memory Management

Part V: Case Studies

> "Controlling complexity is the essence of computer programming."
> — Brian Kernighan

The Final Testing Phase

We were in the final testing phase of an IoT sensor project—a smart building device with 128 KB of RAM that monitored temperature, humidity, and air quality. The firmware had passed all functional tests. Unit tests: green. Integration tests: green. Power consumption: within spec.

The last requirement was a 72-hour continuous operation test. We set up twelve devices in the lab, configured them to report sensor data every second, and let them run.

After three days, I came into the lab expecting to collect the test logs and close the project.

Instead, I found all twelve devices had crashed.

The serial console showed the same error on every device:

```
[72:14:23] malloc failed: out of memory
[72:14:23] Fragmentation: 45%
[72:14:23] System halted
```

My stomach sank. The project was supposed to ship in two weeks. I knew exactly what had happened—and I knew it was going to be painful to fix.

The Textbook Approach

When I'd designed the firmware months earlier, I'd used what seemed like reasonable practices. The device needed to: - Handle network communication (TCP/IP stack) - Process sensor data every second - Store configuration - Perform OTA (Over-The-Air) updates

I used malloc/free from newlib, just like the textbooks teach:

```
void process_sensor_data(void) {
    // Allocate buffer for sensor reading
    sensor_data_t *data = malloc(sizeof(sensor_data_t));

    // Read sensors
    read_temperature(data);
    read_humidity(data);

    // Process and send
    send_to_cloud(data);

    // Free buffer
    free(data);
}
```

Simple. Clean. Textbook-correct.

And after 72 hours of continuous operation, it killed all twelve devices.

The Autopsy

I pulled the memory trace from the crash dump:

```
[72:14:23] malloc failed: out of memory
[72:14:23] Available: 8 KB
[72:14:23] Requested: 16 KB
[72:14:23] Fragmentation: 45%
[72:14:23] Total allocations: 259,200 (72 hours × 3600 seconds/hour)
[72:14:23] Average allocation size: 156 bytes
[72:14:23] System halted
```

45% fragmentation. Out of 128 KB of RAM, only 8 KB was available in contiguous blocks. The firmware needed 16 KB for a network buffer, but couldn't

find it.

The problem wasn't a memory leak—we were freeing everything correctly. The problem was **fragmentation**.

After 259,200 allocations and frees over 72 hours, the heap looked like Swiss cheese:

```
Initial state (128 KB free):
[                                                          ]

After 72 hours:
[used][free][used][free][used][free][used][free]...
      4KB       2KB       8KB       1KB

Total free: 58 KB
Largest contiguous: 8 KB
Can't allocate 16 KB!
```

I had two weeks before the scheduled ship date. I needed a solution that wouldn't require rewriting the entire firmware.

The Redesign: Finding a Path Forward

I couldn't rewrite the entire firmware in two weeks, but I could fix the memory management.

The key insight: **our allocations fell into predictable patterns**.

I analyzed the crash dump and found: - **Sensor data**: 156 bytes, allocated every second - **Network packets**: 1024 bytes, allocated every 5 seconds - **Configuration**: 2048 bytes, allocated once at startup - **Temporary buffers**: 256 bytes, allocated during processing

All predictable sizes. All predictable lifetimes.

I didn't need a general-purpose allocator. I needed **specialized allocators** for each use case.

Strategy 1: Fixed-Size Memory Pools

For the sensor data (156 bytes, allocated every second), I created a fixed-size pool:

```
#define SENSOR_POOL_SIZE 256  // Round up to power of 2
#define SENSOR_POOL_COUNT 10  // Max 10 concurrent readings

typedef struct free_block {
    struct free_block *next;
```

```
} free_block_t;

typedef struct {
    uint8_t memory[SENSOR_POOL_SIZE * SENSOR_POOL_COUNT];
    free_block_t *free_list;
} sensor_pool_t;

static sensor_pool_t g_sensor_pool;

void sensor_pool_init(void) {
    g_sensor_pool.free_list = NULL;

    // Link all blocks into free list
    for (int i = 0; i < SENSOR_POOL_COUNT; i++) {
        free_block_t *block = (free_block_t *)&g_sensor_pool.memory[i * SENSOR_POOL_SIZE];
        block->next = g_sensor_pool.free_list;
        g_sensor_pool.free_list = block;
    }
}

void *sensor_alloc(void) {
    if (!g_sensor_pool.free_list) {
        return NULL;  // Pool exhausted
    }

    void *ptr = g_sensor_pool.free_list;
    g_sensor_pool.free_list = g_sensor_pool.free_list->next;
    return ptr;
}

void sensor_free(void *ptr) {
    free_block_t *block = (free_block_t *)ptr;
    block->next = g_sensor_pool.free_list;
    g_sensor_pool.free_list = block;
}
```

Simple. **O(1) allocation and free.** Zero fragmentation.

I benchmarked it:

```
Test: 10,000 sensor readings (256-byte blocks)

malloc/free:
  Cycles: 2.4M (240 cycles per operation)
  Fragmentation: 18%
  Time: 2.0 ms
```

```
Fixed-size pool (free list):
  Cycles: 120K (12 cycles per operation)
  Fragmentation: 0%
  Time: 0.10 ms

Speedup: 20×
```

20× faster and zero fragmentation. That solved the sensor data problem.

Strategy 2: Static Allocation for Network Buffers

The network stack needed buffers for TCP/IP communication. The original code allocated them dynamically:

```
// BAD: Dynamic allocation
typedef struct {
    char *tx_buffer;
    char *rx_buffer;
    // ...
} uart_context_t;

void uart_init(uart_context_t *ctx) {
    ctx->tx_buffer = malloc(1024);  // Fragmentation!
    ctx->rx_buffer = malloc(1024);
}

// GOOD: Static allocation
typedef struct {
    char tx_buffer[1024];
    char rx_buffer[1024];
    // ...
} uart_context_t;

static uart_context_t g_uart_ctx;  // Static, no malloc

void uart_init(void) {
    // Buffers already allocated, nothing to do
}
```

Advantages: - **Zero fragmentation**: No heap usage - **Zero overhead**: No metadata - **Predictable**: Known at compile time - **Fast**: No allocation time

Trade-off: Uses RAM even when not needed. But for firmware, this is usually acceptable.

Solution 3: Stack-Based Allocation

For temporary buffers, use the stack:

```
// BAD: Heap allocation for temporary buffer
void process_data(void) {
    char *temp = malloc(512);
    // ... use temp
    free(temp);
}

// GOOD: Stack allocation
void process_data(void) {
    char temp[512];  // On stack
    // ... use temp
    // Automatically freed when function returns
}
```

Advantages: - **Fastest**: Just adjust stack pointer - **No fragmentation**: Stack grows/shrinks cleanly - **Automatic cleanup**: No need to free

Limitation: Stack size is limited (typically 4-16 KB). Don't allocate large buffers on stack.

Solution 4: Memory Regions

Partition memory into regions for different purposes:

```
// Memory layout (128 KB total)
#define REGION_STATIC_START   0x20000000
#define REGION_STATIC_SIZE    (64 * 1024)   // 64 KB for static data

#define REGION_POOL_START     (REGION_STATIC_START + REGION_STATIC_SIZE)
#define REGION_POOL_SIZE      (32 * 1024)   // 32 KB for pools

#define REGION_STACK_START    (REGION_POOL_START + REGION_POOL_SIZE)
#define REGION_STACK_SIZE     (16 * 1024)   // 16 KB for stack

#define REGION_DMA_START      (REGION_STACK_START + REGION_STACK_SIZE)
#define REGION_DMA_SIZE       (16 * 1024)   // 16 KB for DMA buffers

typedef struct {
    uint8_t static_data[REGION_STATIC_SIZE];
    uint8_t pool_memory[REGION_POOL_SIZE];
    uint8_t stack[REGION_STACK_SIZE];
    uint8_t dma_buffers[REGION_DMA_SIZE];
} memory_layout_t;
```

```
__attribute__((section(".ram")))
static memory_layout_t g_memory;
```

Why this helps: - **Clear boundaries**: Each region has fixed size - **No interference**: DMA doesn't corrupt stack - **Easy debugging**: Know which region is full - **Cache-friendly**: Related data in same region

---

Solution 5: Slab Allocator

For objects of the same type, use a slab allocator:

```
#define MAX_CONNECTIONS 32

typedef struct {
    int socket_fd;
    char rx_buffer[2048];
    char tx_buffer[2048];
    // ... other fields
} connection_t;

typedef struct {
    connection_t connections[MAX_CONNECTIONS];
    uint32_t free_bitmap;  // 1 bit per connection
} connection_pool_t;

static connection_pool_t g_conn_pool;

connection_t *conn_alloc(void) {
    // Find first free bit
    int idx = __builtin_ffs(g_conn_pool.free_bitmap) - 1;
    if (idx < 0) {
        return NULL;  // Pool exhausted
    }

    // Mark as used
    g_conn_pool.free_bitmap &= ~(1U << idx);

    // Return connection
    return &g_conn_pool.connections[idx];
}

void conn_free(connection_t *conn) {
    int idx = conn - g_conn_pool.connections;
    g_conn_pool.free_bitmap |= (1U << idx);
```

```
}
```

Advantages: - **O(1) allocation**: Just find first set bit - **Cache-friendly**: All connections contiguous - **Type-safe**: Can only allocate connection_t - **Low overhead**: 1 bit per object

The Benchmark

```
Test: 1000 connection allocations

malloc/free:
  Cycles: 240K
  Fragmentation: 12%
  Time: 0.20 ms

Slab allocator (bitmap):
  Cycles: 18K
  Fragmentation: 0%
  Time: 0.015 ms

Speedup: 13.3×
```

Real-World Example: FreeRTOS Heap Management

FreeRTOS offers multiple heap implementations:

heap_1.c: Simple Bump Allocator

```c
static uint8_t heap[configTOTAL_HEAP_SIZE];
static size_t next_free_byte = 0;

void *pvPortMalloc(size_t size) {
    void *ptr = NULL;

    if (next_free_byte + size < configTOTAL_HEAP_SIZE) {
        ptr = &heap[next_free_byte];
        next_free_byte += size;
    }

    return ptr;
}

void vPortFree(void *ptr) {
    // No-op: can't free individual blocks
}
```

Use case: Systems that never free memory (allocate at startup only).

heap_4.c: First-Fit with Coalescing

```
typedef struct A_BLOCK_LINK {
    struct A_BLOCK_LINK *pxNextFreeBlock;
    size_t xBlockSize;
} BlockLink_t;

static BlockLink_t xStart;
static BlockLink_t *pxEnd = NULL;

void *pvPortMalloc(size_t xWantedSize) {
    BlockLink_t *pxBlock, *pxPreviousBlock, *pxNewBlockLink;

    // Find first block large enough
    pxPreviousBlock = &xStart;
    pxBlock = xStart.pxNextFreeBlock;

    while ((pxBlock->xBlockSize < xWantedSize) && (pxBlock->pxNextFreeBlock != NULL)) {
        pxPreviousBlock = pxBlock;
        pxBlock = pxBlock->pxNextFreeBlock;
    }

    if (pxBlock != pxEnd) {
        // Split block if large enough
        // ...
        return (void *)(((uint8_t *)pxPreviousBlock->pxNextFreeBlock) + xHeapStructSize);
    }

    return NULL;
}
```

Use case: General-purpose allocation with some fragmentation tolerance.

heap_5.c: Multiple Regions

```
typedef struct HeapRegion {
    uint8_t *pucStartAddress;
    size_t xSizeInBytes;
} HeapRegion_t;

void vPortDefineHeapRegions(const HeapRegion_t * const pxHeapRegions) {
    // Initialize multiple non-contiguous memory regions
    // ...
}
```

Use case: Systems with multiple RAM regions (internal SRAM + external DRAM).

Putting It All Together: Optimized Firmware Memory

Here's the final optimized firmware combining all techniques:

```
// 1. Memory regions
#define STATIC_REGION_SIZE  (64 * 1024)
#define POOL_REGION_SIZE    (32 * 1024)
#define STACK_REGION_SIZE   (16 * 1024)
#define DMA_REGION_SIZE     (16 * 1024)

// 2. Fixed-size pools
typedef struct {
    fast_pool_t small_pool;   // 32-byte blocks
    fast_pool_t medium_pool;  // 256-byte blocks
    fast_pool_t large_pool;   // 4096-byte blocks
} pool_manager_t;

static pool_manager_t g_pools;

// 3. Static allocation for long-lived objects
typedef struct {
    char tx_buffer[1024];
    char rx_buffer[1024];
    // ...
} uart_context_t;

static uart_context_t g_uart;

// 4. Slab allocator for connections
typedef struct {
    connection_t connections[MAX_CONNECTIONS];
    uint32_t free_bitmap;
} connection_pool_t;

static connection_pool_t g_conn_pool;

// Memory initialization
void memory_init(void) {
    // Initialize pools
    pool_init(&g_pools.small_pool, 32, 128);
    pool_init(&g_pools.medium_pool, 256, 32);
    pool_init(&g_pools.large_pool, 4096, 8);
```

```
    // Initialize connection pool
    g_conn_pool.free_bitmap = 0xFFFFFFFF;  // All free

    // Static objects already initialized
}

// Smart allocation function
void *mem_alloc(size_t size) {
    if (size <= 32) {
        return pool_alloc(&g_pools.small_pool);
    } else if (size <= 256) {
        return pool_alloc(&g_pools.medium_pool);
    } else if (size <= 4096) {
        return pool_alloc(&g_pools.large_pool);
    } else {
        return NULL;  // Too large
    }
}

void mem_free(void *ptr, size_t size) {
    if (size <= 32) {
        pool_free(&g_pools.small_pool, ptr);
    } else if (size <= 256) {
        pool_free(&g_pools.medium_pool, ptr);
    } else if (size <= 4096) {
        pool_free(&g_pools.large_pool, ptr);
    }
}
```

Final Benchmark

```
Test: IoT firmware running for 24 hours

Original (malloc/free):
  Peak memory: 118 KB
  Fragmentation: 45%
  Largest free block: 8 KB
  Crashes: 3 (out of memory)
  Allocation time: 200 cycles (avg)

Optimized (pools + static + slab):
  Peak memory: 96 KB
  Fragmentation: 0%
  Largest free block: 32 KB
  Crashes: 0
```

```
  Allocation time: 12 cycles (avg)

Improvements:
  Memory usage: 18.6% reduction
  Fragmentation: 100% elimination
  Allocation speed: 16.7× faster
  Reliability: No crashes
```

The Retest

After implementing the memory pool redesign, I reflashed all twelve devices and restarted the 72-hour test.

This time, I monitored the memory usage continuously. After 15 minutes, the pattern was already clear: memory usage had stabilized at 96 KB, and fragmentation remained at zero.

After 72 hours, all twelve devices were still running. After a week, still running. We eventually let the test run for three months—no crashes, no memory issues, no fragmentation.

What I Learned

The firmware redesign taught me that **malloc/free is the wrong tool for embedded systems**.

Here's what worked:

1. Fixed-size pools eliminate fragmentation

Pre-allocating blocks of fixed sizes (256 bytes for sensors, 1024 bytes for network packets) provides O(1) allocation with zero fragmentation. The sensor pool was **20× faster** than malloc.

2. Static allocation for long-lived objects

Network buffers, configuration data, and other persistent objects should be statically allocated. Zero overhead, zero fragmentation, and the memory layout is known at compile time.

3. Stack allocation for temporary buffers

Short-lived buffers (like temporary processing buffers) should use the stack. Fastest allocation—just adjust the stack pointer—and automatic cleanup when the function returns.

4. Slab allocators for uniform objects

Connection pools and packet buffers benefit from slab allocation. Using a bitmap for free/used tracking provides O(1) allocation with just 1 bit of overhead per object. **13.3× faster** than malloc.

The Final Numbers

```
Original firmware (malloc/free):
  Peak memory: 118 KB
  Fragmentation: 45%
  Crashes after: 72 hours
  Allocation time: 240 cycles (avg)

Optimized firmware (pools + static + slab):
  Peak memory: 96 KB
  Fragmentation: 0%
  Crashes after: Never (ran for 3+ months)
  Allocation time: 12 cycles (avg)

Improvements:
  Memory usage: 18.6% reduction
  Fragmentation: 100% elimination
  Allocation speed: 20× faster
  Reliability: Zero crashes
```

The lesson: **firmware needs predictable, deterministic memory management**. Avoid malloc/free. Use pools, static allocation, and slab allocators.

And always run extended testing—72 hours minimum—before declaring a project complete. The bugs that matter most are the ones that only appear after days of continuous operation.

Summary

Key insights: - malloc/free causes fragmentation in long-running firmware - Fixed-size pools: 20× faster, zero fragmentation - Static allocation: Best for long-lived objects - Stack allocation: Best for temporary buffers - Slab allocators: 13.3× faster for uniform objects

The IoT sensor firmware: - 18.6% less memory (118 KB → 96 KB) - Zero fragmentation (45% → 0%) - 20× faster allocation (240 cycles → 12 cycles) - Zero crashes (ran for 3+ months continuously)

Takeaway: In embedded systems, predictability matters more than flexibility. Design your memory management for your specific workload, not for general-purpose use.

Chapter 20: Benchmark Case Studies

Part V: Case Studies

> "There are three kinds of lies: lies, damned lies, and benchmarks."
> — Adapted from Mark Twain

It was 2:00 AM when Sarah Chen, lead architect at a processor startup, received the email that would change her company's trajectory. A competitor had published a detailed technical analysis dismantling their flagship product's performance claims. The headline was brutal: "Marketing Hype vs. Reality: How Vendor X Inflated Benchmark Scores by 300%."

The problem wasn't that their processor was slow—it was actually quite good. The problem was the benchmark they'd chosen to showcase it: Dhrystone. Their competitor had shown, line by line, how modern compilers could optimize away most of Dhrystone's work, making the scores meaningless. Worse, they demonstrated that on *real* workloads—the kind customers actually run—the performance advantage evaporated.

Sarah spent the next week doing what she should have done months earlier: understanding what benchmarks actually measure. This chapter is the result of that investigation, examining two industry-standard benchmarks—Dhrystone and Coremark—to understand not just how to run them, but what they reveal about processor performance and, more importantly, what they hide.

20.1 Why Benchmarks Matter (and Why They Fail)

The Purpose of Benchmarks

In an ideal world, we'd measure processor performance by running every customer's actual workload. In reality, we need standardized tests that:

1. **Represent real work**: Reflect actual application behavior
2. **Are reproducible**: Give consistent results across runs
3. **Are portable**: Run on different architectures
4. **Are understandable**: Clearly show what's being measured

The challenge is that these goals often conflict. Make a benchmark too simple, and it doesn't represent real work. Make it too complex, and it's not reproducible or understandable.

How Benchmarks Fail

Benchmarks fail in predictable ways:

Compiler optimization: The compiler recognizes the benchmark pattern and optimizes it away. You're measuring the compiler's cleverness, not the processor's performance.

Narrow workload: The benchmark tests only one aspect of performance (e.g., integer arithmetic) while real applications use a mix of operations.

Unrealistic data: The benchmark uses small, cache-friendly datasets while real applications work with large, scattered data.

Gaming the benchmark: Vendors optimize specifically for the benchmark, not for real workloads.

Let's see how these failures manifest in practice.

20.2 Dhrystone: A Historical Lesson

Origins and Intent

Dhrystone was created in 1984 by Reinhold Weicker as a synthetic benchmark to measure integer performance. The name is a play on "Whetstone," an earlier floating-point benchmark.

Design goals: - Measure typical integer operations - Be small enough to fit in cache - Be simple to port - Avoid floating-point (many embedded processors lacked FPUs)

Workload composition (from the original paper): - 53% assignments - 32% control flow (if/else, loops) - 15% procedure calls - String operations - Record (struct) copying

What Dhrystone Actually Does

Let's look at the core of Dhrystone (simplified):

```
typedef struct record {
    struct record *ptr_comp;
    int discr;
    int enum_comp;
    int int_comp;
    char str_comp[31];
} Rec_Type, *Rec_Pointer;

void Proc_1(Rec_Pointer ptr_val_par) {
    Rec_Pointer next_record = ptr_val_par->ptr_comp;

    // Structure assignment
    *ptr_val_par->ptr_comp = *ptr_val_par;
```

```
    ptr_val_par->int_comp = 5;
    next_record->int_comp = ptr_val_par->int_comp;
    next_record->ptr_comp = ptr_val_par->ptr_comp;

    // Procedure call
    Proc_3(&next_record->ptr_comp);

    // Conditional
    if (next_record->discr == 0) {
        next_record->int_comp = 6;
        Proc_6(ptr_val_par->enum_comp, &next_record->enum_comp);
        next_record->ptr_comp = ptr_val_par->ptr_comp;
        Proc_7(next_record->int_comp, 10, &next_record->int_comp);
    } else {
        *ptr_val_par = *ptr_val_par->ptr_comp;
    }
}
```

String operations:

```
void Proc_2(int *int_par_ref) {
    int int_loc;
    char char_loc;

    int_loc = *int_par_ref + 10;

    do {
        if (Func_1('A', 'C') == 0) {
            char_loc = 'A';
            int_loc++;
        }
    } while (char_loc != 'A');

    *int_par_ref = int_loc;
}
```

The Fatal Flaws

Problem 1: Dead Code Elimination

Modern compilers can prove that much of Dhrystone's work has no observable effect:

```
// Compiler sees:
int x = 5;
x = x + 10;
x = x * 2;
```

```
// Result never used

// Compiler generates:
// (nothing - entire computation eliminated)
```

Problem 2: Constant Propagation

```
// Source code:
if (Func_1('A', 'C') == 0) {
    // ...
}

// Compiler knows 'A' and 'C' are constants
// Evaluates Func_1 at compile time
// Replaces entire if statement with constant branch
```

Problem 3: Unrealistic Data Access

Dhrystone's data fits entirely in L1 cache (a few KB). Real applications have cache misses. Dhrystone measures best-case performance, not typical performance.

Problem 4: No Pointer Chasing

While Dhrystone uses pointers, the access patterns are predictable. Modern processors prefetch the data before it's needed.

The Compiler Optimization Disaster

Here's what happens with `-O3` optimization:

```
$ gcc -O0 dhrystone.c -o dhry_O0
$ gcc -O3 dhrystone.c -o dhry_O3
$ ./dhry_O0
Dhrystones per second: 500,000

$ ./dhry_O3
Dhrystones per second: 5,000,000
```

10x speedup from compiler flags alone! You're not measuring the processor—you're measuring the compiler's ability to recognize and eliminate Dhrystone's patterns.

Different compilers give wildly different results: - GCC 10.2: 4.2 DMIPS/MHz
- Clang 12: 5.1 DMIPS/MHz
- ICC 21: 5.8 DMIPS/MHz

Same processor, different scores. The benchmark is broken.

What We Learn from Dhrystone

Dhrystone teaches us what *not* to do:

1. Don't use predictable, constant inputs
2. Don't allow dead code elimination
3. Don't use unrealistically small datasets
4. Don't focus on a single operation type

But it also teaches us what benchmarks *should* do—which brings us to Coremark.

20.3 Coremark: A Modern Approach

Design Philosophy

Coremark was created in 2009 by EEMBC (Embedded Microprocessor Benchmark Consortium) specifically to address Dhrystone's flaws.

Design goals: 1. Resist compiler optimization 2. Represent diverse real-world operations 3. Be portable across architectures 4. Have clear, enforceable run rules

The Four Workloads

Coremark consists of four distinct workloads, each testing different aspects of processor performance:

Workload 1: Linked List Operations

```
typedef struct list_data_s {
    int16_t data16;
    int16_t idx;
} list_data;

typedef struct list_head_s {
    struct list_head_s *next;
    struct list_data_s *info;
} list_head;

// Find element in list
list_head *core_list_find(list_head *list, list_data *info) {
    if (info->idx >= 0) {
        while (list && (list->info->idx != info->idx))
            list = list->next;
        return list;
    } else {
```

```
        while (list && ((list->info->data16 & 0xff) != info->data16))
            list = list->next;
        return list;
    }
}

// Reverse list
list_head *core_list_reverse(list_head *list) {
    list_head *next = NULL, *tmp;
    while (list) {
        tmp = list->next;
        list->next = next;
        next = list;
        list = tmp;
    }
    return next;
}
```

What it tests: - Pointer chasing (cache misses) - Unpredictable branches - List traversal patterns (Chapter 5)

Why it resists optimization: - List contents determined at runtime - Search criteria varies - Results are used (CRC'd at end)

Workload 2: Matrix Operations

```
typedef int16_t mat_elem;
typedef mat_elem *matrix_row;

// Matrix multiply (simplified)
void core_bench_matrix(mat_params *A, int16_t seed) {
    uint32_t N = A->N;
    matrix_row *C = A->C;
    matrix_row *A_mat = A->A;
    matrix_row *B = A->B;

    // C = A * B
    for (uint32_t i = 0; i < N; i++) {
        for (uint32_t j = 0; j < N; j++) {
            mat_elem temp = 0;
            for (uint32_t k = 0; k < N; k++) {
                temp += A_mat[i][k] * B[k][j];
            }
            C[i][j] = temp;
        }
    }
}
```

What it tests: - Arithmetic intensity - Cache blocking opportunities - Memory access patterns (Chapter 4)

Why it resists optimization: - Matrix size determined at runtime - Results verified with checksum - Multiple operations prevent constant folding

Workload 3: State Machine

```
enum CORE_STATE {
    CORE_START = 0,
    CORE_INVALID,
    CORE_S1,
    CORE_S2,
    CORE_INT,
    CORE_FLOAT,
    CORE_EXPONENT,
    CORE_SCIENTIFIC,
    NUM_CORE_STATES
};

// State machine for parsing numbers
enum CORE_STATE core_state_transition(uint8_t **instr, uint32_t *transition_count) {
    uint8_t *str = *instr;
    uint8_t ch;
    enum CORE_STATE state = CORE_START;

    for (; *str && state != CORE_INVALID; str++) {
        ch = *str;
        (*transition_count)++;

        switch (state) {
        case CORE_START:
            if (isdigit(ch)) {
                state = CORE_INT;
            } else if (ch == '+' || ch == '-') {
                state = CORE_S1;
            } else if (ch == '.') {
                state = CORE_FLOAT;
            } else {
                state = CORE_INVALID;
            }
            break;

        case CORE_S1:
            if (isdigit(ch)) {
                state = CORE_INT;
```

```
            } else if (ch == '.') {
                state = CORE_FLOAT;
            } else {
                state = CORE_INVALID;
            }
            break;

        case CORE_INT:
            if (ch == '.') {
                state = CORE_FLOAT;
            } else if (!isdigit(ch)) {
                state = CORE_INVALID;
            }
            break;

        // ... more states
        }
    }

    *instr = str;
    return state;
}
```

What it tests: - Branch prediction - Switch statement performance - String processing (Chapter 14)

Why it resists optimization: - Input strings vary - State transitions unpredictable - Transition count prevents elimination

Workload 4: CRC Calculation

```
uint16_t crcu16(uint16_t newval, uint16_t crc) {
    uint8_t i;

    for (i = 0; i < 16; i++) {
        if ((crc & 0x8000) != 0) {
            crc = (crc << 1) ^ 0x1021;
        } else {
            crc = crc << 1;
        }

        if ((newval & 0x8000) != 0) {
            crc ^= 0x1021;
        }

        newval = newval << 1;
    }
```

```
    return crc;
}

// CRC all results
uint16_t core_bench_crc(void *memblock, uint32_t size) {
    uint16_t crc = 0;
    uint8_t *data = (uint8_t *)memblock;

    for (uint32_t i = 0; i < size; i++) {
        crc = crcu16(data[i], crc);
    }

    return crc;
}
```

What it tests: - Bit manipulation - Loop optimization - Data-dependent operations (Chapter 13)

Why it resists optimization: - CRC depends on all previous data - Cannot be parallelized easily - Result must match known value

Preventing Compiler Optimization

Coremark uses several techniques to prevent dead code elimination:

1. Runtime-determined inputs:

```
// Not this (compiler can optimize):
int data[100] = {1, 2, 3, ...};

// But this (runtime-determined):
void init_data(int *data, int seed) {
    for (int i = 0; i < 100; i++) {
        data[i] = (seed * i) & 0xFF;
        seed = (seed * 1103515245 + 12345) & 0x7FFFFFFF;
    }
}
```

2. Result verification:

```
// All results are CRC'd
uint16_t final_crc = 0;
final_crc = crcu16(list_result, final_crc);
final_crc = crcu16(matrix_result, final_crc);
final_crc = crcu16(state_result, final_crc);

// Must match known value
if (final_crc != EXPECTED_CRC) {
```

```
    printf("ERROR: Invalid results!\n");
    return -1;
}
```

3. Volatile results:

```
// Prevent optimization of result storage
volatile uint16_t results[4];
results[0] = list_crc;
results[1] = matrix_crc;
results[2] = state_crc;
results[3] = crc_crc;
```

Run Rules

Coremark has strict run rules to ensure fair comparison:

1. **Minimum iterations**: Must run for at least 10 seconds
2. **No source modifications**: Core algorithms cannot be changed
3. **Validation**: Results must match known CRC values
4. **Reporting**: Must report iterations/second and iterations/MHz
5. **Compiler flags**: Must be disclosed

Example valid run:

```
CoreMark 1.0 : 12500.00 / GCC 10.2.0 -O3 -march=rv64gc / Heap
CoreMark/MHz: 5.00
```

20.4 Performance Analysis

Understanding the Scores

Dhrystone reports DMIPS (Dhrystone MIPS): - DMIPS = (Dhrystones/sec) / 1757 - 1757 is the score of a VAX 11/780 (the reference) - DMIPS/MHz normalizes for clock frequency

Coremark reports iterations/second: - Higher is better - CoreMark/MHz normalizes for clock frequency - Typical range: 2.5-5.5 CoreMark/MHz

What Affects Coremark Scores?

1. Compiler optimization:

```
# -O0 (no optimization)
CoreMark/MHz: 1.2

# -O2 (standard optimization)
CoreMark/MHz: 4.5
```

```
# -O3 (aggressive optimization)
CoreMark/MHz: 5.0

# -O3 -funroll-loops
CoreMark/MHz: 5.2
```

2. ISA extensions:

```
# RV64GC (base)
CoreMark/MHz: 4.8

# RV64GC + B extension (bit manipulation)
CoreMark/MHz: 5.1

# RV64GC + V extension (vector) - scalar mode
CoreMark/MHz: 5.0
```

3. Cache configuration:

```
16 KB I$ + 16 KB D$: 4.2 CoreMark/MHz
32 KB I$ + 32 KB D$: 4.8 CoreMark/MHz
64 KB I$ + 64 KB D$: 5.0 CoreMark/MHz
```

4. Memory latency:

```
SRAM (1 cycle):  5.2 CoreMark/MHz
DRAM (100 cycles): 3.8 CoreMark/MHz
```

Typical Scores (Public Data)

Based on EEMBC's published results and academic papers:

Embedded processors (RV32): - Simple in-order: 2.5-3.0 CoreMark/MHz - With caches: 3.0-3.5 CoreMark/MHz

Application processors (RV64): - In-order, single-issue: 3.5-4.0 CoreMark/MHz - In-order, dual-issue: 4.0-4.5 CoreMark/MHz - Out-of-order: 4.5-5.5 CoreMark/MHz

For comparison (x86/ARM): - ARM Cortex-A53: 3.5 CoreMark/MHz - ARM Cortex-A72: 4.5 CoreMark/MHz - Intel Atom: 4.0 CoreMark/MHz - Intel Core i7: 5.0+ CoreMark/MHz

What Coremark Doesn't Measure

Coremark is better than Dhrystone, but it's not perfect:

Missing workloads: - Floating-point operations - Vector/SIMD operations - System calls - I/O operations - Multi-threading

Unrealistic aspects: - Small dataset (fits in cache) - No OS overhead - No interrupts - Deterministic execution

What it measures well: - Integer arithmetic - Pointer chasing - Branch prediction - Compiler effectiveness - Cache performance (for small datasets)

20.5 Benchmark Design Principles

Lessons from History

Comparing Dhrystone and Coremark teaches us how to design good benchmarks:

Principle	Dhrystone	Coremark
Diverse workloads	Mostly assignments	4 distinct workloads
Resist optimization	Easily optimized	Multiple techniques
Runtime inputs	Compile-time constants	Seed-based generation
Result verification	Weak	CRC validation
Run rules	Informal	Strict, enforceable
Portability	Good	Excellent
Understandability	Simple	More complex

Designing Your Own Benchmark

When you need to create a benchmark for your specific use case:

1. Identify the workload:

```
// Don't benchmark generic "performance"
// Benchmark specific operations:

//  Too generic
void benchmark_processor(void);

//  Specific workload
void benchmark_packet_processing(void);
void benchmark_image_filtering(void);
void benchmark_crypto_operations(void);
```

2. Use realistic data:

```
//  Unrealistic
int data[100] = {1, 2, 3, 4, ...};  // Fits in cache

//  Realistic
#define DATA_SIZE (1024 * 1024)  // 1 MB
int *data = malloc(DATA_SIZE * sizeof(int));
init_random_data(data, DATA_SIZE, seed);
```

3. Prevent optimization:

```
//  Compiler can eliminate
int sum = 0;
for (int i = 0; i < n; i++) {
    sum += data[i];
}
// sum never used

//  Force computation
volatile int result;
int sum = 0;
for (int i = 0; i < n; i++) {
    sum += data[i];
}
result = sum;  // Volatile prevents elimination
```

4. Validate results:

```
//  Checksum validation
uint32_t expected_crc = compute_expected_crc(seed);
uint32_t actual_crc = run_benchmark(data, size);

if (actual_crc != expected_crc) {
    fprintf(stderr, "ERROR: Benchmark validation failed!\n");
    fprintf(stderr, "Expected: 0x%08x, Got: 0x%08x\n",
            expected_crc, actual_crc);
    return -1;
}
```

5. Report methodology:

```
printf("=== Benchmark Results ===\n");
printf("Workload: Packet processing\n");
printf("Data size: %d packets\n", num_packets);
printf("Iterations: %d\n", iterations);
printf("Compiler: %s %s\n", COMPILER_NAME, COMPILER_VERSION);
printf("Flags: %s\n", COMPILER_FLAGS);
printf("Time: %.2f ms\n", elapsed_ms);
printf("Throughput: %.2f Mpps\n", packets_per_sec / 1e6);
```

Common Pitfalls

Pitfall 1: Measuring the wrong thing:

```
//  Measures malloc, not computation
start_timer();
int *data = malloc(size);
compute(data, size);
```

```
free(data);
stop_timer();

//  Measures only computation
int *data = malloc(size);
start_timer();
compute(data, size);
stop_timer();
free(data);
```

Pitfall 2: Insufficient warm-up:

```
//  First run includes cold cache
for (int i = 0; i < 100; i++) {
    start_timer();
    benchmark();
    stop_timer();
}

//  Warm up first
for (int i = 0; i < 10; i++) {
    benchmark();  // Warm-up, don't measure
}
for (int i = 0; i < 100; i++) {
    start_timer();
    benchmark();
    stop_timer();
}
```

Pitfall 3: Ignoring variance:

```
//  Single measurement
double time = measure_once();
printf("Time: %.2f ms\n", time);

//  Statistical analysis
double times[100];
for (int i = 0; i < 100; i++) {
    times[i] = measure_once();
}
printf("Mean: %.2f ms\n", mean(times, 100));
printf("Median: %.2f ms\n", median(times, 100));
printf("Std dev: %.2f ms\n", stddev(times, 100));
printf("Min: %.2f ms\n", min(times, 100));
printf("Max: %.2f ms\n", max(times, 100));
```

20.6 Summary

Key Takeaways

Dhrystone is obsolete: - Modern compilers optimize away most of the work - Scores vary wildly between compilers - Doesn't represent real workloads - Use only for historical comparison

Coremark is better, but not perfect: - Resists compiler optimization through multiple techniques - Represents diverse integer workloads - Has strict, enforceable run rules - But: small dataset, no FP/SIMD, no OS overhead

Benchmark design principles: 1. Use diverse, realistic workloads 2. Prevent dead code elimination 3. Use runtime-determined inputs 4. Validate results 5. Report full methodology 6. Understand limitations

Benchmarks are tools, not goals: - A high Coremark score doesn't guarantee good performance on *your* workload - Understand what the benchmark measures - Supplement with application-specific benchmarks - Profile real applications

The Bigger Picture

This chapter examined two benchmarks in detail, but the lessons apply broadly:

From Chapter 3 (Benchmarking): Statistical rigor matters. Run multiple iterations, report variance, control for confounding factors.

From Chapter 2 (Memory Hierarchy): Cache behavior dominates performance. Benchmarks with unrealistic data access patterns (like Dhrystone) miss this.

From Chapters 5, 11, 13, 14: Real applications use diverse data structures. Good benchmarks (like Coremark) test multiple patterns.

Looking forward: As you design systems, remember that optimization targets matter. Optimizing for a benchmark is easy. Optimizing for real workloads—with their messy, unpredictable access patterns and diverse operations—is the real challenge.

Practical Advice

When evaluating processors: 1. Look beyond the headline number 2. Ask: "What benchmark? What compiler? What flags?" 3. Run your own workload if possible 4. Understand the benchmark's limitations

When designing benchmarks: 1. Start with real application traces 2. Identify the critical operations 3. Create a minimal reproducible test 4. Validate against the real application 5. Document everything

When reporting results: 1. Full disclosure: hardware, compiler, flags 2. Statistical analysis: mean, median, variance 3. Methodology: warm-up, iterations, validation 4. Limitations: what the benchmark doesn't measure

---

Sarah Chen's company learned these lessons the hard way. After the public embarrassment, they switched to Coremark and, more importantly, developed application-specific benchmarks based on actual customer workloads. Their next product launch focused not on benchmark scores, but on real-world performance improvements—and customers noticed.

The best benchmark is the one that matches your workload. Everything else is just a proxy.